Living Review: Carrying Code Review Comments Safely Across Resubmissions

Raisa Jose, Noel Alex Samuel, Khadheeja Safar, and Kalyanaraman P

***Abstract*—Review comments must follow revised code, and a changed location must not be mistaken for a resolved concern. We present Living Review, the resubmission model of a capstone review portal, and evaluate its safety layer: change labels, indentation and copy guards, a rule that keeps the reviewed quotation, and issue states that return an issue to its supervisor whenever an anchor changed or disappeared. The portal's four-tier content cascade is not a competitive locator: a contextual line tracker re-finds edited code more often, and on a human-annotated multi-language benchmark the region mapper CodeMapper handled 91.5% of regions correctly against 66.0%. Safety failures were rare but present: on real histories 1.3% of changed or deleted targets were not returned for re-check, all through a pre-existing identical copy, and on constructed anchors with an identical copy nearby the deployed cascade missed 89.7% of needed re-checks (20.7% with a copy guard). Three guards cut missed re-checks from 3.1% to 0.2% on held-out student-authored repositories. The layer does not depend on the locator. Over CodeMapper on a held-out stress seed it cut missed re-checks from 19.3% to 3.1% and silent misplacements from 7.0% to 3.7%, with placement unchanged. With the portal's own language model as the locator, the model's judgment of change left 41 of 1,930 needed re-checks unrequested on open-source histories, and the layer left 0. Both gains cost more requests on preserved code. Across ten resubmissions, keeping the reviewed quotation cut the changes since review that the latest request failed to signal from 563 to 8, the changes no hop had detected. At the default threshold the cascade relocated only 53.6% of edited real-history statements but reported 92.4% of deleted ones as removed, and safety-constrained thresholds did not transfer between project groups. A bounded model and scenario tests confirmed that no system event resolves an issue, but a deleted issue is not carried forward, and dismissal memory is a heuristic. Most rates rest on a syntax-tree reference audited only in-house, whose identification errors bound the deployed miss rate between 0.2% and 2.9%. No supervisor or student behavior was measured.**

***Index Terms*—code review, annotation anchoring, source code tracking, software evolution, language models**

I. Introduction

A review comment refers both to a location and to a concern. When the code is revised, the location may move while the concern remains, or the concern may be addressed while most of the quoted code survives. Review threads routinely outlive the lines they were written on. In the five open-source projects of our real-history study, 601 of 931 inline review threads on merged pull requests (64.6%; 95% pull-request cluster interval 57.9–70.8%) were marked outdated by GitHub, and 99.0% of the outdated threads could no longer be placed on any line (Section II-A). These counts describe professional projects and GitHub's diff-based outdating, not student work, but they show how often a reviewer must reconstruct where an earlier remark now applies.

The authors are with the School of Computer Science and Engineering, Vellore Institute of Technology, Vellore, India.

Living Review is the resubmission model of a capstone review portal in which faculty supervisors review a team's code and report over several versions. Two problems must be kept apart. The *technical* problem is to re-find each anchored piece of code in the next version and to report how it changed. The *educational* problem is whether carrying feedback forward helps supervisors re-review and helps students act on feedback; feedback-literacy research motivates retaining and revisiting comments [1], [2], but does not show that relocating annotations improves either. This paper addresses the technical problem and the issue-state rules that connect it to supervision. It measures no learning, comprehension, re-review time or faculty-workload outcome; those require a consented classroom study.

Content-bearing anchors, fuzzy matching, diff-based position tracking, syntax-aware differencing and region mapping across commits are established [3]–[8], and review tools already port or outdate comments [9], [10]. We therefore do not claim a new matching principle. The contribution is a *safety-oriented, version-aware feedback workflow*: relocation results become explicit evidence labels; the labels decide whether an issue returns to its supervisor; only the supervisor resolves an issue; and the evaluation measures, in the terms that matter for that workflow, how often the matcher hides a change or points the supervisor at the wrong code.

We fix the evaluation objective before reporting results. A wrongly placed issue misdirects a supervisor, whereas a missing location is visible and prompts a manual search. Outcomes are therefore ordered by consequence: (i) *missed re-checks*, changed or deleted code that is not returned to the supervisor, and *silent misplacements*, an issue shown on the wrong code without a re-check request, are the primary safety failures; (ii) *lost* anchors cost a manual search; (iii) *unnecessary re-checks* cost attention. Each is reported separately, with its denominator, and is never folded into a single accuracy figure without stated weights. We do not set a pass/fail limit: the acceptable rate is an institutional choice, so we report the trade-off frontier instead.

Research questions. RQ1 (location): how accurately is anchored code re-found on controlled edits, real Python revision histories and an independent human-annotated multi-language benchmark, compared with conventional review behaviors, a contextual line tracker, a syntax-aware differencer and a state-of-the-art region mapper? **RQ2** (safety): how often does carry-forward fail to return changed or deleted code for re-check, or misplace an issue silently, and which mechanisms cause these failures? **RQ3** (design and calibration): how do search scope,

tier order and similarity threshold affect accuracy and safety, and does a threshold chosen on some projects transfer to others? **RQ4** (stress and scale): how do duplicated, short, re-indented and compound-edited anchors, repeated resubmissions, multi-anchor issues and large repositories change these results, and what do targeted safety guards cost? **RQ5** (locator transfer): does the safety-oriented workflow improve the reliability of an already strong locator, and does keeping the reviewed quotation make identity transitions across many versions visible? **RQ6** (state contract): does the implemented issue lifecycle match its specification, in particular that no system action resolves an issue?

The paper makes four contributions. (1) A content-based code-anchor relocation cascade that distinguishes identical, content-matched, moved, changed and removed code (Section III), with three guards derived from its failure analysis, and evidence that the locator-independent part of this layer (the change labels, the indentation and copy guards and the issue-state policy), placed over the stronger locator CodeMapper, makes it safer without changing its placement (Section XII). (2) A human-controlled review-issue lifecycle, comprising carry-forward with lineage, dismissal memory and anchored media notes, specified as a transition relation and checked against its implementation (Sections III–IV), with an identity rule that keeps the reviewed quotation across versions (Section III-E). (3) An evaluation of relocation accuracy and safety on controlled transformations, real Python histories and external human-annotated data (Sections V–XI). (4) An analysis of the relocation–deletion trade-off and of cross-project threshold calibration, including the negative result that empirically safety-constrained thresholds did not transfer between project groups.

II. Background and Related Work

A. How Often Review Comments Outlive Their Lines

To quantify the problem in a public setting, we collected every merged pull request with at least one inline review thread from Requests, Flask, Click, HTTPX and Rich between January 2022 and June 2026, through GitHub's GraphQL interface [11]. Of 931 inline threads, 91.6% were followed by at least one further commit to the pull request, and 64.6% (pull-request cluster bootstrap 95% interval 57.9–70.8%) were marked outdated; 99.0% of the outdated threads no longer had a line in the current diff. GitHub marks a thread outdated when the diff hunk it was written on changes [10]; outdated threads remain readable, but the reviewer must find where the concern now applies. The sample is professional open source, the snapshot reflects force-pushes and squashes, and resolution does not show that a concern was fixed, so the figures motivate rather than measure the educational problem.

B. Robust Annotation Anchoring

Robust document annotation predates this system. Brush et al. examined annotation positioning after document edits [3]; Phelps and Wilensky developed robust intra-document locations [12]. Hypothesis describes quote-based fuzzy anchoring [13], and the W3C Web Annotation Data Model specifies text-quote and text-position selectors [4]. For code, recent systems keep annotations attached as programs evolve: Meta-Manager records the provenance of edits in the editor [14], and Magic Markup re-tags document-external markup with a language model [15]. These systems target a single developer's working copy and its edit stream; Living Review receives whole submitted versions without an edit stream and must decide, for each earlier concern, whether its supervisor needs to look again.

C. Code Review and Warning Tracking

Modern review combines line-specific discussion with human judgment [16], [17]. Review tools differ in how comments survive a revision, and a fixed-line model describes none of them fully. GitHub keeps a comment on its line while the line is unchanged and marks it outdated when the exact anchoring line changes [10]. Gerrit ports unresolved comments to a newer patch set by calculating their position in the new file, and attaches them at file level when no position can be calculated; ported comments look like ordinary ones, and resolved comments are not ported [9]. GitLab can be configured to resolve diff threads automatically when a push makes them outdated [18], the opposite of the human-only resolution adopted here. CodeReviewQA tests whether language models understand review comments by asking them to recognize the change type, localize the change and identify the solution, on 900 curated examples in nine languages [19]. It localizes the code a comment asks to change within one version, whereas Living Review must find where already-reviewed code went in the next. Static-analysis warning trackers retain issue identity across versions and remember earlier review decisions about warnings [20]–[23]; their findings are produced by the analyzer, so an issue that disappears from the analysis can be treated as fixed, whereas a supervisor's concern cannot. ViolationTracker builds precise violation histories with introduced, fixed, deleted and reopened states [24]; there too a violation is fixed when the analyzer stops reporting it. Whether a revision touched the reviewed code and whether the concern about that code still holds are different questions; a Living Review change label answers only the first, so it only requests a check. Table I compares documented behaviors. Apart from GitHub's outdated flag, which the suggestion oracle measures, we did not benchmark these products; our conventional baselines reimplement their documented policies, and product documentation supports the table, not performance claims.

D. Source Differencing and Region Mapping

Line differencing underlies many revision tools [25]. Our matcher uses Python's sequence matcher, which extends the Ratcliff–Obershelp approach [26], [27]. LHDiff combines line content and context to track source lines [5]; related work tracks changed lines [28], surveys program-element matching [29] and tracks source locations [7]. ChangeDistiller and GumTree provide syntax-aware differencing [6], [30]; a refactoring-aware syntax-tree differencer built on RefactoringMiner adds multi-mappings and commit-level diffs, with a benchmark of node mappings from bug-fixing and refactoring commits [31]. Origin analysis considers renamed, merged and split entities

TABLE I
DOCUMENTED BEHAVIOR OF REVIEW TOOLS AND CODE-MAPPING METHODS

Criterion	GitHub [10]	Gerrit [9]	GitLab [18]	CodeMapper [8]	LHDiff-style [5]	Living Review
Follows unchanged code shifted by edits above	yes	calculated position	not documented	yes	yes	yes
Follows code edited in place	no: marked outdated	calculated position or file level	no: outdated	yes	yes	yes, if similarity $\geq \theta$
Follows code moved to another file	not documented	not documented	not documented	via file renames	no	yes, exact text
Change status shown to reviewer	outdated	none (looks unchanged)	outdated	n/a (mapping only)	n/a	content-match, moved, changed, removed
Who resolves	people	people	people, or automatically when outdated (option)	n/a	n/a	supervisor only
Dismissal remembered on later versions	not documented	not documented	not documented	n/a	n/a	yes, by finding key
Published relocation accuracy	none found	none found	none found	yes	yes	this paper

Product cells summarize the cited documentation as accessed on 31 October 2026, not measured behavior. "Not documented": no statement found in the vendor's documentation. "None found": no published evaluation of the product's relocation accuracy was found in its documentation or in our related-work search. n/a: the method maps code regions and has no review state.

[32]. History trackers follow named program elements through a commit history with refactoring awareness: CodeShovel builds method histories [33], CodeTracker tracks methods and variables [34] and code blocks [35], and HistoryFinder revisits the oracles these tools were evaluated on [36]. They track syntactic elements, whereas a review anchor is an arbitrary range of lines. ReCite finds Linux-kernel comments whose referenced function was renamed or removed, traces the reference through the history and proposes a repair with a language model for people to accept [37]. The closest recent work is CodeMapper, which maps an arbitrary code region from one commit to another, language-independently, by combining several git diff algorithms, movement detection and text search with context-aware similarity selection [8]. ReviewAnchorBench turns bot-written review suggestions into executable mapping cases: a target is represented by its quoted lines and its proposed replacement, and located, ambiguous and deleted outcomes, as well as abstention, are scored against oracles [38]. It measures the recovery of one anchor across one revision; Living Review keeps an issue's state across many revisions and leaves every decision to the supervisor. Section XIV-E runs our locators on it. These methods answer *where is this code now*. None defines what a reviewer should be told about the change, or what happens to a concern when the answer is uncertain; that is the part Living Review adds, and Section VIII compares the matchers directly.

E. Issue Lifecycles

Issue trackers, review discussions and warning trackers all carry items across time with states such as open, resolved and closed. The relevant distinction is who may close an item and on what evidence. Analyzer-driven trackers may close an issue the analysis no longer reports; some review tools may close a thread when its lines change [18]. Living Review restricts every closing transition to the supervisor and lets relocation evidence move an issue only towards needs re-check. We do not claim that human-controlled resolution or persistent dismissal is new in isolation; the contribution is binding them to relocation labels with an explicit, checked transition relation (Section III).

F. Educational Feedback

Feedback research motivates specific, actionable comments [39], [40], and feedback literacy concerns how learners interpret and use them [1], [2]. Keuning et al. found that programming-feedback tools support diagnosing mistakes more than planning corrections [41]. This literature establishes that revisiting feedback matters; it does not establish that relocating annotations improves students' understanding or response to feedback, and we know of no study that does. That causal link is the open question this paper's technical results make testable, not one they answer.

III. DESIGN

A. Versions and Anchors

A submission version V_t contains files (π, T) and extracted report text. A code anchor is $a = (\pi, s, e, c, q)$: path, first and last line of an original span of height $k = e - s + 1$, whole-file SHA-256 digest and quoted text. Let $\bar{q}$ be the nonblank quoted lines with surrounding whitespace removed. The quotation is matching evidence, not a globally unique identifier of a program entity. An issue may hold up to twelve anchors. Report anchors store quoted text; media attachments can refer to an issue, a code range or the review. Their bytes are content-addressed. The original benchmark scores only code anchors and checks file/start location. Section VII-H separately evaluates AST source-line extents on surviving real-history targets.

B. Relocation

Given V_{t+1} and, when available, the previous text of π, relocation returns a new position and a label (Algorithm 1, Fig. 1). Tier 1 accepts an untouched file. Tier 2 searches $\bar{q}$ as a contiguous run of stripped non-blank lines in the same file and takes the occurrence nearest the old start. Tier 3 handles edits

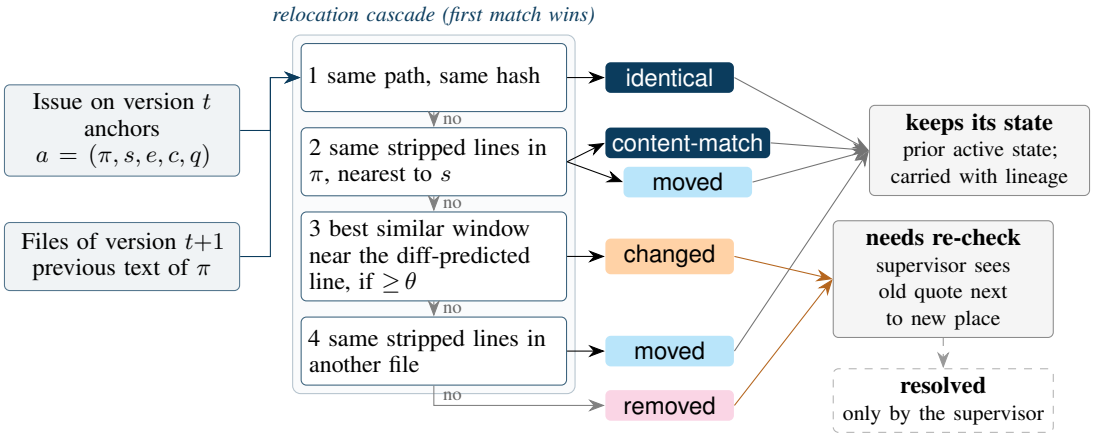


Fig. 1. Carry-forward of one issue. Each anchor is re-found by the first tier that matches; the issue needs a re-check if any anchor is changed or removed. No automated transition leads to resolved: only the supervisor closes an issue.

Algorithm 1 Relocate a code anchor $a = (\pi, s, e, c, q)$

Require: new files F (path $\mapsto$ (digest, text)); previous text P of π; threshold θ

if $\pi \in F$ **and** digest$(F[\pi]) = c$ **then return** $(\pi, s, e,$ identical$)$
end if
$\bar{q} \leftarrow$ stripped non-blank lines of q; $k \leftarrow e - s + 1$
if $\pi \in F$ **then**
  $R \leftarrow$ starts of runs equal to $\bar{q}$ in $F[\pi]$
  if $R \neq \emptyset$ **then**
    $s' \leftarrow \arg\min_{x \in R} |x - s|$
    return $(\pi, s', \cdot, s' = s$? content-match : moved$)$
  end if
  $p \leftarrow$ image of s under the line diff of P and $F[\pi]$; $r \leftarrow \max(10, 2k)$
  $(\rho, s') \leftarrow \max_{x \in [p-r,\, p+r]} \text{ratio}(\bar{q}, \text{window}(F[\pi], x, k))$
  if $\rho \geq \theta$ **then return** $(\pi, s', s' + k - 1,$ changed$)$
  end if
end if
for each other path π' in F, in sorted order **do**
  if $\bar{q}$ is a run in $F[\pi']$ at x **then return** $(\pi', x, \cdot,$ moved$)$
  end if
end for
return removed

in place. It maps the old start line through a line diff [27] of the previous and new file to an expected start p, scores every window of height k starting within $p \pm r$, $r = \max(10, 2k)$, by the similarity ratio of the joined stripped lines, and accepts the best window if the ratio reaches θ (deployed default 0.85). Tier 4 searches $\bar{q}$ in the other files, in sorted path order. Tier 3 runs before tier 4 so that an edited block is recognized in place rather than matched to an identical copy elsewhere, and its window stays near the diff-predicted line to avoid distant look-alike code; Section X measures both choices. A report anchor is matched in the same way against the sentences of the new report, with ratio threshold 0.8.

For normalized lines joined by a single space, let u and v be the resulting quotation and candidate strings. The fuzzy score is

$$\text{ratio}(u, v) = \frac{2 \sum_i m_i}{|u| + |v|}, \tag{1}$$

where m_i are the matching-block lengths returned by Python's sequence matcher with its automatic junk heuristic disabled. Only full-height windows are considered. Starts are scanned in ascending order and the first maximum is kept; exact-run distance ties likewise select the first run. If the previous file text is unavailable, the predicted start defaults to the old start. These deterministic details decide ambiguous matches and are part of the evaluated implementation.

TABLE II
RELOCATION LABELS: DEFINITION, LIMITS AND EFFECT ON THE ISSUE

Label	Holds when	Does *not* establish	Issue
identical	same path and same file digest (tier 1)	semantic review of the file	keeps state
content-match	normalized quotation $\bar{q}$ found at the old start (tier 2)	byte identity, equal indentation, equal behavior	keeps state
moved	$\bar{q}$ found at another start or in another file (tiers 2, 4)	that the match is the original copy	keeps state
changed	best near-predicted window has ratio $\geq \theta$ (tier 3)	that the concern is affected	re-check
removed	no tier succeeds	that the code was deleted	re-check

The deployed code emits a single identifier, `exact`, for both identical and content-match; this paper separates them because only the first is a byte-level property. Normalization strips each line's leading and trailing whitespace and skips blank lines.

C. Labels and Their Limits

Table II defines the five labels. The word *exact* in earlier versions of this work suggested more than the matcher establishes, so we use content-match for equality after normalization. Normalization is what lets an anchor survive blank-line and formatting edits, but in Python it also hides indentation, which carries meaning: a statement dedented out of a conditional keeps its stripped text and can keep its line number while its behavior changes. A content-match or moved label is therefore textual evidence, not semantic evidence. A second limit concerns copies. When the original is edited or deleted and an identical copy exists elsewhere, tiers 2 and 4 can return the copy as moved; because moved keeps the issue's state, the

change is not returned to the supervisor. Sections IX and XI measure both limits.

D. Proposed Safety Guards

The failure analysis of Section IX motivated three switches on the deployed cascade, and the compound-edit stress test a correction to one of them. They are evaluated alongside it under protocols whose text and code hashes were recorded before any held-out run (Section V); the deployed cascade remains the system of record.

- **Indentation guard (IG).** When a normalized run is accepted at tier 2 or 4, the leading-whitespace widths of its non-blank lines (tabs expanded to eight columns) are compared with the quotation's, both relative to the first line and in absolute base indentation. For Python files any difference turns the label into changed with reason *layout*; for other languages only a relative difference does. Placement is unchanged.
- **Pre-existing-copy guard (AG).** At tier 2 a run that starts exactly at the diff-predicted line is preferred over the run nearest the old start. When a tier-2 or tier-4 run is accepted anywhere other than the predicted line, the guard counts occurrences of $\bar{q}$ in the old and new versions of the candidate file. A genuine move introduces the text, so only an increased count yields a clean moved; otherwise the placement is kept but re-check is requested with reason *pre-existing copy*. The previous version's files are already loaded by carry-forward, so the guard needs no new input.
- **Insertion fix (AGi).** AG as first specified flagged every clean insertion directly above an anchor: the pinned diff mapping sends the old start of such an anchor to the first inserted line, so the unmoved anchor never sits at its predicted start (Section XI-E). AGi is AG with one change: a run at the line to which the diff maps the anchor's own first line also counts as the predicted start. It needs only the returned window and the two versions of the file, so it applies to the output of any locator (Section XII).
- **Variable-height window (VW).** Tier 3 also scores windows of heights $h \in [\max(1, k-d),\, k+d]$, $d = \max(1, \lceil k/2 \rceil)$, so that statements that grow or shrink can be recovered with their new extent. Ties prefer the predicted start, then the original height.

None of the guards proves identity. Two copies introduced in the same revision defeat AG, a copy inserted just below the original can make the diff align the original's lines with the copy and so defeat AGi, and IG is a layout signal, not an equivalence test. We also tested a provenance rule (AG2) that accepts a window by where its lines came from in the diff instead of by copy counts. It removed more unnecessary requests but could not tell a move from an edited original with an identical copy elsewhere, and on the held-out duplicate suite it missed nearly as many re-checks as no guard at all (Section XII). Content alone cannot decide which of two identical copies is the reviewed one; it can only decide when to ask.

E. Keeping the Reviewed Text

When tier 3 accepts a changed window, carry-forward replaces the anchor's quotation with the matched text, and the next version is matched against that text. A wrong fuzzy match therefore becomes the reference for every later version, and a later content-match label then certifies only that the wrong code did not change again. We evaluate an identity rule (ID) that keeps the quotation the supervisor reviewed, q_0, unchanged beside the current one, q_t. Relocation still tracks q_t, but two checks use q_0. Every changed hop must also reach similarity 0.5 to q_0 (the calibrated threshold, Section X); otherwise the transition is uncertain, the anchor is reported removed and its issue returns, so the anchor cannot drift step by step into unrelated code. And the request is computed against q_0: an issue returns for re-check whenever the located code differs from what the supervisor reviewed, however many versions ago that was. The interface would show q_0 beside q_t. A supervisor's confirmation sets a new q_0; the replay in Section XI-H assumes no supervisor action. ID needs no particular locator and is evaluated over both the cascade and CodeMapper.

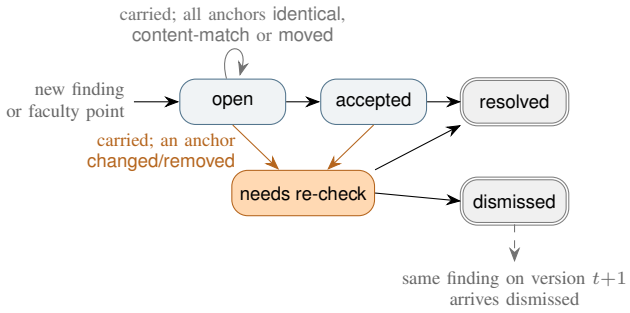


Fig. 2. Lifecycle of a review issue across versions. Black arrows are faculty actions, orange arrows are carry-forward. Carry-forward moves an issue to needs re-check when any anchor is changed or removed, and otherwise keeps its state. Supervisors may set any state from any state; no system or assistant action resolves an issue. A new finding matching an issue dismissed on the preceding version is created dismissed, and a deleted issue is not carried (Section XIII).

F. Complexity

Let the anchored file have L lines, the version N lines in F files, the anchor k lines and its joined quotation q characters. Tier 1 compares stored digests in $O(1)$. Tier 2 normalizes the file and scans for runs in $O(L+ck)$, where c counts lines equal to the first quoted line, so $O(Lk)$ in the worst case. Tier 3 first maps the old start through a line diff, quadratic in L in the worst case for Python's sequence matcher [27], and then scores $2r+1$ windows at $O(q^2)$ each, giving $O(L^2 + rq^2)$ with $r = \max(10, 2k)$; VW multiplies the window term by $2d+1$. Tier 4 normalizes and scans every other file, $O(Nk)$. One anchor therefore costs $O(L^2 + rq^2 + Nk)$ in the worst case, and a review with A anchors $O\big(A(L^2 + rq^2 + Nk)\big)$, because the implementation neither caches normalized files nor indexes lines across anchors. The ANk term dominates for anchors whose code has disappeared from a large repository: every other file is scanned. A line index keyed by the first normalized line would reduce tier 4 to the cost of verifying candidate runs. Section XI-I measures these costs on large repositories.

G. Issue States

When a new version arrives, active issues (open, accepted or needs re-check) are copied into its review with relocated anchors and a lineage reference. Any changed or removed anchor requests re-check; otherwise the previous active state is retained. No relocation transition resolves an issue. A finding raised again is matched to a carried issue by category, title and overlapping evidence, so that a carried concern is not duplicated; the carried issue takes the fresh evidence, an active one returns for re-check, and a resolved or dismissed one keeps its state. A newly raised finding is also compared with the findings dismissed on the immediately preceding version, by category and the first paragraph of its description (first 200 normalized characters); a match is created already dismissed, with a note naming that version. The key is a heuristic: findings that share an opening paragraph are conflated, rewording evades it, and the memory lapses if a version does not raise the finding (Section XIII). Fig. 2 summarizes the lifecycle and Table XXVI specifies it.

H. Media Notes and Release

Notes attached to carried issues follow their issue; notes pinned to code are re-found by the cascade. A note whose code is gone is kept on the review as a whole and records where it was ("it was on *path:lines*"). At publication, notes on issues the supervisor shares are listed in each student's immutable feedback snapshot, and the file route serves a note only to students whose snapshot lists it.

IV. System Architecture

Living Review is the resubmission model of the review page of a capstone review portal used by faculty supervisors and student teams. It is not a separate service: the cascade, the issue states and the media-note rules run inside the portal's review composition, against the same database and file store as the rest of the review. This section describes the implemented architecture (Fig. 3), the data the method reads and writes (Fig. 4), how a review is composed for a new version (Algorithm 2), and the integrity properties that the evaluation does not measure but the design depends on.

A. Deployment

The portal runs as a small set of containers. A web container is the single front door: Caddy serves the static web client used by faculty and students and proxies API calls. The api container runs the FastAPI application, which owns authentication, authorization on every protected route, validation and all durable state. A worker and a scheduler run Celery tasks on a Redis queue: analysis of a submission (rule checks, a language-model review of the submission and the partner claim-to-code tracer, whose results arrive as signed bundles) and the composition of the review. PostgreSQL holds versions, review issues, anchors, revisions, the change journal and publications; schema changes are forward-only migrations. Submitted archives, extracted files and media notes are kept in an artifact store addressed by their SHA-256 digest. An optional assistant runs as its own service and edits review points only through journaled, undoable tool calls (Section XIII). Relocation needs no extra component: its matching core uses Python's standard-library sequence matcher [27].

B. Data Model

A report version records its predecessor, so the versions of a project form a chain (Fig. 4). The files of a version are rows with a safe path, a SHA-256 digest and a text flag; their bytes stay in the artifact store, and only text files below a size limit are read for relocation. Each version has one review document, which holds the supervisor's review: issues, a summary, stated limitations and a carry record (from which version issues were carried, how many need a re-check, how many dismissals were remembered). An issue has a status among open, accepted, needs re-check, resolved and dismissed, a severity, a category, a title and body, and up to twelve anchors stored as JSON in its evidence. A carried issue's origin reference has the form `carry:v`t`:id`; it records the issue's lineage and makes composition idempotent. Every edit of an issue appends an attributed revision, and a write must name the revision it was based on, so concurrent edits are detected rather than lost. Media notes are attachment rows that point to bytes stored once under their digest; a note is anchored to an issue, to a code range or to the whole review, and is private or shared. A per-version change journal records every write by faculty, the system or the assistant, and supports undo. At release, the approved shared points are frozen into a packet per student.

C. Review Composition

When a version arrives, the worker analyses it and the review is composed (Algorithm 2). Composition first locks the version's row, because the faculty home page, open project tabs and the worker may all compose the same version at once. Carry-forward comes first, so that a finding the new analysis raises again is matched to the issue the supervisor already has rather than duplicated. Carry-forward reads only the immediately preceding version's review; if that review was never composed, its issues reach the next version only after it is composed and the later version recomposed. If the new finding has the same category and title as a carried issue and an overlapping anchor, the carried issue keeps the supervisor's wording and takes the fresh evidence; an active issue asks for a re-check, and a resolved or dismissed one keeps its state. If it matches an issue the supervisor dismissed on the previous version, it is created already dismissed, with a note naming that version. Media notes follow: a note attached to a carried issue moves with it; a note pinned to code is re-found by the cascade and, when its code is gone, stays on the whole review with a note of where it was. A copy is a new row that points to the same stored bytes. Finally the carry counts are recorded, and the review page shows them as a banner ("N points carried over $\cdot$ M need a re-check"), with a "From version n" tag on each carried issue and a plain-language note on each anchor.

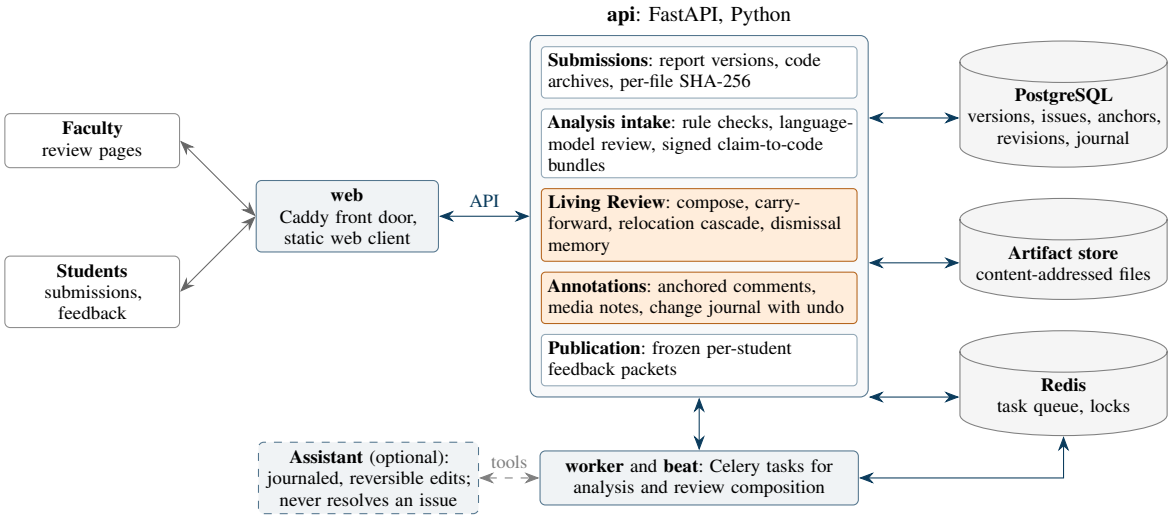


Fig. 3. Architecture of the review portal. Faculty and students reach the portal through one front door. Caddy serves the static client and the api container provides protected application routes; components that implement Living Review are shaded orange. Durable state lives in PostgreSQL and a content-addressed artifact store; analysis and review composition also run as queued worker tasks. The optional assistant can edit a draft only through journaled, reversible tool calls.

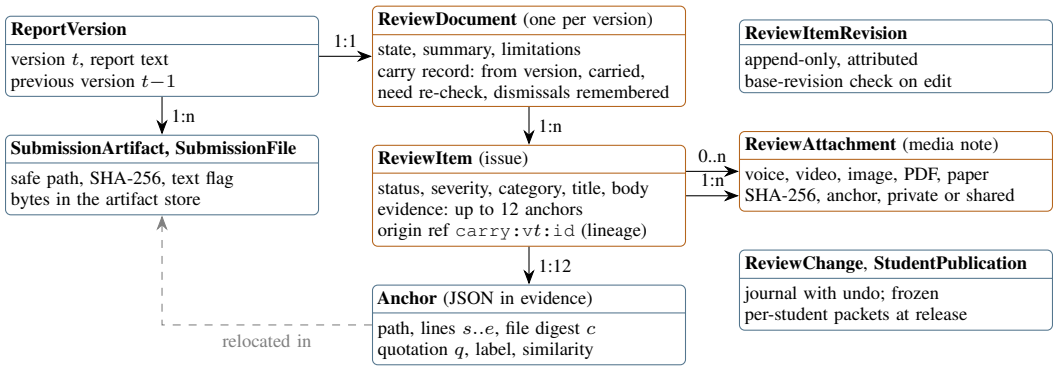


Fig. 4. Data model behind Living Review (simplified; names follow the portal's tables). Orange entities are written by carry-forward. A carried issue is a new row in the next version's review whose origin reference names the version and issue it came from; its anchors are re-found in that version's files.

D. Integrity Properties and Their Evidence

The architecture implements more than the experiments evaluate. Table III separates the two: each property is listed with its mechanism and the kind of evidence this paper provides for it, namely a measurement, a pass/fail scenario test against the pinned implementation, or a description of the design only.

An optional assistant can create, edit and delete review points through journaled, undoable tool calls, but cannot publish, and a resolve or dismiss it suggests takes effect only when a supervisor applies it. Its deletions, like a supervisor's, stop an issue from being carried forward (Section XIII). It plays no part in relocation, and its usefulness to supervisors is not evaluated.

The review studio, the supervisor's review page, shows an anchor with the deployed `exact` label as "Unchanged in this version", although that label also covers content-match anchors whose whitespace or line endings changed (Table II). Its "Compare with version n" panel shows the immediately preceding version only; the original quotation of a long-lived issue survives in its stored lineage.

E. Implementation

The portal uses Python, FastAPI, SQLAlchemy and PostgreSQL. The evaluated relocation and carry-forward functions come from a pinned source snapshot, identified by commit in the artifact; later application changes must not be assumed to share these results.

V. Evaluation Method

A. Corpus

We used every Python file of 60–1,500 lines in the backend of the review portal and in a partner team's claim-to-code tracer, exported from fixed commits so that the corpus does not drift with development: 117 files and 32,446 lines. The code is real, maintained application code, not generated text. Real-history projects are described in Section VII; the human-annotated external benchmark and the student-authored repositories in Section VIII.

B. Trials and Ground Truth

A trial picks a file and a 1–10-line anchor. It starts on a nonblank line, contains at least 30 non-space characters,

Algorithm 2 Compose the review of version $t{+}1$

Require: version $t{+}1$ with files F and report text; review D_t of version t; new findings Φ

lock version $t{+}1$; $D \leftarrow$ its review document (created if absent)
$K \leftarrow$ keys (category, first description paragraph) of issues dismissed in D_t
for each issue $i \in D_t$ that is open, accepted or needs re-check **do**
    if D already holds an issue with origin `carry:`vt`:`i **then continue**
    end if
    $A \leftarrow$ anchors of i re-found in F (Algorithm 1) or in the report text
    $s \leftarrow$ needs re-check **if** any anchor in A is changed or removed **else** status of i
    add a copy of i to D with anchors A, status s and origin `carry:`vt`:`i
end for
for each finding $f \in \Phi$ **do**
    if a carried issue has f's category and title and an overlapping anchor **then**
        refresh its anchors; if it is active, set it to needs re-check
    else if the key of f is in K **then**
        add f as dismissed, naming the version where it was dismissed
    else
        add f as open
    end if
end for
carry media notes: with their carried issue, or re-found by Algorithm 1
record the carry counts for the banner; mark D ready

has distinct lines and has a quotation unique in the source file. One of twelve edit operators supplies a reference start location or a gone target (Table IV). Inserted filler is copied from other corpus files. Up to twenty-five additional sampled files accompany the edited version. There are 300 trials per operator and 3,600 per seed. The main and replication seeds are 20261018 and 20261027. A separate tuning seed, 7, supplies 150 trials per operator. The default threshold was fixed for the reported main runs; no externally registered protocol is documented here. These are controlled text mutations of real source, not necessarily valid or behavior-preserving Python programs.

C. Baselines

The comparators are implemented policies, not executions of commercial review products. *Fixed line* retains the original path and span and returns no match if the normalized quotation differs there. *Diff drift* follows equal line-diff blocks but returns no match if an anchored line changes or the range ceases to be contiguous. *Exact quote* searches the same file for the nearest stripped quotation. These three policies do not attempt fuzzy recovery of edited quotations. *Cascade v1*, an earlier internal version, is a stronger comparator than these three policies: it searches other paths before fuzzy matching, searches the whole original file and uses $\theta = 0.75$. It differs from the final default in three respects, so its default comparison is compound rather than a single-factor ablation. Equal-threshold comparisons separate out threshold choice but still change order and scope together. The original LHDiff binary is not available to us. GumTree's official release and the pinned CodeMapper artifact [8] are evaluated on the external benchmark (Section VIII). A separately pinned third-party hybrid tracker is evaluated on controlled trials in Section VI-D and retained real-history observations in Section VII-G.

TABLE III
INTEGRITY PROPERTIES, MECHANISMS AND EVIDENCE

Property	Mechanism	Evidence here
No automatic resolution	no system or assistant code path writes resolved; relocation can only request re-check; the only system-written closed state is a new finding created dismissed by dismissal memory	scan of 26 status writers; bounded model check (Sec. XIII)
Idempotent composition	carried issues are recognized by their lineage reference; composition never overwrites an existing issue	repeated-composition scenario
Concurrency	the version row is locked during composition; every edit names its base revision, though a restore does not	PostgreSQL concurrent-composition scenario
Dismissal memory	a new finding whose category and opening description match an issue dismissed on the preceding version is created dismissed, with a note naming that version	collision scenarios; heuristic key, limits in Sec. XIII
Media notes follow code	notes on carried issues move with them; notes pinned to code are re-found by the cascade and become document-level notes, with an explanation, when the code is removed	three-version scenario
Attribution and undo	every change is an attributed, append-only revision; undo appends a restoring entry	design only
Verified quotations	an anchor is created only after its quotation is checked against the stored version	design only
Private and shared feedback	released wording is frozen per student; a student reaches source only through an anchor in their own packet	design only

"Design only": implemented and covered by the portal's own integration tests, but not assessed in this paper. No security assessment, penetration test or formal access-control verification was performed; the access controls are implemented design controls, not established security guarantees.

TABLE IV
EDIT TYPES AND GROUND TRUTH

Edit	What happens	Correct outcome
no change	file untouched	same lines
edit elsewhere	one distant line edited	same lines
insert / delete above	1–40 / 1–15 lines	shifted lines
re-indent	anchor indented 4 spaces	same lines
move in file	block moved >30 lines	new location
move to other file	block moved to file π'	new file, location
duplicate above	copy inserted ≥ 40 above	shifted original
token edit	one identifier renamed	same lines
insert + token edit	both	shifted lines
rewrite	replaced by other code	reported gone
delete	block deleted	reported gone

The first eight preserve the anchored content; the last four change it.

D. Outcomes and Metrics

A location is correct if its file and starting line equal the reference, or if it returns no match when the reference target is gone. It is lost if the target survives but no location is accepted, and wrong if it is placed elsewhere. The original scoring function does not check the returned end line, range overlap or continued relevance of the review concern; the added range analysis in Section VII-H addresses the first two on a defined subset. The benchmark's *silent error* means a returned location labeled unchanged under its fixed-height normalized-quote definition when the quote changed or the location is wrong; for the cascade this corresponds to an erroneous content-match label (deployed identifier `exact`). A moved label can still fail to trigger re-check, so Section IX reports missed re-checks and silent misplacements as separate rates. Synthetic proportions use nominal 95% Wilson score intervals [42].

For paired correctness, b_{01} counts trials correct only for the final cascade and b_{10} counts trials correct only for the comparator. McNemar's framework compares the discordant pairs [43]. Conditional on $d = b_{01} + b_{10}$, the null assigns each direction probability $1/2$. The archived implementation computes the two-sided exact binomial tail

$$p = \min\left\{1,\ 2\sum_{j=0}^{\min(b_{01},b_{10})}\binom{d}{j}2^{-d}\right\}, \tag{2}$$

with $p = 1$ when $d = 0$. This specifies the calculation reported in Table VI. These nominal trial-level calculations do not model source-file or project dependence. Runtime is measured per relocation on one laptop core, not per complete review.

E. Scoring Placement and Classification Separately

Earlier versions scored one outcome per trial, correct, wrong or lost, which mixes two questions. We now score *placement* (file and start line on targets that survive) and *classification* (the label against the reference class: preserved in place, shifted or moved to another file; edited; gone) separately, and derive the safety outcomes of Section IX from both. A placement can be right while its change status is wrong, and the converse. Wrong placement and lost anchors are reported separately throughout, because a wrong location misdirects the supervisor while a missing one is visible. Table V gives every reported rate with its numerator and denominator; the denominators differ by design, so rates with different names are not comparable.

F. Dependence-Aware Uncertainty

Trials share source files, and real-history observations share commit pairs and projects. Besides nominal Wilson intervals, we report cluster-bootstrap 95% intervals from 10,000 fixed-seed resamples of source files (controlled), commit pairs and, separately, projects (real histories), for headline accuracies and every paired difference, together with the sign of each per-project difference. A conclusion is stated as directional only if it survives the project-level interval. Every percentage is rounded independently from its raw counts, so the parts of a partition may sum to 99.9 or 100.1.

TABLE V
REPORTED RATES AND THEIR DENOMINATORS

Rate (column names)	Counts	Out of
Correct outcome (correct, Acc)	surviving targets placed at the reference file and start, and gone targets reported gone	all targets
Placement (Place)	targets placed at the reference file and start	surviving targets
Classification (Class)	targets whose label agrees with the reference class	all targets
Missed re-check (Miss)	targets labeled identical, content-match or moved	edited or gone targets
False re-check (Extra)	targets labeled changed or removed	content-preserved targets
Silent misplacement (Silent)	targets placed at a wrong location without a re-check request	all targets
Lost	targets for which no location is returned	surviving targets

Section IX gives the formal definitions. Correct outcome and placement differ on the same data whenever targets are gone: on the human-annotated regions the deployed cascade's correct outcome is 66.0% and its placement 62.2%.

G. Protocols and Held-Out Evidence

Every new analysis in this revision followed a written protocol whose text and executable-code hashes were recorded, with a timestamp, before the analysis it governs; these are local records, not external preregistrations. Before any new result was produced, a reproduction gate re-ran the archived methods and required per-observation agreement with all 7,200 archived controlled trials and all 15,064 real-history observations, including the 143 execution failures of the external tracker; the gate passed. The real-history observations had been inspected in earlier revisions, and the safety guards were designed after examining the real-history missed re-checks, so all guard results on these histories are retrospective. The held-out evaluations of the guards are the stress suites generated after their code was fixed (Section XI), the external human-annotated benchmark (Section VIII) and the student-authored repositories (Section VIII-B).

H. Calibration Objectives

Threshold calibration previously maximized the number of correct outcomes. Because the stated objective ranks safety failures first, we also report safety-constrained calibration, which maximizes accuracy subject to limits on the missed re-check rate and on the rate at which deleted code is reported present, and leave-one-project-out calibration over all nine projects (Section X). The oracle-excluded observations cannot enter any calibration without independent labels, so every threshold is calibrated on the oracle's decidable subset only.

VI. RQ1: Location on Controlled Edits

A. Start-Location Accuracy

The cascade reached a correct outcome on 98.5% of trials (95% CI 98.0–98.8; Table VI); its placement on surviving anchors, the Place column of Table XVI, was 98.8%. Exact-quote search reached 74.9%, diff drift 58.3% and the fixed-line policy 41.7%. The reported paired tests detect differences on

TABLE VI
OVERALL RESULTS ON 3,600 TRIALS (SEED 20261018)

Method	Correct % [95% CI] seed 20261018	Wrong lines %	Lost %	Paired vs. ours b_{01}/b_{10}, p	Replication seed 20261027	ms (median / p95)
Cascade (ours)	98.5 [98.0, 98.8]	1.5	0.1	—	98.6	0.08 / 65.1
Cascade v1 (internal)	96.4 [95.7, 96.9]	3.6	0.0	84/8, $< 10^{-16}$	97.1	0.07 / 498.4
Exact quote	74.9 [73.4, 76.3]	0.3	24.9	868/18, $< 10^{-16}$	74.9	0.05 / 0.2
Diff drift	58.3 [56.7, 59.9]	0.0	41.7	1465/20, $< 10^{-16}$	58.3	0.26 / 1.8
Fixed line	41.7 [40.1, 43.3]	0.0	58.3	2065/20, $< 10^{-16}$	41.7	0.02 / 0.1

The paired column gives the trials only the cascade got right (b_{01}) and only the other method got right (b_{10}), with the two-sided exact McNemar p. The replication column repeats the whole benchmark with seed 20261027.

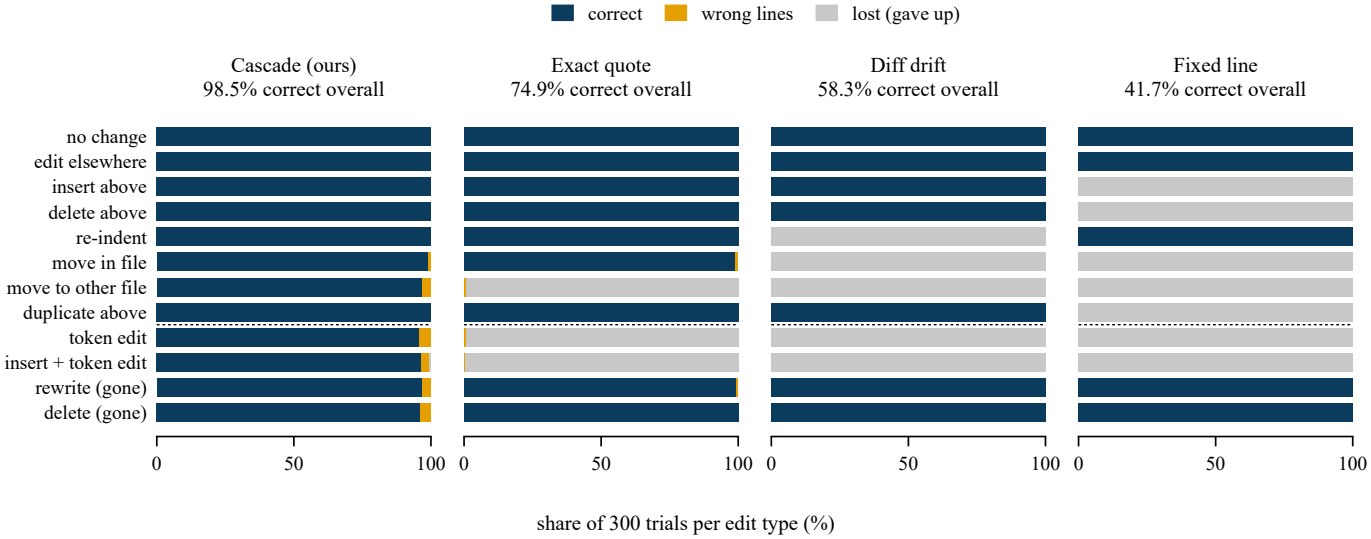


Fig. 5. Outcome per edit type for each method (300 trials per bar, seed 20261018; dashed line separates edits that preserve the anchored content from edits that change it). Baselines fail by giving up (grey): the fixed-line policy loses every shifted anchor, diff drift every anchor whose lines were touched (including a pure re-indent), exact quotes every edited anchor and every anchor moved to another file. The cascade's failures are mostly wrong placements (orange); an erroneous moved label can still miss workflow re-check (Section IX).

these trials: exact quotes were right on only 18 trials where the cascade was not, against 868 the other way. Fig. 5 and Table XXXII show where the gap comes from. The baselines seldom place an anchor on wrong lines; they fail by giving up. These losses describe the implemented policies and do not estimate the behavior of a complete review product.

The fixed-line and diff-drift policies succeed on exactly five and seven controlled edit types, respectively, explaining their unchanged scores across seeds. Exact-quote outcomes can also depend on quotation ambiguity. The cascade's 1.5% wrong placements fall in three groups. For code moved to another file (3.0%), a similar window near the old place is taken as an in-place edit. For token edits (4.3%) and insert-plus-edit (2.7%), an adjacent look-alike window scores higher than the edited original. For code that was rewritten or deleted (3.0% and 3.7%), a similar window near the old place passes the threshold, or an identical copy exists in another file. The three original quotation-based methods are misled in the same 1.0% of in-file moves. This is consistent with a copy of the block that differs only in whitespace lying closer to the old position, a residual ambiguity of the benchmark rather than of any one method.

B. Combined Design on Controlled Edits

The compound comparator changes search scope, tier order and threshold together (Table VI, *Cascade v1*). Cascade v1 recovers slightly more cross-file moves (99.0% against 97.0%) under its combined settings. However, it placed rewritten and deleted anchors on look-alike code elsewhere in the file: 18.0% and 13.7% of them, against 3.0% and 3.7% for the final order. Overall the final design is better by 98.5% to 96.4% ($b_{01}/b_{10} = 84/8$, $p < 10^{-16}$). Fig. 6 shows the threshold study on the tuning seed. The whole-file variant attaches more removed anchors to look-alike code at thresholds that recover genuine edits. In the final design, $\theta = 0.85$ keeps both tuning curves high; above about 0.9, genuine edits increasingly return no match. These comparisons do not separate window scope from tier order; the factorial analysis of Section X does.

C. Anchor Height, File Length and Cost

On the fresh replication seed the cascade reached 98.6%, against 74.9%, 58.3% and 41.7% for the baselines (Table VI). Accuracy stayed between 95.6% and 100.0% across anchor heights of 1–10 lines and between 98.1% and 98.9% across file-length bands (Fig. 7). One-line anchors are the hardest case

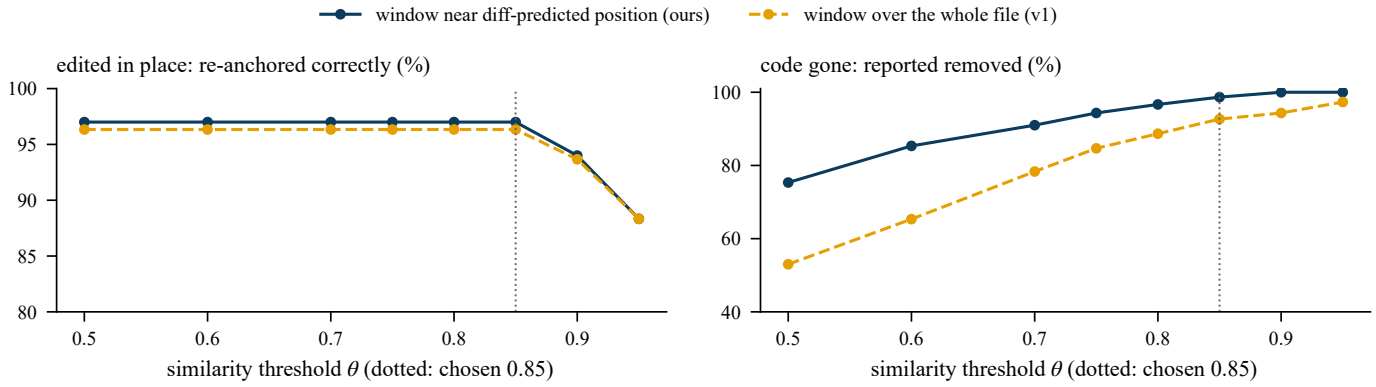


Fig. 6. Fuzzy-tier threshold on the separate tuning seed (seed 7, 150 trials per edit type). Left: anchors edited in place that are re-anchored correctly. Right: anchors whose code is gone that are correctly reported removed rather than attached to look-alike lines. Dotted line: the reported default $\theta = 0.85$.

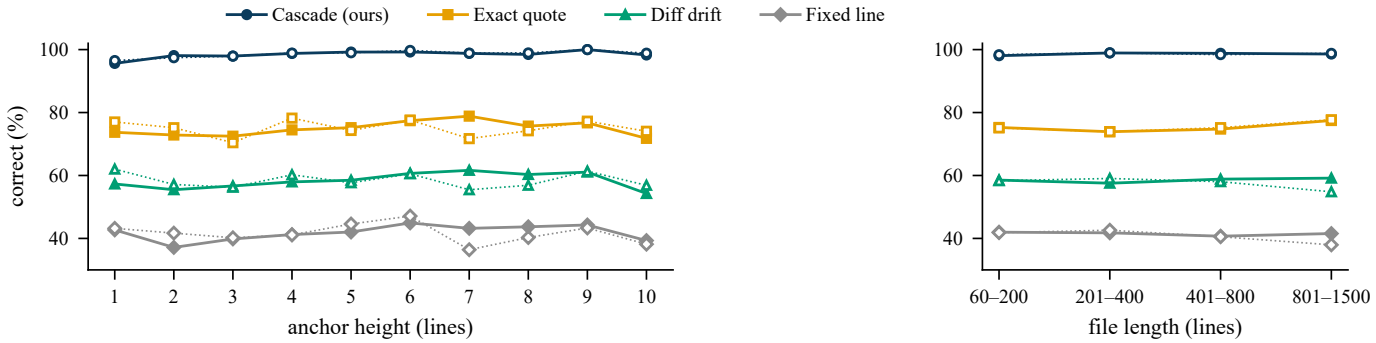


Fig. 7. Correct outcome by anchor height (left) and file length (right). Solid lines and filled markers: main seed; dotted lines and open markers: replication seed.

(95.6%), because a single edited line has many look-alikes nearby. The median relocation took 0.08 ms (95th percentile 65.1 ms, maximum 255 ms). Tier 3 is the only expensive step, and it scores at most $2r + 1$ windows. These per-anchor times do not establish whole-review or end-to-end latency; hardware, runtime configuration and review size must be specified before making that comparison.

D. Independent Hybrid Tracker

The three policy baselines do not attempt general recovery of edited lines. To test whether the aggregate advantage persists against an independently implemented contextual tracker, we added an exploratory comparison with the MIT-licensed Go implementation by Oselvar [44]. This implementation cites LHDiff and combines line-content Levenshtein similarity with surrounding-context cosine similarity. It is not the original paper authors' binary: its documentation states that it omits SimHash optimization and line-splitting support. We pinned revision `5a9f7fd` and used its command-line defaults: four context lines, content/context weights of 0.6/0.4 and the implementation's 0.45 combined-similarity cutoff. No threshold was tuned on our trials.

The adapter asks for a mapping of the old anchor's first line in the same path. A missing first-line mapping returns no match. This follows the original benchmark's path-and-start scoring; neither method receives credit for end-range validity. The tracker has no cross-file search or review-state policy. We therefore compare location outcomes only, and report cross-file and within-file strata separately. The original corpus, operators and seeds are unchanged. Trial metadata and the cascade outcomes are checked against the independently reproduced runs, and all 7,200 tracker outcomes are retained. Upstream tests and the adapter's one-based coordinate check pass. Go and Python timings are not compared, since the adapter adds inter-process communication and the implementations use different runtimes.

TABLE VII
INDEPENDENT TRACKER ON THE CONTROLLED BENCHMARK

Stratum	Method	Main correct %	Replication correct %
All ($n = 3,600$)	Cascade	98.5	98.6
	Hybrid tracker	91.3	91.5
Within-file ($n = 3,300$)	Cascade	98.6	98.8
	Hybrid tracker	99.6	99.8
Cross-file ($n = 300$)	Cascade	97.0	96.3
	Hybrid tracker	0.0	0.0

Denominators are per seed. The independent tracker is a pinned third-party implementation with same-file scope. All outcomes use the benchmark's existing start-location criterion. Its lack of cross-file recovery is reported explicitly rather than interpreted as poor within-file matching.

The tracker correctly handles 3,288 of 3,600 main observations, 91.3% (nominal Wilson interval 90.4–92.2), versus 3,545 for the cascade. The paired counts are 299 correct only for the cascade and 42 correct only for the tracker. On the

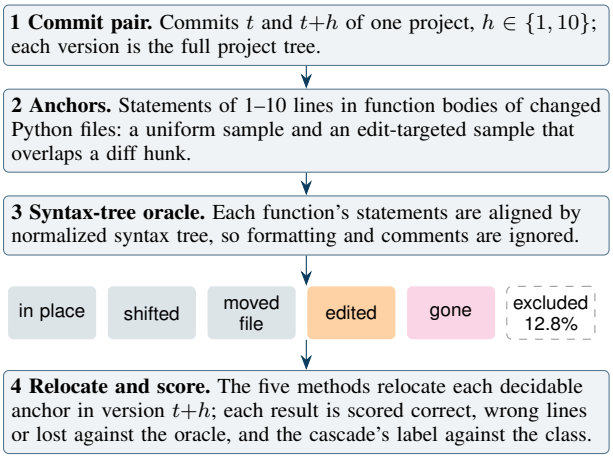


Fig. 8. Real-history procedure. Reference locations come from syntax-tree alignment; anchors the oracle cannot decide are excluded for every method.

replication seed, the tracker handles 3,293 observations (91.5%, interval 90.5–92.3). However, it loses all 300 cross-file targets in each seed because they are outside its scope. Among the other 3,300 observations, it correctly handles 3,288 (99.6%) versus 3,254 (98.6%) for the cascade on the main seed, and 3,293 versus 3,260 on replication. In-place token edits also favor the independent implementation: it recovers 588 of 600 token-edit and insertion-plus-edit targets on the main seed, versus 577 for the cascade.

The overall difference therefore reflects task scope and the benchmark's equal operator weights, not universal superiority of the cascade's within-file matching. The independent tracker is stronger on the within-file subset. Its implementation limits mean that neither this comparison nor the real-history extension in Section VII-G establishes the performance of the canonical LHDiff algorithm. The integration can in principle use a stronger within-file tracker while retaining lineage and human resolution, but that substitution has not been evaluated here.

VII. RQ1: Location on Real Histories

The synthetic benchmark controls every edit, but its twelve operators and their uniform mix are our choice. To test the findings on edits nobody designed, we repeated the comparison on the version histories of nine Python projects, 3,951 first-parent commits in total. Four come from our group: the review portal itself, a partner team's claim-to-code tracer, a language-model training system and an evidence-ingestion system (795 commits, a small group of developers). Five are widely used open-source libraries: Requests, Flask, Click, HTTPX and Rich. We made shallow clones of each default branch (depth 400), which gave 3,156 first-parent commits written by about a thousand contributors (1,036 names, counted per project). The source was only parsed, never executed. Fig. 8 summarizes the procedure. This section reports location; definition sensitivity and complete-source checks explain the reference's limits (Sections VII-E and VII-F), followed by independent tracking and full-range recovery (Sections VII-G and VII-H). Re-checks, calibration and repeated resubmission are reported in Sections IX–XI.

Algorithm 3 Syntax-tree oracle for an anchored statement

Require: statement x, the i-th statement of function g (qualified name) in file π of the old version; new version V'
$T(\cdot) \leftarrow$ syntax tree without positions, formatting or comments
$\pi' \leftarrow$ new path of π (Git rename detection)
if g is defined once in $V'[\pi']$ **then**
  align T of the body statements of g in both versions (sequence matching)
  if x is aligned to an equal statement y **then return** location of y
  end if
  if x has an offset-matched y of the same type in an equal-cardinality replacement **then return** location of y (edited)
  end if
  if x is in any other replacement **then return** *undecidable*
  end if
end if
$H \leftarrow$ function-body statements of V' with tree $T(x)$
if $|H| = 1$ **then return** location of its element (moved)
end if
if $|H| > 1$ **then return** *undecidable*
end if
if g is gone and a changed file has a function similar to g **then return** *undecidable*
end if
return *gone*

A. Commit Pairs and Anchors

We formed pairs of consecutive commits (a small revision) and non-overlapping pairs ten commits apart (a second, larger revision horizon). In each pair, anchors are statements of 1–10 lines directly inside function bodies of Python files the later commit modified, renamed or deleted, under the constraints of Section V. We drew two samples per pair. The *uniform* sample takes up to 8 anchors (at most 3 per file). The *edit-targeted* sample takes the same number from statements that overlap a hunk of the commit's diff, a deliberately enriched sample; diff overlap does not itself establish that a statement changed. Each version holds the full project tree of the later commit, so moved code can land anywhere.

B. Syntax-Tree Oracle

The reference procedure uses Python's abstract syntax tree (Algorithm 3). Functions are identified by qualified name, and their top-level body statements are aligned by normalized trees. Equal statements give a new start. In an equal-cardinality replacement hunk, the statement at the corresponding offset is accepted if its AST node type matches the old statement's type; this can pair unrelated statements. Deleted targets can be found as a unique equal tree elsewhere. Ambiguous replacements, copies and renamed-and-rewritten functions are excluded for all methods: 2,205 observations (12.8%), including 1324 unequal-cardinality replacements. Both the oracle and matcher use sequence alignment, on different representations. The recorded manual audit covers 40 in-house decisions, stratified between preserved and difficult classes; 39 were accepted and one pairing was doubtful. That audit is neither a blinded multi-rater validation nor evidence of oracle accuracy on public projects. After finding a start, the harness defines changed content by comparing an old-height normalized window there with the

old quotation. Thus its *edited* class is not identical to the set of changed AST statements (Section VII-E).

Direction of reference error. The reference calls a target unchanged only when the quoted lines, ignoring whitespace, stand at its location and occur once in that file; each of the 12,284 real-history targets it called unchanged passes this check. The check shows that the text there is unchanged, not that it is the reviewed code. For 12,117 of these targets identity rests on an equal syntax tree at the aligned position of the same function and for 21 on same-type pairing; the other 146 left their function and were placed on the unique equal statement elsewhere in the new version. Reading the earlier version of the public histories (tools/v11/audit_reference_copies.py), 8 of the 110 relocated targets landed where an identical statement already stood before the revision, so the reference may have taken a pre-existing copy for the reviewed code; the deployed cascade requested no check on any of them. Counted as hidden misses they raise the deployed miss rate from 1.3% to 1.6%, and counting also the 36 relocations in our own projects, whose earlier versions this check could not read, gives 2.9%. In the other direction, the same-type pairing of replaced statements decides 2,343 of the 2,780 changed or deleted targets. Of the deployed cascade's 37 real-history misses (Section IX), 6 rest on an equal syntax tree at the aligned position, 24 on same-type pairing and 7 on a deletion. Even if every pairing and deletion behind a miss were wrong, and the copy the cascade found were the true target, the miss rate would fall to 0.2%. Under these reference errors the deployed miss rate lies between 0.2% and 2.9%. The bounds cover missed re-checks only: identification errors can move placement accuracy either way and can hide unnecessary re-checks. The student-authored repositories use the same reference unchanged.

C. Fairness and Statistics

When Git detects a rename, the three simple policies receive the new path; both cascades retain the original path. All methods are scored on the same retained observations. Intervals are 95% percentile intervals from 2,000 bootstrap resamples of commit pairs, with paired differences resampled at the same level. The 15,064 rows represent 14,830 distinct pair-and-anchor-start targets: 234 observations repeat a target across the two sampling schemes. Statements may also recur across adjacent or overlapping histories. Pair clustering does not account for all temporal or project dependence. The default cascade uses $\theta = 0.85$, which no dated prospective protocol fixed; project-disjoint calibration is a secondary analysis rather than a prospective unseen-test claim.

D. Start Accuracy on Real Edits

On the uniform samples (11,449 observations), most targets are untouched or shifted and the content-based policies are near the ceiling (Table VIII). The cascade reaches 99.3% pooled over nine projects. Edit-targeting enriches the sample for harder locations without establishing that every selected statement changed: default accuracy is 66.8% on public projects and 70.0% in-house. The cascade leads all three simple policies in each project (Fig. 9), but Cascade v1 is stronger at its lower default threshold. Pairwise targeted accuracy is similar across the two sampled horizons (68.1% for adjacent commits and 67.4% ten commits apart); this does not test sequential carry.

By reference class (Table IX, Fig. 10), the cascade recovers every in-place and shifted target and 97.3% of cross-file moves. Its edited-class start recovery is 53.6%, whereas the three simple policies recover none by construction. Fixed lines lose shifted targets, and quote search and diff drift recover only those cross-file moves that Git supplies as renames. These findings establish an advantage over the specified policies on the retained reference set; the independent tracker comparison below tests a stronger within-file alternative.

E. Sensitivity to the Definition of Change

The stored rows retain both the oracle's alignment event and the quote-window class. These answer different questions. A statement may expand beyond the old span while leaving the first old-height window unchanged; conversely, reformatting an equal AST statement can change that window. We re-counted the recorded default labels using the oracle's replacement or deletion event as a second reference definition, without rerunning the matcher or changing its threshold (Table X).

There are 2,364 same-type AST-replacement observations and 144 deletion observations. Of these 2,508 reference events, 52 were labeled content-match or moved and therefore did not request re-check: recall is 97.9%. Eleven replacement observations were labeled content-match. Twenty-one replacements were classified as preserved by the original window definition (11 content-match and 10 moved). In the other direction, 293 aligned equal-tree observations were classified edited because the quote window differed. The two denominators are not nested, so the 97.9% and 98.7% rates must not be subtracted as a paired performance improvement.

This is a sensitivity analysis of the same retained observations, including repeats. The oracle's replacement marker is not a human judgment of semantic relevance and can itself contain pairing errors. Nevertheless, the disagreement is sufficient to bound the original interpretation: zero errors under a quote-window definition does not establish that no structural statement change was labeled content-match. The no-auto-resolution invariant remains separate from both matching metrics.

F. Verification Against Complete Source Revisions

Saved quotations alone cannot establish whether a reference points into the correct revision. We recovered the recorded heads of all nine histories and reconstructed each pair from its first-parent index and horizon. The five public heads were fetched at their recorded revisions with depth 400. Their first-parent counts matched the archive: 626, 448, 589, 400 and 1,093. Full head identifiers, traversal lists, shallow boundaries, pair revisions and file hashes are retained in the verification artifact. Private source remained local. No project source was executed.

We separately reran the original sampling and AST-reference procedure from the pinned histories, omitting only matcher evaluation and timing. In-house repositories used the original shared random stream and order; each separately collected

TABLE VIII
CORRECT OUTCOME (%) ON REAL EDIT HISTORIES

	Our four projects		Five open-source projects	
Method	uniform	edit-targeted	uniform	edit-targeted
Cascade (ours)	99.1 [98.7, 99.5]	70.0 [67.0, 73.0]	99.4 [99.2, 99.6]	66.8 [64.6, 69.1]
Cascade v1	99.3 [98.9, 99.6]	75.4 [72.4, 77.9]	99.6 [99.4, 99.7]	74.3 [72.3, 76.3]
Exact quote	98.1 [97.3, 98.7]	45.7 [42.1, 49.3]	98.2 [97.9, 98.5]	22.8 [20.9, 24.9]
Diff drift	98.0 [97.3, 98.7]	43.8 [40.4, 47.3]	98.1 [97.7, 98.4]	21.2 [19.3, 23.2]
Fixed line	36.4 [33.8, 38.9]	16.8 [14.5, 19.2]	53.3 [51.6, 54.8]	11.5 [10.1, 13.0]
anchors (commit pairs)	2,644 (457)	1,161 (354)	8,805 (1,616)	2,454 (874)

Brackets: 95% intervals from a bootstrap over commit pairs, both horizons pooled. Uniform: random statements in changed files. Edit-targeted: statements overlapping a diff hunk; overlap does not itself establish a changed statement. Reference labels from the syntax-tree procedure.

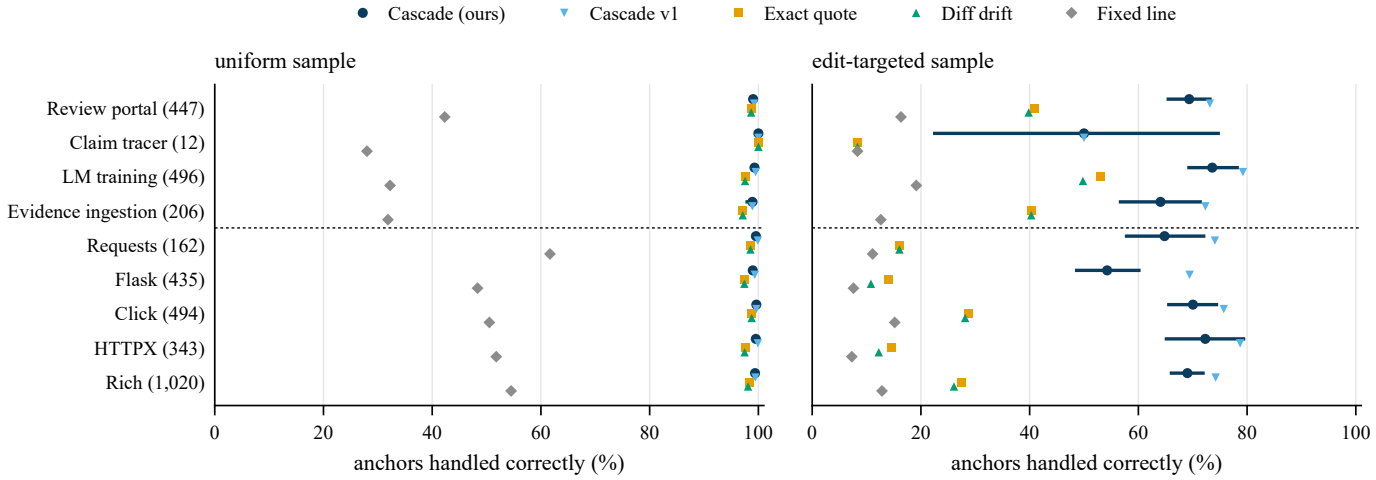


Fig. 9. Correct outcome per project on real edit histories (left: uniform sample; right: edit-targeted sample; in brackets the number of edit-targeted anchors). Bars: 95% bootstrap interval of the cascade over commit pairs. Above the dashed line our projects, below it the open-source ones. The claim tracer has few anchors, hence its wide interval.

TABLE IX
CORRECT OUTCOME (%) BY ORACLE CLASS ON REAL EDITS

Oracle class	n	Cascade	v1	Quote	Drift	Line
in place	5,966	100.0	100.0	100.0	99.9	100.0
shifted	6,207	100.0	100.0	100.0	98.7	0.0
moved file	111	97.3	97.3	21.6	21.6	18.0
edited	2,636	53.6	64.8	0.0	0.0	0.0
gone	144	92.4	71.5	95.1	100.0	100.0

All nine projects, both samples and both horizons pooled.

TABLE X
RE-CHECK RECALL UNDER TWO REFERENCE DEFINITIONS

Reference definition	n	Missed	Content-match	Recall
Quote window changed / gone	2,780	37	0	98.7%
AST replacement / deletion	2,508	52	11	97.9%

Same 15,064 retained observations and default cascade; the denominators select different reference events. Missed means labeled content-match or moved, neither of which requests re-check. Content-match is the number of misses with that label. The AST row uses the oracle's replacement/deletion marker and is a sensitivity analysis, not a human semantic judgment. Repeated observations are retained to match the original analysis.

public repository restarted seed 20261028. All 15,064 ordered retained observations matched the saved selection, source coordinates, quotation text and reference class. The 2,205 exclusions and every per-project exclusion-reason count also matched. This verifies deterministic regeneration under Python 3.12.14, not independent human validity of the reference rule. The retained per-project reports distinguish this check from the complete-source coordinate check and the saved-row aggregation.

All 15,064 old quotations matched the reconstructed old revision. Every assigned reference's saved old-height target window also matched the new revision. For 14,917 non-deleted observations, the old and referenced new function-body statements could be uniquely recovered and compared structurally: all 2,364 replacement markers had different syntax trees, while the 12,553 aligned/relocated markers had equal trees. Three further observations had ambiguous same-line statement extraction in this additional checker; 144 had no assigned surviving target. These checks validate recorded coordinates and structural relations. They do not establish that a same-type replacement is the same conceptual entity, independently prove deletion, or replace human adjudication.

The complete source explains the 21 observations where the

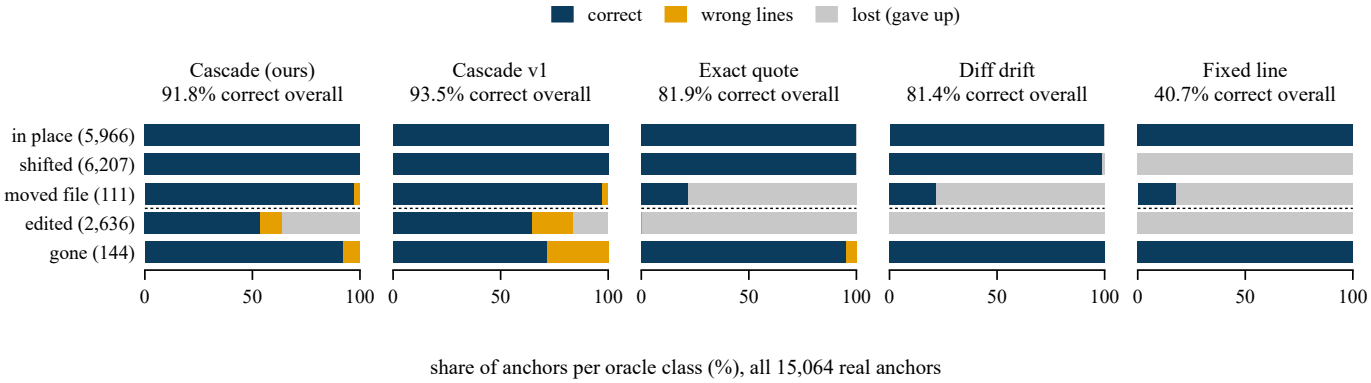


Fig. 10. Outcome per oracle class for each method on all real anchors, the counterpart of Fig. 5. Above the dashed line the anchored content is preserved; below it the content was edited or is gone. Baselines lose every edited anchor; the fixed-line policy also loses every shifted one.

quote window is preserved but the reference statement changes. Nineteen observations (18 distinct pair/anchor targets) grow beyond the old range. The remaining two change decorators above a function definition, outside the line interval the original AST-based sampler used. Together these are 20 distinct targets, since one growing target occurs in both sampling schemes. Eleven observations retain a content-match label: ten growing statements and one decorator change. The other ten are labeled moved. These are concrete limits of the quoted interval, rather than evidence that every such change violates the matcher's normalized-text contract.

For example, a Click revision renames the decorator call from `resultcallback` to `result_callback` above an otherwise unchanged function [45]. The AST includes the decorator; the sampled quote begins at the function's definition line and omits it. The content-match label is therefore compatible with identical quoted lines, but cannot certify an unchanged complete statement. Similarly, retaining the old number of lines omits a growing statement's new suffix. These examples motivate evaluating complete source extents and relevant context separately from quote preservation.

We also prepared a source-based review packet with expected targets and matcher outcomes withheld. It contains 50 cases stratified equally across the original five reference classes and the 20 distinct quote/AST disagreements, shuffled in a recorded order. The unblinding key is separate. No independent reviewer responses have been collected, so the packet is preparation for validation, not a human-validation result.

G. Independent Tracker on Real Histories

We extended the pinned hybrid tracker from Section VI-D to the same 15,064 retained real-history observations. The input is the complete old file and its corresponding new path, using Git rename detection as for the original simple policies. The reference target path is never supplied as a search hint. Mapping the old anchor's first line gives a candidate start; scoring retains the original file/start reference. Thus this experiment compares location recovery, not the review-state policy or complete returned ranges. It remains conditional on the original oracle and exclusions.

The unmodified tracker dependency crashes on some non-BMP Unicode characters: its Levenshtein implementation indexes a 65,536-element table with a larger code point. We reproduced that failure with a minimal changed emoji line. A failed process is discarded before another file pair is evaluated, preventing partially mutated global state from affecting later requests. Neither source text nor the matching algorithm is patched. Failed file pairs are recorded as execution errors, not silently removed or relabeled as a successful no-match. The run completes with 143 affected observations across 37 distinct source-content file pairs, all in the public group. The 143 observations comprise 85 uniform and 58 edit-targeted cases. A regression check confirms both explicit failure reporting and a fresh successful request afterward.

The tracker correctly locates 1,043 of 1,161 in-house edit-targeted observations (89.8%) and 2,052 of 2,454 public edit-targeted observations (83.6%, including execution failures in the denominator). The default cascade reaches 70.0% and 66.8%, respectively. On public edit-targeted observations the default cascade-minus-tracker difference is -16.8 percentage points, with a commit-pair interval of $[-19.3, -14.4]$. This reverses any interpretation that the cascade's advantage over simple policies establishes generally stronger within-file tracking.

Calibration narrows the public edit-targeted gap: 82.6% for the cascade versus 83.6% for the tracker. Their paired difference is -1.0 point, interval $[-3.1, 1.1]$; this analysis does not resolve its direction and is not an equivalence test. On the 2,396 public edit-targeted observations where the tracker completes, its accuracy is 85.6%, compared with 82.2% for the calibrated cascade on that same subset. Failures and conditional accuracy therefore need separate reporting. The in-house edit-targeted sample has no tracker execution failures and still favors the tracker over both cascade settings.

By original reference class, the tracker correctly handles 2,192 of 2,636 edited observations (83.2%), including 51 execution errors as unsuccessful outcomes, versus 1,412 (53.6%) for the default cascade. The cascade is stronger on the 111 cross-file observations (108 correct versus 25, largely those available through Git renames) and on 144 gone targets (133 versus 122). The comparison supports distinct engineering choices: stronger within-file localization is available, while

TABLE XI
INDEPENDENT TRACKER ON RETAINED REAL-HISTORY OBSERVATIONS

Project group	Sample	n	Tracker correct % [95% CI]	Execution errors	Cascade default / calibrated %
In-house	uniform	2,644	99.5 [98.9, 99.9]	0	99.1 / 99.4
In-house	edit-targeted	1,161	89.8 [87.4, 92.1]	0	70.0 / 81.8
Public	uniform	8,805	98.7 [98.2, 99.1]	85	99.4 / 99.7
Public	edit-targeted	2,454	83.6 [81.6, 85.5]	58	66.8 / 82.6

Correct percentages use all retained observations; an execution error contributes no correct location. Intervals are 2,000 percentile bootstrap resamples of commit pairs, seed 20261028. Calibration is the existing 0.5 setting chosen on in-house observations; the independent tracker uses its unchanged defaults. On completed public observations alone, tracker accuracy is 99.6% for uniform and 85.6% for edit-targeted cases. These conditional figures describe a different subset, so paired comparisons must restrict both methods to that same subset.

cross-file recovery, removal conservatism and implementation robustness matter separately. Keeping persistent issue state independent of the chosen tracker allows that component to be evaluated or replaced without treating a recovered location as proof of resolution.

H. Source-Line Range Recovery

The original location metric checks the reference file and start, not the returned end line. We ran the pinned cascade again at the unchanged default and previously calibrated thresholds on every retained observation, using the complete new-version Python file map and previous file. All 15,064 default labels and both thresholds' path/start outcomes matched the saved records. A local exploratory protocol preceded this run; it was not externally registered. No threshold was tuned for the new metric.

The primary denominator is the 14,917 surviving observations whose old and referenced new statements were uniquely recovered in Section VII-F. An exact range must match the reference file, start and the new AST node's inclusive `end_lineno`. We exclude the three ambiguous same-line extractions from both thresholds' range metrics. The 144 gone observations remain a separate removal outcome: 133 correctly reported removed at the default threshold and 89 at the calibrated value. This AST line extent is a syntactic proxy, not a validated boundary of the review concern; decorators above the definition and other relevant context remain outside it.

On the 2,359 surviving public edit-targeted observations, calibrated accuracy falls from 83.7% for file/start to 76.8% for the exact range (Table XII). The paired reduction is 6.9 percentage points, with a 95% commit-pair bootstrap interval of $[5.8, 7.9]$. The default falls from 65.8% to 62.0%, a 3.8-point reduction, interval $[3.0, 4.6]$. These intervals use 2,000 paired resamples, seed 20261028, with the same pair-cluster/percentile convention as the other real-history analyses. The denominators differ from the original all-target accuracy because gone targets are not assigned a surviving range.

The boundary limitation is concentrated in changed line spans. Of 824 observations whose new reference span differs from the original height, only four have exact ranges at either threshold. The default finds 182 correct starts in this subset, but 165 ranges truncate the statement and 13 extend beyond it; calibration finds 321 correct starts, with 274 truncated and 43 extended ranges. Across all 14,917 surviving observations, start/range accuracy is 91.8%/90.6% at default and 95.8%/93.7% after calibration. Within the original edited class, the corresponding exact-range figures are 47.6% and 65.3% on 2,634 recoverable observations. These are conditional benchmark results, not estimates of classroom prevalence.

The mechanism explains the result: the fuzzy tier preserves the old window height, while exact normalized runs can span blank-line changes. Lowering the similarity threshold recovers more starts, but does not adapt the end boundary to a growing or shrinking statement. Mean line-interval intersection-over-union across surviving targets is 0.923 at default and 0.967 after calibration; overlap can therefore improve without exact boundaries. Recovering a usable location, recovering the complete syntactic extent, and deciding whether a concern is resolved remain separate tasks.

VIII. RQ1: INDEPENDENT BENCHMARK AND STUDENT CODE

A. Human-Annotated Multi-Language Benchmark

Our syntax-tree reference shares its construction with syntax-aware differencers and is not human ground truth. We therefore evaluated every method on the public CodeMapper benchmark [8]: 200 code regions whose later locations were annotated by people in two sets (one with manually and one with automatically chosen source regions), plus tool-derived Java histories and a suppression set, 2,392 regions in all. We ran CodeMapper's pinned artifact unchanged and GumTree's official release as additional comparators. The protocol, adapters and frozen guard code were fixed before any of these data were evaluated, so the results are held out for every method. Two limits apply. Living Review sees only the pair of files in each case, not a whole project, so cross-file recovery is not exercised. And our fresh CodeMapper runs did not reproduce its published numbers exactly (for example 76 against 77 exact regions of 100 in the first set); we report our own runs.

On the human-annotated regions the deployed cascade found the correct line, or correctly reported the region gone, for 66.0%, against 85.5% for the contextual tracker and 91.5% for CodeMapper (Table XIII). The gap is larger for exact regions: CodeMapper recovered 74.0% of the annotated character regions exactly, the cascade 24.5%. Outside Python the cascade reached 63.7% (CodeMapper 89.7%). On the full external set the gap persisted (57.5% against 86.2%). CodeMapper, a region mapper built for this task, is substantially more accurate at

TABLE XII
START AND EXACT SOURCE-LINE RANGE ACCURACY ON THE SAME SURVIVING TARGETS

Project group	Sample	n	Default $\theta = 0.85$ File/start %	Default $\theta = 0.85$ File/start/end %	Calibrated $\theta = 0.5$ File/start %	Calibrated $\theta = 0.5$ File/start/end %
In-house	uniform	2,636	99.1	98.9	99.4	99.0
In-house	edit-targeted	1,136	69.7	63.3	82.1	70.8
Public	uniform	8,786	99.4	99.3	99.7	99.6
Public	edit-targeted	2,359	65.8	62.0	83.7	76.8

Both metrics in each row use exactly the same surviving observations. Three ambiguous AST extractions and 144 gone targets are outside this table. Complete returned coordinates are retained. The line-only external tracker has no statement-range policy and is not assigned an invented end line.

TABLE XIII
HUMAN-ANNOTATED (INDEPENDENT) AND MULTI-LANGUAGE BENCHMARKS

	Human A/B (200)				Non-Py	All
Method	Line	Chars	Missed	Silent	Line	Line
Deployed cascade	66.0	24.5	8.5	3.5	63.7	57.5
All guards, $\theta = .85$	70.0	26.0	1.5	0.5	67.8	64.5
All guards, $\theta = .55$	77.0	17.5	1.5	0.5	77.4	70.9
Cascade v1	70.0	21.5	8.5	3.5	67.1	59.5
Contextual tracker	85.5	28.0	3.8	0.0	86.3	70.2
GumTree	62.0	17.0	1.5	0.5	71.2	66.8
CodeMapper	91.5	74.0	4.6	1.0	89.7	86.2

Percentages. Line: file and start line, including correct absence. Chars: exact character region. All guards: IG, the original copy guard AG and VW (Table XXIV uses the corrected AGi). CodeMapper: this evaluation's run and change adapter (Table XXIV uses a later run with the cascade's quotation labels). Non-Py: the 146 human-annotated regions outside Python. All: all 2,392 external regions, including tool-derived Java histories. Living Review sees only the paired files. GumTree execution failures count as incorrect. The pinned CodeMapper is not fully deterministic (Section XI-H).

TABLE XIV
STUDENT-AUTHORED REPOSITORIES (HELD OUT)

	All observations				Edit-targeted	
Method	Acc	Missed	Silent	Extra	Acc	Missed
Deployed cascade	91.5	3.1	1.5	0.0	73.6	2.6
Pre-existing-copy guard	91.5	0.2	0.4	4.4	73.6	0.1
All guards, $\theta = .85$	92.8	0.2	0.4	4.7	78.2	0.1
Cascade, $\theta = .55$	94.6	1.2	1.1	0.2	84.1	0.9
All guards, $\theta = .55$	95.9	0.1	0.4	4.7	88.7	0.0
Cascade v1	91.3	8.2	2.6	0.0	73.0	7.8
Contextual tracker	94.9	0.5	0.2	3.3	85.4	0.3
Diff drift	76.1	0.1	0.1	5.4	20.9	0.0
Exact quote	77.8	0.5	0.2	3.2	25.7	0.3
Line number	43.6	0.1	0.0	47.3	13.1	0.1

Percentages over 7,882 observations (1,857 edit-targeted) from 20 repositories. Acc: file/start, including correct absence. Missed: changed or gone targets without a re-check request. Silent: wrong placement without a request. Extra: requests on preserved targets.

locating annotated code than the cascade, and the cascade's fixed-height windows recover region boundaries poorly.

Safety is a different question. The deployed cascade left 8.5% of the needed re-checks unrequested on the human-annotated regions and placed 3.5% of all regions on wrong code without a request. With the three guards in the version evaluated here (IG, the original copy guard AG and VW; Section XII uses the corrected AGi) these fell to 1.5% and 0.5%, and accuracy rose to 70.0% (77.0% at the calibrated threshold). The price is paid on preserved code: re-check requests for the 70 regions whose content was unchanged rose from 34.3% to 50.0%. CodeMapper missed 4.6% of the re-checks under this evaluation's change adapter, which compares its mapped text with the original; it has no review-state model of its own. Section XII reports a later run of the same pinned CodeMapper with a read-only path resolver and the cascade's own quotation labels, which missed 4 of 130 (3.1%). These are small samples (200 regions), and the human annotations were made for region tracking, not for review concerns. They support two conclusions: the cascade is not a competitive locator, and its value lies in the state contract it feeds; a deployment could replace the matcher with a region mapper such as CodeMapper and keep the same labels, guards and issue lifecycle.

B. Student-Authored Repositories

The real-history projects of Section VII are professional or our own. As a stand-in for student code we selected 20 public Python repositories that describe themselves as capstone, senior-design, final-year or course projects, by a frozen search and screening protocol: team authorship, at least 2,000 lines of Python and 60 first-parent commits, and at least 90% of commits before December 2022. They are student work, but they are not supervised resubmissions, and commits are not submissions. The original commit-pair selection and syntax-tree reference were applied unchanged, yielding 7,882 observations from 1,436 commit pairs (1,857 edit-targeted) after 1,659 undecidable targets were excluded. No threshold, guard or design was chosen on these repositories, so every method is evaluated here on held-out data (Table XIV).

The deployed cascade placed 91.5% of the targets correctly (95% commit-pair cluster interval 90.5–92.4%), and the contextual tracker 94.9%. The tracker's lead (paired cluster interval of deployed minus tracker: -4.6 to -2.2 points) held in 13 of the 20 repositories, with 2 ties, and widened on edit-targeted observations (73.6% against 85.4%). This replicates the real-history finding of Section VII-G on independent code: the cascade's location recovery is not state of the art. Here the tracker was given Git's rename detection. The common-input comparison of Section XII gives every locator the anchor's original path only; under that rule the tracker placed 91.5% of the same targets, the cascade's figure, because 271 of its placements were on renamed files. Its lead on edit-targeted observations remains (83.6%).

The safety results replicate as well, and here they are held out. The deployed cascade failed to request 3.1% of the needed

TABLE XV
REAL-HISTORY MISSED RE-CHECKS OF THE DEFAULT CASCADE BY TIER, REFERENCE CLASS, ORACLE EVENT, PROJECT AND HORIZON

Tier	Reference class	Oracle event	Project	Horizon	Misses
tier 4 (other file)	edited	aligned	LM training	1	3
tier 4 (other file)	edited	aligned	Review portal	1	2
tier 4 (other file)	edited	edited	HTTPX	1	1
tier 4 (other file)	edited	edited	LM training	1	4
tier 4 (other file)	edited	edited	LM training	10	1
tier 4 (other file)	edited	edited	Evidence ingestion	1	1
tier 4 (other file)	edited	edited	Rich	1	6
tier 4 (other file)	edited	edited	Rich	10	3
tier 4 (other file)	edited	edited	Review portal	1	1
tier 4 (other file)	edited	edited	Review portal	10	1
tier 2 (same file)	edited	aligned	Review portal	1	1
tier 2 (same file)	edited	edited	Flask	1	2
tier 2 (same file)	edited	edited	Flask	10	1
tier 2 (same file)	edited	edited	HTTPX	1	1
tier 2 (same file)	edited	edited	LM training	1	1
tier 2 (same file)	edited	edited	Review portal	1	1
tier 2 (same file)	gone	deleted	LM training	1	1
tier 2 (same file)	gone	deleted	Rich	1	4
tier 2 (same file)	gone	deleted	Rich	10	2

All cases and normalized copy coordinates, same/other-file counts, distances, anchor height and output coordinates are in missed-recheck-cases.csv and error-analysis.json. These are mechanistic attributions, not human adjudication.

re-checks (interval 2.0–4.3%) and placed 1.5% of the issues on wrong code without a request. The pre-existing-copy guard alone reduced missed re-checks to 0.2%; all three guards reduced them to 0.2% and silent misplacement to 0.4%, while raising accuracy to 92.8%. Their cost was 4.7% extra requests on preserved targets, concentrated on edit-targeted ones (30.3%). With the threshold of 0.55 chosen by leave-one-project-out calibration on the original projects, all three guards reached 95.9% accuracy with 0.1% missed re-checks, close to the tracker's 94.9% and 0.5%. The guards were designed from the original histories' failures; that they transfer to unseen student repositories is the strongest evidence in this paper that they address general mechanisms rather than the cases that motivated them.

IX. RQ2: SAFETY OF CARRY-FORWARD

Safety is a defining property of Living Review, so we report it as a family of rates rather than as a single count. For a set of targets with reference classes, let C be the targets whose code was edited or is gone, P the content-preserved targets, S the targets that survive and A all targets, and let $R = \{\textsf{changed}, \textsf{removed}\}$ be the labels that request re-check. Then

$$\text{missed re-check} = |\{c \in C : \ell(c) \notin R\}| \,/\, |C|, \quad (3)$$

$$\text{false re-check} = |\{p \in P : \ell(p) \in R\}| \,/\, |P|, \quad (4)$$

$$\text{silent misplacement} = |\{x \in W : \ell(x) \notin R\}| \,/\, |A|, \quad (5)$$

where $W \subseteq A$ are the targets placed at a wrong location, and the lost rate is the share of S for which no location is returned. The re-check rule is the deployed one: changed and removed request re-check; identical, content-match and moved keep the issue's state. Earlier versions of this work scored every label other than the deployed `exact` as flagged; that adapter counted a moved label as a re-check and therefore could not observe the failures below. Placement (file and start line on surviving targets) and classification (label against reference class) are scored separately, because a location can be right while its change status is wrong.

A. Missed Re-checks Are Rare but Not Absent

On controlled edits and real histories, no observation receives a content-match label when the code inside its quoted span changed; edits around the span are outside this check, as the applied review suggestions show. That is true but incomplete. On real histories, 37 of 2,780 edited or gone targets (1.3%; Wilson 95% interval 1.0–1.8%) are not returned for re-check, and 40 of 15,064 targets (0.3%) are placed on the wrong code without a request. On the controlled main seed the corresponding counts are 9 of 1,200 and 15 of 3,600. The cascade never requested re-check for preserved real-history content (0 of 12,284); its cost on real histories is instead 6.4% lost anchors, which request re-check and leave the supervisor to search manually.

Every one of the 37 real-history misses carries the moved label (Table XV). In 23 cases tier 4 found the quotation in another file and in 14 cases tier 2 found it elsewhere in the same file, while the original had been edited (30) or deleted (7). The mechanism is a pre-existing identical copy: the matcher follows the text, and the text still exists. 27 of the 37 anchors are a single line, the kind of short statement most likely to recur verbatim. This is the identical-code ambiguity anticipated in Section III, and it is the dominant safety failure of the method; Section XI shows that it becomes frequent when copies are common.

B. What a Missed Re-check Means for the Supervisor

A missed re-check does not hide an issue. The issue stays active and visible in the new review, with its carried quotation, its new location and the plain-language note "the code it points to moved; it is shown at its new place". What is lost is the prompt: the issue is not marked needs re-check, and the location it shows is a copy of the old code, so a supervisor who trusts the location sees the old code apparently unchanged. A silent misplacement is the same failure seen from the location side. Whether supervisors notice such cases was not measured. The interface's only mitigation is that the carried quotation is shown beside the new location. That quotation is the text at the previous version, not necessarily the one the supervisor reviewed (Section IV-D), and showing it does not replace an empirical check.

C. Scope of the Guarantee

The state contract guarantees, for the evaluated implementation, that no system action marks an issue resolved; Section XIII verifies this against every status write in the code. It does not guarantee that every change is detected or that every anchor is placed correctly. Those are empirical properties of the matcher, bounded by the rates above for the evaluated distributions only.

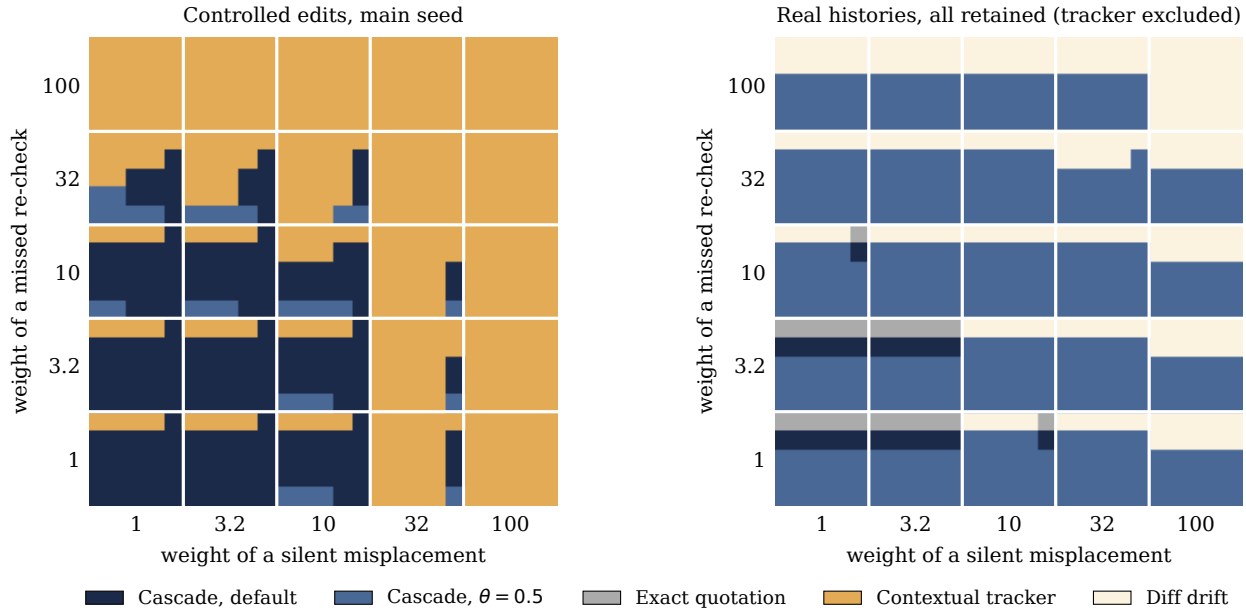


Fig. 11. Cheapest method under the additive effort model. Weights are chosen values in units of the cost of one lost anchor; each outer cell fixes the weights of a silent misplacement (horizontal) and a missed re-check (vertical), and its inner grid varies the weights of an unnecessary re-check (horizontal) and of a wrong placement that requests re-check (vertical) over the same log-spaced grid. The real-history panel omits the contextual tracker, whose rates Table XVI reports, because its execution failures have no defined inspection cost; the evidence package reports that sensitivity separately.

TABLE XVI
SAFETY AND EFFORT OUTCOMES OF CARRY-FORWARD

	Controlled edits, main seed (3,600 trials)						Real histories, all retained observations (15,064)					
Method	place	class	missed	silent	lost	false	place	class	missed	silent	lost	false
Cascade, default	98.8	99.1	0.8	0.4	0.1	0.2	91.8	93.4	1.3	0.3	6.4	0.0
Cascade, θ=0.5	96.5	93.6	0.6	0.3	0.1	3.2	95.8	99.2	0.5	0.1	0.4	0.1
Cascade v1	98.8	97.1	1.4	0.6	0.0	0.0	93.8	96.6	1.8	0.3	2.8	0.0
Contextual tracker†	89.6	91.3	0.0	0.0	10.4	12.5	95.8	96.2	0.3	0.1	2.7	1.5
Exact quotation	69.9	75.0	0.4	0.3	29.8	12.4	81.7	81.9	0.5	0.1	18.2	0.7
Diff drift	50.0	58.3	0.0	0.0	50.0	37.5	81.2	81.4	0.0	0.0	18.8	1.4
Line number	30.0	41.7	0.0	0.0	70.0	62.5	40.1	40.7	0.0	0.0	59.9	51.3

Percentages. *place*: correct file and start among surviving targets; *class*: label agrees with reference class; *missed*: Eq. (3), among edited or gone targets; *silent*: wrong location without re-check, among all targets; *lost*: surviving target with no location; *false*: re-check requested for preserved content. †The tracker has no review-state policy; a declared adapter requests re-check when the normalized quotation at its returned line differs, and its 143 execution failures count as incorrect, as GumTree's do in Table XIII. Raw counts and Wilson intervals for every cell are in the evidence package.

D. Safety–Utility Trade-off

Configurations trade the outcomes against one another rather than improving all of them. Lowering the threshold to 0.5 cuts real-history lost anchors from 6.4% to 0.4% and missed re-checks from 1.3% to 0.5%, but raises wrong placements from 1.9% to 4.1%, nearly all of which do request re-check. The contextual tracker has the lowest real-history missed and silent rates of the evaluated matchers but cannot follow code between files. Under an additive cost model with the cost of a lost anchor fixed at one and log-spaced weights for the other outcomes, no method minimizes cost across the whole weight range (Fig. 11); the ranking changes with the relative price of a silent misplacement and a manual search. We therefore state the deployment objective explicitly: minimize missed re-checks and silent misplacements first, and only then manual searches and unnecessary re-checks. Under that objective the guards of Section III-D, not a lower threshold, are the relevant intervention. The weights are illustrative; no faculty cost was measured.

TABLE XVII
RELOCATION–DELETION TRADE-OFF OF THE FINAL CASCADE ON REAL HISTORIES

Threshold	Edited correct / n	Edited %	Gone correct / n	Gone %
0.5	2020/2636	76.6	89/144	61.8
0.7	1870/2636	70.9	118/144	81.9
0.85	1412/2636	53.6	133/144	92.4
0.95	699/2636	26.5	136/144	94.4

All retained real-history observations, both sampling schemes and all projects. Edited and gone have separate denominators; the columns are not a percentage partition.

X. RQ3: DESIGN FACTORS AND CALIBRATION

A. The Default Does Not Dominate Its Predecessor

On real edit-targeted public histories, the earlier cascade v1 relocates more targets than the final default (74.3% versus

TABLE XVIII
FACTORIAL CONTRASTS ON SHARED OBSERVATIONS, AVERAGED ACROSS THE THRESHOLD GRID (PERCENTAGE POINTS)

Population	Contrast, averaged across thresholds	Accuracy pp	Missed pp	Gone correct pp	Wrong pp
controlled, main	cross-file first − fuzzy first	0.96	0.79	0.00	-0.96
controlled, main	whole file − local window	-2.35	-0.06	-11.40	2.41
controlled, main	order × scope interaction	0.86	0.13	0.00	-0.86
controlled, replication	cross-file first − fuzzy first	1.14	0.34	0.00	-1.14
controlled, replication	whole file − local window	-2.07	-0.05	-9.52	2.14
controlled, replication	order × scope interaction	1.03	0.11	0.00	-1.03
real, internal	cross-file first − fuzzy first	-0.23	1.99	0.00	0.23
real, internal	whole file − local window	-0.25	-0.01	-14.00	0.46
real, internal	order × scope interaction	0.03	0.03	0.00	-0.03
real, public	cross-file first − fuzzy first	-0.05	0.51	0.00	0.05
real, public	whole file − local window	-0.58	-0.04	-14.30	0.81
real, public	order × scope interaction	0.04	0.08	0.00	-0.04

Contrasts use full-height windows. Scope and order are crossed at each threshold; the final row for each population is the order-by-scope difference in differences. Full cells, threshold interactions and the legacy-partial factor are retained in factorial-summary.json.

TABLE XIX
LEAVE-ONE-PROJECT-OUT THRESHOLD SELECTION AND HELD-OUT EDIT-TARGETED OUTCOMES

Held-out project	Search	Selected cell	Targeted correct	Targeted missed
Click	threshold only	fuzzy first, local, full, 0.55	421/494 (85.22)	0/355 (0.00)
Click	full cell	fuzzy first, local, full, 0.55	421/494 (85.22)	0/355 (0.00)
Flask	threshold only	fuzzy first, local, full, 0.55	343/435 (78.85)	3/387 (0.78)
Flask	full cell	fuzzy first, local, full, 0.55	343/435 (78.85)	3/387 (0.78)
HTTPX	threshold only	fuzzy first, local, full, 0.55	300/343 (87.46)	1/282 (0.35)
HTTPX	full cell	fuzzy first, local, full, 0.55	300/343 (87.46)	1/282 (0.35)
LM training	threshold only	fuzzy first, local, full, 0.55	421/496 (84.88)	2/236 (0.85)
LM training	full cell	fuzzy first, local, full, 0.55	421/496 (84.88)	2/236 (0.85)
Evidence ingestion	threshold only	fuzzy first, local, full, 0.55	162/206 (78.64)	0/123 (0.00)
Evidence ingestion	full cell	fuzzy first, local, full, 0.55	162/206 (78.64)	0/123 (0.00)
Requests	threshold only	fuzzy first, local, full, 0.55	130/162 (80.25)	0/145 (0.00)
Requests	full cell	fuzzy first, local, full, 0.55	130/162 (80.25)	0/145 (0.00)
Rich	threshold only	fuzzy first, local, full, 0.55	843/1020 (82.65)	6/761 (0.79)
Rich	full cell	fuzzy first, local, full, 0.55	843/1020 (82.65)	6/761 (0.79)
Claim tracer	threshold only	fuzzy first, local, full, 0.55	7/12 (58.33)	0/11 (0.00)
Claim tracer	full cell	fuzzy first, local, full, 0.55	7/12 (58.33)	0/11 (0.00)
Review portal	threshold only	fuzzy first, local, full, 0.55	356/447 (79.64)	1/269 (0.37)
Review portal	full cell	fuzzy first, local, full, 0.55	356/447 (79.64)	1/269 (0.37)

Training objective uses all anchors of the other projects. Edit-targeted training-objective sensitivity, all-anchor held-out outcomes and default comparisons are in calibration.json. Previously inspected histories make this retrospective transfer analysis.

66.8%; internal 75.4% versus 70.0%), although the final default leads on controlled edits (98.5% versus 96.4%). The reversal is a primary result, not a footnote: the default threshold of 0.85 was tuned on controlled token edits and is too strict for naturally edited statements, on which it relocates only 53.6% of edited statements correctly (Table XVII). v1 also differs in tier order, search scope and threshold, so the comparison does not isolate any single choice.

B. Factorial Ablation

To separate the choices, we evaluated every combination of tier order (local fuzzy search before cross-file exact search, as deployed, or after it, as in v1), search scope (a window around the diff-predicted line or the whole file) and threshold $\theta \in \{0.50, 0.55, \ldots, 0.95\}$ on both controlled seeds and all real-history observations. The two corners matching the deployed cascade and v1 reproduce the archived outcomes exactly. Averaged over thresholds, searching the whole file instead of the local window lowers the rate at which deleted code is correctly reported as removed by 14.00 points on internal and 14.30 points on public projects: a distant look-alike is accepted in place of an honest removed. Running cross-file exact search before local fuzzy search raises missed re-checks by 1.99 and 0.51 points, because an edited block is then matched to an identical copy elsewhere, which is exactly the failure analyzed in Section IX. Both deployed choices are therefore supported on safety grounds. Accuracy alone favors lower thresholds, and at equal thresholds the final design leads v1 in all nine projects after calibration; on internal projects the equal-threshold advantage of 1.64 points has a project-cluster interval of $[-0.46, 2.17]$, so its direction is not established there. Table XVIII reports the averaged contrasts, and the evidence package retains every cell; these are matched contrasts on shared observations, not randomized experiments.

C. The Relocation–Deletion Trade-off

Lowering the threshold recovers more edited statements and reports fewer deleted ones correctly (Table XVII). At $\theta = 0.50$, 76.6% of edited statements are relocated but only 61.8% of deleted statements are reported removed; at the default 0.85 the figures are 53.6% and 92.4%. No threshold is best for both. A deleted statement that is relocated attaches a concern to different code, which is the more consequential failure, whereas an edited statement that is not relocated becomes a visible removed anchor and requests re-check. This trade-off

TABLE XX
DUPLICATE STRESS SUITE ON THE HELD-OUT SEED AT $\theta = 0.85$ (%)

Switches	Correct	Missed	Wrong/silent	False request	Extra/100	Range
None	30.53	89.73	69.28	0.13	0.00	35.60
IG	30.53	89.47	69.04	0.36	0.22	35.60
AG	32.40	20.73	27.12	28.27	28.13	37.73
VW	30.51	89.67	69.25	0.13	0.00	35.57
IG+AG	32.40	20.73	27.07	28.36	28.22	37.73
IG+VW	30.51	89.40	69.01	0.36	0.22	35.57
AG+VW	32.37	20.67	27.09	28.27	28.13	37.70
IG+AG+VW	32.37	20.67	27.04	28.36	28.22	37.70

IG: indentation guard; AG: pre-existing-copy guard; VW: variable-height window. Missed among targets needing re-check; wrong/silent among all targets; false request among preserved targets; extra requests per 100 preserved targets relative to the deployed cascade; range: exact file/start/end on surviving targets.

is intrinsic to similarity thresholding and is the central practical decision when deploying the method.

D. Cross-Project Calibration

In leave-one-project-out calibration over the nine projects, choosing the threshold that maximizes accuracy on eight projects selects $\theta = 0.55$ in all 9 folds, both for the threshold alone and for the full factorial cell, and for both training objectives (Table XIX). The selection is stable across projects. Applied to public edit-targeted histories, $\theta = 0.55$ relocates 2,037 of 2,454 targets (83.0%), against 82.6% at the archived calibrated value of 0.50 and 66.8% at the default. These histories had been inspected in earlier revisions, so this is retrospective transfer evidence, not a prospective held-out result.

E. Safety-Constrained Calibration Did Not Transfer

Calibrating for accuracy is misaligned with the stated objective, so we also chose thresholds on the four internal projects to maximize accuracy subject to a missed re-check rate of at most α and a deleted-reported-present rate of at most β, for $\alpha \in \{1, 2, 5\}\%$ and $\beta \in \{5, 10, 20\}\%$. Of the design and limit combinations, 10 were feasible on the internal projects and 8 were not. Evaluated once on the public projects, 8 of the 10 feasible choices violated at least one of their own limits. For example, $\theta = 0.65$ met an internal 20% cap on deleted code reported present but reached 24.2% on public edit-targeted observations. Empirical rates from one group of projects therefore do not bound the rates in another; a deployment that needs a guaranteed limit would require a calibration method with explicit transfer assumptions, and the projects here do not justify exchangeability.

F. Choosing a Threshold in Practice

For an institution with no history of its own, the evidence supports three steps. Keep the local search scope and the deployed tier order, which are favoured on safety grounds in every stratum. Treat θ as a policy choice between relocating edited code and reporting deleted code honestly: $\theta \approx 0.55$ maximized accuracy in every held-out project, whereas the default 0.85 favors honest removal. Monitor the outcomes after deployment, since supervisors' own corrections of misplaced anchors are the data a local calibration would need.

XI. RQ4: STRESS CONDITIONS AND SAFETY GUARDS

The controlled benchmark samples anchors that are long and unique enough to be found again, and its twelve edit types each receive equal weight. Its aggregate of 98.5% is therefore a score on a balanced benchmark of single, controlled transformations, not an estimate for naturally occurring revisions; on its four content-changing edit types alone the default reaches 96.4%. This section deliberately constructs harder conditions. Their rates describe the constructed cells and are not estimates of how often such cases occur; real-history prevalence is given by Section IX. All stress inputs were generated, and the guards' code fixed, before evaluation on the held-out seed (protocol in the artifact).

A. Indentation

Normalization hides indentation (Section III). To measure the consequence we constructed Python edits that preserve every normalized line but change nesting: a statement dedented out of its block, statements absorbed into the preceding block, one inner line of a multi-line anchor changing level, and a block moved one level with its header. Every generated file parses, and an edit counts as behavior-changing when the abstract syntax tree of the enclosing scope changes. The deployed cascade requested no re-check for any of these: 1,200 of 1,200 behavior-changing edits kept their state. With the indentation guard none did (0 of 1,200). The guard's cost appears on layout-only edits whose syntax tree is unchanged: it requested re-check for every continuation-line re-alignment (300 of 1,200 layout-only edits overall).

The same experiment exposes a defect in the original benchmark. Its *re-indent* operator, scored as an unchanged success, indents an anchor by four spaces without regard to syntax; of 300 trials on the main seed, 211 produced files that no longer parse and 40 produced parseable files with a changed syntax tree (replication: 186 and 47 of 300). Its 100.0% score measures normalized-text relocation, not safe handling of re-indented Python. We retain the original scores for comparability and report this qualification with them.

Real histories contain few such cases. Among all retained observations, 46 returned spans match the old quotation after normalization but differ in indentation, and at the oracle's own locations 27 do, all of them with unchanged statement trees. Indentation changes that alter behavior are thus rare in the studied histories, and their real-history frequency, unlike their construction, is not established here.

B. Duplicated, Short and Low-Information Anchors

Supervisors often annotate short statements such as `return result`, an import or a repeated validation line. We constructed two stress suites. In the duplicate suite an identical copy of the anchored code is inserted near or far, above or below, or in another file, while the original is kept, edited, deleted or moved. In the short suite, one-line anchors are stratified by how often their normalized text recurs in the file and passed through the original edit types.

Duplicates are the cascade's weakest condition (Table XX). The deployed cascade missed 89.7% of the needed re-checks

and placed 69.3% of the issues on the wrong code without a request; tier 2 chooses the copy nearest the old start, and a copy inserted just above the original occupies exactly that position. The pre-existing-copy guard reduced these to 20.7% and 27.1%. It cannot help when the copy and the move are introduced in the same revision. One-line anchors fared better but still missed 70.5% of needed re-checks, concentrated in the most frequently recurring lines. These results support a design change at annotation time as well as at relocation: when a supervisor anchors a line whose normalized text recurs in the file, the interface should ask for a longer selection.

C. Guard Costs on Real Histories

Each guard requests re-checks that the deployed cascade would not. Per 100 content-preserved real-history references, the indentation guard added 0.24 requests and the pre-existing-copy guard 1.27 at the default threshold, rising to 4.39 and 12.60 on public edit-targeted references; the variable-height window added none. On the 37 archived missed re-checks, the pre-existing-copy guard left 7 unrequested and the combination of all three guards at $\theta = 0.55$ left 1. Because these cases motivated the guard, this is a mechanism check, not held-out evidence; the held-out evidence is the stress suites above and the external benchmark in Section VIII.

D. Variable-Height Windows

Fixed-height windows cannot follow statements that grow or shrink (Section VII-H). With the variable-height window, public edit-targeted accuracy at the default threshold rose from 66.8% to 72.8% and exact file/start/end recovery on surviving targets from 62.0% to 64.6%, with no additional re-check requests.

E. Compound Edits

A resubmission rarely applies one transformation. We composed $k \in \{1, 2, 3, 4\}$ distinct operators on the same anchor: a move within the file or to another file, an identifier rename inside the anchor, a layout-only wrap or re-alignment, an inserted or deleted inner statement, statements inserted directly above, and a token edit nearby. Lines carry construction tags, so the anchor's final location is known by construction, and an anchor counts as changed when the syntax trees of its statements differ. Every edited file parses. To keep moves and insertions parseable, anchors were restricted to complete runs of sibling statements; this and the rejection of inapplicable operator combinations make the cells a constructed stress test, not a sample of revisions. Each cell has 300 trials per seed on two seeds (600 per k pooled; Table XXI, Fig. 12).

Placement accuracy of the deployed cascade fell from 82.5% for one operator to 52.8%, 25.8% and 11.5% for two, three and four. The cascade failed by losing anchors, not by misplacing them: the lost share rose from 6.8% to 82.3%, while silent misplacement stayed at or below 0.7% and missed re-checks at or below 3.7%. A lost anchor returns its issue for re-check, so the composite failure is extra supervisor work rather than hidden error. The contextual tracker placed more starts from three operators on (55.3% and 41.8%), but returns no range and never follows code into another file. The three guards together raised placement to 96.5%–18.0% and range recovery to 94.7%–17.3%, mainly through the variable-height window, at a large cost in requests on preserved code: 69.3% of the single-operator preserved anchors were returned, against 13.2% for the deployed cascade. Two causes dominate, and both concern the pre-existing-copy guard. By design it flags every move within a file, because a move does not increase the number of copies. It also flagged every pure insertion directly above an anchor: the pinned diff mapping places the expected start of an anchor whose preceding lines were inserted at the first inserted line, so the unmoved anchor never sits at its expected start. The insertion fix (AGi, Section III-D) removes this second cause. On a further compound seed generated after the fix was specified, AGi lowered the requests on single-operator preserved anchors from 58.5% to 37.2% (all guards: 71.3% to 50.0%) without a single additional missed re-check or silent misplacement. A stronger locator does not remove the decline with k: on the held-out seed alone (Table XXIV; Table XXI pools both seeds), at four operators CodeMapper placed 12.0% of the anchors and the contextual tracker 43.0%, against 10.7% for the deployed cascade. Under every guarded locator, the compound failures were lost anchors, which return their issue, rather than silent ones (Section XII).

TABLE XXI
COMPOUND EDITS: BOTH SEEDS POOLED

k	Method	Place	Range	Missed	Silent	Lost
1	Deployed cascade	82.5	74.0	3.7	0.2	6.8
	All guards, $\theta = 0.85$	96.5	94.7	0.0	0.0	2.0
	All guards, $\theta = 0.55$	94.8	92.3	0.0	0.0	0.0
	Cascade v1	84.8	0.0	3.7	0.2	1.7
	Contextual tracker	86.8	0.0	2.1	0.0	12.3
	Diff drift	31.5	0.0	0.0	0.0	68.5
2	Deployed cascade	52.8	40.5	2.0	0.7	36.0
	All guards, $\theta = 0.85$	68.5	65.8	0.0	0.0	27.5
	All guards, $\theta = 0.55$	67.3	63.8	0.0	0.0	16.7
	Cascade v1	66.0	0.0	2.3	0.7	12.2
	Contextual tracker	69.7	0.0	0.3	0.0	29.3
	Diff drift	5.7	0.0	0.0	0.0	94.3
3	Deployed cascade	25.8	19.7	0.4	0.2	66.5
	All guards, $\theta = 0.85$	35.8	34.3	0.0	0.0	61.2
	All guards, $\theta = 0.55$	37.8	35.8	0.0	0.0	37.3
	Cascade v1	45.2	0.0	0.9	0.5	31.2
	Contextual tracker	55.3	0.0	0.0	0.0	44.3
	Diff drift	0.0	0.0	0.0	0.0	100.0
4	Deployed cascade	11.5	9.0	0.0	0.0	82.3
	All guards, $\theta = 0.85$	18.0	17.3	0.0	0.0	78.5
	All guards, $\theta = 0.55$	19.2	17.7	0.0	0.0	46.2
	Cascade v1	27.3	0.0	0.2	0.2	42.8
	Contextual tracker	41.8	0.0	0.0	0.0	57.8
	Diff drift	0.0	0.0	0.0	0.0	100.0

Percentages. Place: file and start among surviving targets. Range: exact file/start/end (blank for line-only methods, scored as incorrect). Missed: changed targets without a re-check request. Silent: wrong placement without a request.

F. Multi-Anchor Issues

The deployed rule returns an issue for re-check when any of its anchors changes or disappears. Its consequences grow

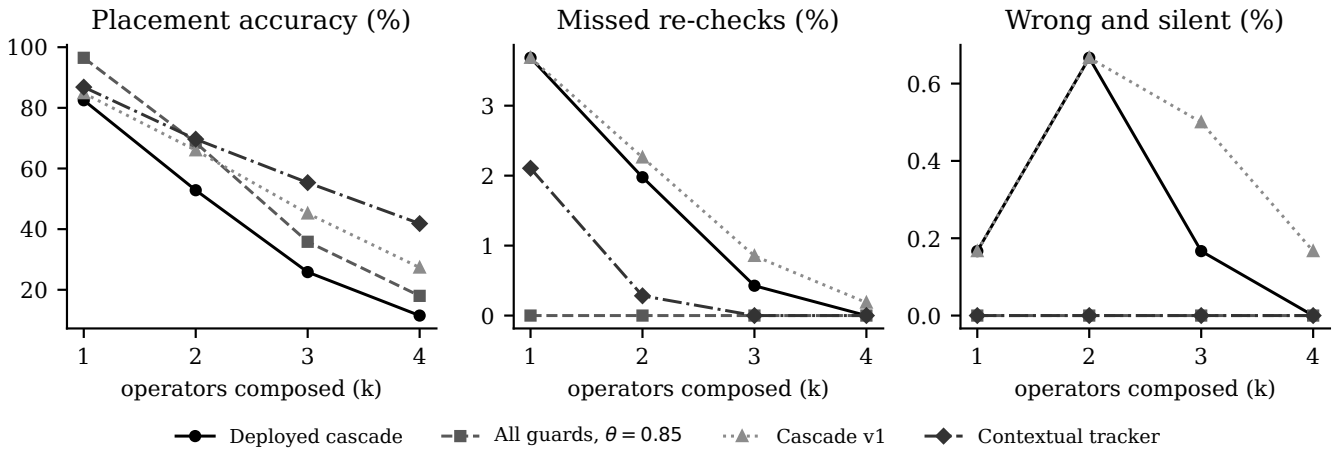


Fig. 12. Compound edits: placement, missed re-checks and silent misplacement against the number of composed operators, both seeds pooled.

with the number of anchors. We grouped k distinct real-history anchors from the same commit pair into synthetic issues (14,830 distinct observations; the first anchor is the primary). The grouping is synthetic: anchors from one commit pair need not belong to one concern. With one anchor, 17.5% of issues were returned; with two, three, five, eight and twelve anchors, 31.7%, 39.6%, 60.7%, 76.8% and 97.0% were (337 twelve-anchor groups from 337 commit pairs). With two anchors, 42.5% of the returned issues were returned only because of a supporting anchor, rising to 71.1% with eight; a policy that watched the primary anchor alone would have missed 43.0% of the two-anchor issues whose code had changed and 71.3% of the eight-anchor ones, against at most 1.3% for the any-anchor rule. The any-anchor rule is therefore necessary for safety, but on many-anchor issues it returns nearly every issue after an edit-heavy revision. The interface should show which anchor triggered the request, as the studio's per-anchor notes already do, and supervisors should prefer few, precise anchors per issue.

G. Ten Successive Revision Hops

Pairwise relocation does not test the effect of carrying an updated anchor repeatedly. We replayed all 4,000 original horizon-10 observations across their intervening first-parent commits: 367 intervals and 3,959 distinct interval/anchor-start targets, retaining repeated samples. Each comparison uses the same initial anchor, final revision and archived endpoint reference. The direct route makes one initial-to-final call; the sequential route invokes the pinned code-anchor carry function ten times, feeding its returned quote, digest and coordinates into the next hop with the actual previous file map. Both thresholds are unchanged. Direct labels, outcomes and coordinates reproduce the archived and range-run evidence. Intermediate no-match results do not stop or exclude a chain. A local exploratory protocol preceded execution; it was not externally registered.

Endpoint accuracy changes little in this selected sample (Table XXII). At default, 3,569 observations are correct directly and 3,575 sequentially: eight become correct and two become incorrect. At the previously calibrated threshold, the counts are 3,770 and 3,775: eight gains and three losses. On public edit-targeted observations, the sequential-minus-direct differences are 0.4 points, 95% interval $[-0.2, 1.1]$, and 0.2 points, interval $[-0.4, 0.9]$, respectively. Intervals use 2,000 paired commit-interval resamples, seed 20261028. No equivalence margin was specified, so small net changes do not establish drift-free operation. Exact-range correctness on the 3,961 eligible surviving endpoints increases by only three observations at default and one after calibration.

The trace labels expose a different question from endpoint accuracy. At default, 82 endpoint errors end with an identical or content-match last-hop label; 74 of these have an earlier changed/removed request. After calibration, 188 errors end with such a label and 184 have an earlier request. Under an explicit no-faculty-action simulation, the OR of all requests therefore preserves information that the latest label alone omits. It is not complete protection: ten default endpoint errors and five calibrated errors have no re-check request anywhere in the chain. These counts are neither new semantic-error labels nor faculty workload estimates, and the replay does not exercise database lifecycle or human decisions.

A source-checked Flask trace illustrates the mechanism [46]. Its original negative assertion is present at line 491 in both endpoint revisions. At hop 7, fuzzy matching selects a nearby positive assertion and replaces the stored quote; when the negative assertion returns, the updated quote follows the positive one to line 494 and later receives content-match labels. Direct relocation finds line 491. The earlier re-check request is retained by the cumulative indicator, but the final local content-match label cannot certify the original referent. This is a diagnostic example selected after the comparison, not independent adjudication of every intermediate identity. The complete revisions, output coordinates and quote hashes are retained.

H. Repeated Resubmissions

Section XI-G compared one direct relocation with ten successive ones only at the tenth commit. To see how errors

TABLE XXII
ENDPOINT FILE/START ACCURACY: DIRECT RELOCATION VERSUS TEN SUCCESSIVE CARRIES

Project group	Sample	n	Default $\theta = 0.85$ Direct %	Default $\theta = 0.85$ Ten-hop %	Previously calibrated $\theta = 0.5$ Direct %	Previously calibrated $\theta = 0.5$ Ten-hop %
In-house	uniform	503	99.0	99.0	99.6	99.6
In-house	edit-targeted	292	69.5	70.2	82.2	83.2
Public	uniform	2,228	99.5	99.5	99.6	99.6
Public	edit-targeted	977	66.7	67.1	82.9	83.1

Each row compares routes on the same original-to-final endpoint references, including correct no-match for gone targets. All 39 gone endpoint references are retained. Intermediate identity is not independently labeled; these are Git-history replays, not observed student resubmissions.

accumulate, we re-ran the original syntax-tree reference from each chain's first commit to commits 1, 2, 3, 5 and 10, with the original exclusions; recomputed at commit 10 it reproduced every archived reference, and 3,996 of the 4,000 targets were decidable at every prefix (Fig. 13). The intermediate references are the same heuristic, not independent identity labels.

Successive and direct relocation stayed within a few tenths of a point of each other at every prefix. The figures in this paragraph are for the 3,996 targets decidable at every prefix; Table XXIII uses all 4,000 targets decidable at commit 10, so its counts differ slightly. Endpoint accuracy of the deployed cascade fell from 99.0% after one commit to 96.5%, 94.1% and 89.4% after three, five and ten, mostly through lost anchors (8.4% at ten commits) rather than wrong placements (2.3%). Re-check requests persist in the issue's state, so the relevant safety measure is cumulative: of the 986 targets whose code had changed by commit 10, 0.8% had never been returned, and 0.3% of all targets sat on wrong code without any request. The last hop's label alone is a poor guide: a changed hop rewrites the stored quotation, so later hops compare against the rewritten text, and the final label failed to signal 56.9% of the changes since the original. In 82 chains the endpoint was wrong while the last label read identical or content-match. The issue state, not the latest label, is what keeps such cases in front of the supervisor; once a supervisor clears a re-check, later labels describe change since the last cleared version only.

The calibrated threshold traded these failures differently. At $\theta = 0.5$, accuracy after ten commits was 94.4%, but wrong placements reached 5.2%, against 2.3% at the default, and 4.8% of the targets sat on wrong code under a final label that requested nothing (0.1% had never been returned). Over repeated resubmissions the low threshold's misplacements accumulate, while the cumulative request history still catches nearly all of them. The three guards at the default threshold reduced cumulative missed re-checks to 0.1% and silent misplacement to 0.0% after ten commits, with accuracy 91.8%.

The identity rule (Section III-E, Table XXIII) closes the gap between the last label and the history. Replayed over the same chains with the default cascade, it reduced the changes since the original that the tenth label failed to signal from 563 to 8, the number never returned at any hop, and the endpoints on wrong code under a final label that requested nothing from 85 to 10. The latest request now carries the history. With the recommended guards (IG+AGi+VW) at the calibrated threshold, the rule cut the unsignaled changes from 858 to 3 and the silent wrong endpoints from 93 to 4; AG2 in place of AGi reached the same endpoints (97.0% correct) but left 1 change never returned, against 0. At the default floor of 0.5 the rule moved no cascade endpoint at either threshold, and a floor of 0.3 gave the same results. A floor of 0.7 left the default threshold unchanged but, at the calibrated one, turned changed hops less than 0.7 similar to the reviewed text into losses: correct endpoints 96.1% against 97.0%, wrong 2.0% against 2.5%, lost 1.9% against 0.5%. The rule matters for a stronger locator. CodeMapper replayed hop by hop placed 97.7% of the endpoints after ten commits, against 89.4% for the deployed cascade, with 2.0% on wrong code; with the rule, wrong placements fell to 0.9% and lost anchors rose from 0.3% to 2.0%, so uncertain transitions became visible losses instead of silent drift. With IG+AGi over CodeMapper, no change since review went unreturned at any hop (0, against 3 with labels only). The identity rule does not decide identity; it records which transitions were uncertain and keeps them in front of the supervisor.

TABLE XXIII
TEN SUCCESSIVE COMMITS: IDENTITY RULE

Method	Correct	Wrong	Lost	Never returned	Last silent	Wrong, silent
Cascade	89.4	2.2	8.4	8	563	85
+ identity rule	89.4	2.2	8.4	8	8	10
Guarded, θ=0.55	97.0	2.5	0.5	0	858	93
+ identity rule	97.0	2.5	0.5	0	3	4
CodeMapper, labels	97.7	2.0	0.3	3	865	71
+ identity rule	97.0	0.9	2.0	3	3	3
CodeMapper, guarded	97.7	2.0	0.3	0	865	71
+ identity rule	97.0	0.9	2.0	0	3	3
CodeMapper, direct	97.7	2.0	0.3	0	0	0

Endpoint after ten commits on 4,000 targets decidable at commit 10; correct, wrong and lost in %. Never returned and last silent count, of the 989 targets whose code had changed since the first commit, those never returned at any hop and those not returned by the last hop's request; wrong, silent counts endpoints on wrong code with no request at the last hop. Guarded: the cascade with IG+AGi+VW; CodeMapper with IG+AGi (placement as labels only, by construction). CodeMapper is replayed hop by hop; direct maps the first version to the last, guarded, and is shown for reference.

Repeating CodeMapper on the same chains exposed a reproducibility limit. For the AGi replay every CodeMapper trace was recomputed, and 3 of the 4,000 chains differed from the first run (6 traces or direct mappings); re-running those chains in one process returned different regions for identical inputs. The pinned CodeMapper is therefore not fully deterministic. The cascade reproduced exactly. All CodeMapper rows of Table XXIII come from the second run, so its guarded

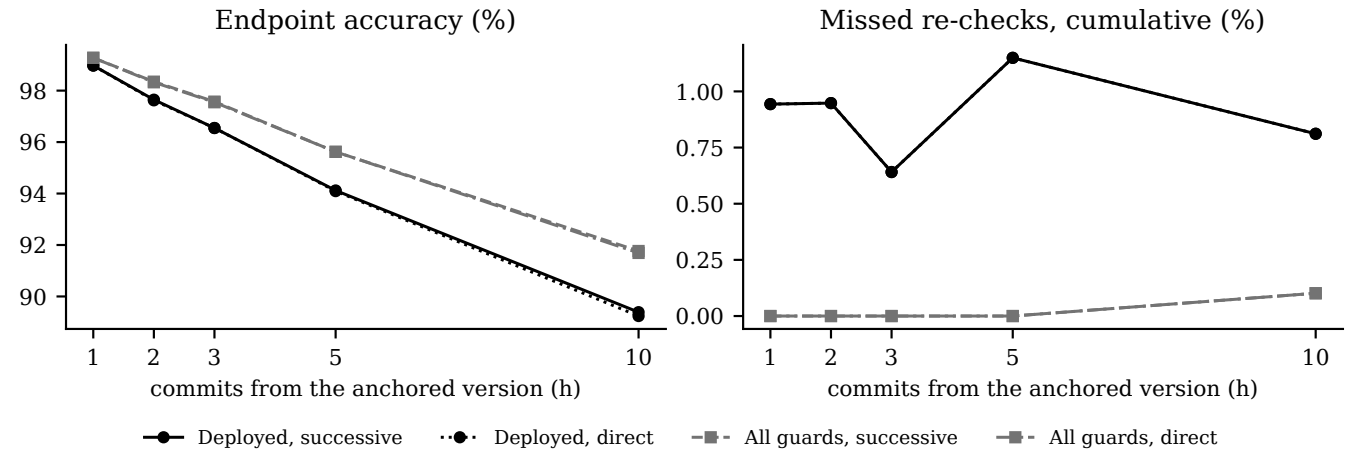


Fig. 13. Accuracy and cumulative missed re-checks after h commits, successive versus direct relocation, on the 3,996 targets decidable at every prefix.

and unguarded rows are paired.

I. Scale

For an anchor of s lines and q characters in a file of L lines, in a version of N lines in total, the tiers cost: a digest comparison (tier 1); $O(L\,s)$ line comparisons for the exact run search (tier 2); a line diff of the old and new file, quadratic in L in the worst case, plus $2r+1$ window comparisons with $r = \max(10, 2s)$, each quadratic in q in the worst case (tier 3); and $O(N\,s)$ for the cross-file search (tier 4). The pinned code recomputes the normalized lines of every file for each anchor that reaches tier 4, so a composition of A such anchors costs $O(A\,N\,s)$; caching normalized files per composition would remove the factor A from the file scan.

We timed composition on two large public trees, Django (883 files, 157,865 lines) and the CPython standard library (1,722 files, 877,535 lines), relocating 1,000 anchors sampled from files changed between consecutive releases (pinned commits in the artifact). The deployed cascade needed a median of 0.1 ms per anchor on Django and 0.2 ms on CPython (95th percentiles 0.2 and 1.1 ms), and 0.3 s and 4.6 s for the 1,000-anchor composition. Anchors found nowhere dominate: on CPython the 21 anchors labeled removed took 89% of the composition time. The adversarial case, a quotation found nowhere, forces the full cross-file scan and took a median of 36 ms per anchor on Django and 162 ms on CPython. The research implementation of the three guards is not optimized (median 1.7 and 4.1 ms per anchor). Timings come from one workstation and are descriptive; a capstone submission is far smaller than either tree.

XII. RQ5: A Stronger Locator Inside the Safety Workflow

Sections VIII and VIII-B show that the cascade is not the best locator. Its change labels, two of its three guards and its issue states, however, do not depend on how a window was found. The indentation guard and the copy guard inspect only the returned window; the variable-height window belongs to the cascade's own search and has no counterpart over another locator. This section tests that separation directly: can the safety-oriented workflow improve the reliability of an already strong locator? We placed CodeMapper [8], the most accurate locator in our comparisons, and the contextual tracker inside the workflow, and then a language model. Each locator returns a window or no match. The window passes through the quotation comparison that assigns the cascade's labels (*labels only*: content-match or moved when the normalized quotation is found unchanged, changed otherwise, removed on no match) and then through the indentation guard and the corrected copy guard, IG+AGi. The guards never move a window, so placement stays the locator's own and only the requests change. Because CodeMapper sees one file pair at a time, we also evaluated it with the cascade's cross-file search as a fallback when it reports no match.

The protocol, adapters and hypotheses were recorded with code hashes before any guarded output was computed (protocol I in the artifact), and each amendment was recorded before the outcomes it concerns. The guarded locators were evaluated on the real histories, the student-authored repositories, the human-annotated benchmark and a new stress seed generated after the protocol was fixed, which holds out 23,550 indentation, duplicate, short-anchor and compound cases. AGi was specified after the student and compound results of an earlier copy-guard revision had been read, so for AGi only the new seed is held out. Table XXIV gives the results.

The safety layer made the strong locator safer. With labels only, CodeMapper left 19.3% of the needed re-checks unrequested on the held-out seed and placed 7.0% of the targets on wrong code without a request. With IG+AGi these fell to 3.1% and 3.7% (paired cluster interval of the change in missed re-checks -17.5 to -15.0 points). On the student repositories missed re-checks fell from 9 to 2 of 1,770 and silent misplacements from 13 to 6; on the real histories both fell from 12 to 1; on the human-annotated regions missed re-checks fell from 4 to 3 of 130. The contextual tracker responded in the same way. The gain has two sources. IG catches indentation changes that leave the normalized text intact, which neither

TABLE XXIV
LOCATORS WITH AND WITHOUT THE SAFETY LAYER (%)

	Real histories n=15,064						Student repositories n=7,882						Human-annotated n=200						Held-out stress n=23,550					
Method	Place	Miss	Silent	Misdir	Lost	Extra	Place	Miss	Silent	Misdir	Lost	Extra	Place	Miss	Silent	Misdir	Lost	Extra	Place	Miss	Silent	Misdir	Lost	Extra
Cascade																								
deployed	91.8	1.3	0.3	1.6	6.4	0.0	91.8	3.1	1.5	1.7	5.5	0.0	62.2	8.5	3.5	9.0	27.6	34.3	66.0	71.8	37.1	0.9	3.3	0.9
+IG+AGi+VW	93.7	0.1	0.0	1.0	5.4	0.7	93.2	0.2	0.4	2.0	4.9	3.8	67.3	3.8	0.5	9.0	26.3	44.3	69.7	4.9	5.2	29.9	3.0	32.8
+IG+AGi+VW, θ=0.55	97.8	0.0	0.0	2.1	0.4	0.7	96.6	0.1	0.4	3.0	0.7	3.8	87.2	3.8	0.5	21.0	1.9	44.3	69.6	4.7	5.0	32.6	1.4	34.0
CodeMapper																								
labels only	97.9	0.4	0.1	1.5	0.5	0.8	92.4	0.5	0.2	3.6	3.6	6.1	93.6	3.1	0.0	7.5	1.3	35.7	83.0	19.3	7.0	4.6	10.6	13.4
+IG+AG	97.9	0.0	0.0	1.6	0.5	1.9	92.4	0.1	0.1	3.6	3.6	8.3	93.6	0.8	0.0	7.5	1.3	50.0	83.0	2.0	3.0	8.6	10.6	38.0
+IG+AGi	97.9	0.0	0.0	1.6	0.5	1.3	92.4	0.1	0.1	3.6	3.6	7.4	93.6	2.3	0.0	7.5	1.3	44.3	83.0	3.1	3.7	7.9	10.6	37.0
CodeMapper, cross-file fallback																								
labels only	98.3	0.4	0.1	1.5	0.1	0.3	95.2	1.1	0.8	3.6	0.3	2.0	93.6	3.1	0.0	7.5	1.3	35.7	85.7	22.3	8.7	4.6	7.2	8.8
+IG+AGi*	98.3	0.0	0.0	1.6	0.1	1.0	95.2	0.1	0.1	4.2	0.3	4.9	93.6	2.3	0.0	7.5	1.3	44.3	85.7	4.7	4.6	8.7	7.2	33.3
Contextual tracker																								
labels only	95.6	0.3	0.1	0.6	2.9	1.7	91.9	0.5	0.2	1.2	7.2	7.2	82.7	3.8	0.0	3.0	14.7	35.7	82.0	22.0	7.7	0.1	11.8	13.2
+IG+AGi	95.6	0.0	0.0	0.7	2.9	2.1	91.9	0.1	0.1	1.2	7.2	8.9	82.7	3.1	0.0	3.0	14.7	44.3	82.0	3.3	4.1	3.7	11.8	37.9

Place: correct start among targets that survive. Miss: changed or deleted targets not returned for re-check. Silent: targets placed on wrong code without a request, over all targets. Misdir: targets placed on wrong code with a request, over all targets, including deleted code relocated onto other code. Lost: surviving targets with no location. Extra: requests on targets whose content is preserved. *Labels only* passes a locator's window through the quotation comparison that assigns the label; IG and AGi never move a window, so the other locators' placement is unchanged, whereas the cascade's guarded rows also include VW, which changes window height and placement. Every locator sees the anchor's original path only (no Git rename hint). Real histories and student repositories are Python; the human-annotated regions span several languages and are seen as file pairs, so cross-file recovery is not exercised there. Held-out stress is a fresh seed of the indentation, duplicate, short-anchor and compound suites generated after the protocol was fixed. The pinned CodeMapper is not fully deterministic (Section XI-H); its guarded rows reuse the windows of its unguarded rows, so the two are paired. *Composed after outcomes were read (exploratory).

locator reports: CodeMapper missed 85.0% of the behavior-changing ones, the guarded version 0.0%. AGi catches windows that land on an identical copy. The price is paid on preserved code: extra requests rose from 13.4% to 37.0% on the stress seed, where most cases are adversarial, and from 6.1% to 7.4% on the student repositories.

The corrected copy guard costs less but does not dominate. Over CodeMapper, AGi asked for fewer unnecessary re-checks than the original copy guard on every population (stress seed 37.0% against 38.0%; student repositories 7.4% against 8.3%; real histories 1.3% against 1.9%). It missed more only where an identical copy sits just below the original, the case in which the line diff aligns the original's lines with the copy: on the duplicate suite 18.5% against 11.9%, which accounts for the whole difference on the stress seed, and 3 against 1 misses on the human-annotated regions. The registered hypothesis that AGi lowers unnecessary requests without losing safety therefore held on the held-out short-anchor and compound suites but not on the held-out seed as a whole. The provenance rule AG2 (Section III-D) removed more unnecessary requests but failed exactly where a copy guard is needed: on the held-out duplicate suite the cascade with AG2 missed 89.4% of the needed re-checks, against 29.2% with AGi and 89.7% with no guard. We recommend AGi, keep the original guard where duplicates are expected, and report AG2 as a negative result.

A language model as the locator. Magic Markup re-anchors annotations with a language model [15], so we also placed one inside the workflow. The protocol was registered before any evaluation call, and its amendments, including a change of model made before that model's first evaluation call, are in the artifact. The model is Beam 1.0, the model the deployed portal is configured with, at temperature 0. For each anchor it receives the old region with 15 lines of context and the complete new version of the paired file. It returns where the region now is and whether it changed. Its own answer can decide the request (*own judgment*), or its window can pass through the same labels and guards as CodeMapper's (Table XXV). Only public code was sent: the human-annotated regions, the open-source edit-targeted histories and the student repositories.

The model located well, but its own judgment of change was not a safe signal. It placed 94.4% of the surviving open-source targets and 90.2% of the student ones. CodeMapper placed 92.2% and 83.2%, and the deployed cascade 65.8% and 73.6%. On the human-annotated regions CodeMapper placed more (93.6% against 91.7%). Judging change by itself, the model still left needed re-checks unrequested: 41 of 1,930 on the open-source histories (2.1%), 7 of 1,402 on the student repositories and 8 of 130 on the human-annotated regions. It also placed 11, 7 and 4 targets on wrong code without a request.

The safety layer made it safe. With IG+AGi the model left 0 needed re-checks unrequested on the open-source histories, 0 on the student repositories and 4 on the human-annotated regions. Silent misplacements fell to 0, 1 and 0. Both registered hypotheses held on every population. The paired cluster intervals of the change in missed re-checks were -3.0 to -1.4, -0.9 to -0.2 and -6.4 to -0.7 points (J1), and those for silent misplacements also excluded zero (J2). On the human-annotated regions the result rests on few discordant regions, and the exact paired tests were not significant ($p = 0.125$ and $p = 0.125$). As a secondary check, Holm's adjustment over the 6 exact J1 and J2 tests leaves only the open-source pair below 0.05 (adjusted p of <0.001 and 0.005); the student tests, which also rest on few discordant targets, give 0.063 and 0.094. The cost fell on preserved code: extra requests rose from 11.3% to 18.3% on the open-source histories and from 25.3% to 40.4% on the student repositories. A second model run

TABLE XXV
A LANGUAGE-MODEL LOCATOR WITH AND WITHOUT THE SAFETY LAYER (%)

	Human-annotated n=200					Open-source, edit-targeted n=2,454					Student, edit-targeted n=1,857				
Method	Place	Miss	Silent	Misdir	Extra	Place	Miss	Silent	Misdir	Extra	Place	Miss	Silent	Misdir	Extra
Language model (Beam 1.0)															
own judgment	91.7	6.2	2.0	3.0	35.7	94.4	2.1	0.4	3.1	11.3	90.2	0.5	0.4	4.6	25.3
labels only	91.7	6.2	1.5	3.5	37.1	94.4	0.4	0.3	3.2	10.5	90.2	0.3	0.3	4.7	25.9
+IG+AGi	91.7	3.1	0.0	5.0	45.7	94.4	0.0	0.0	3.5	18.3	90.2	0.0	0.1	5.0	40.4
Second model (Nemotron 3.5 Super)															
own judgment	88.5	5.4	1.5	3.0	35.7	95.3	0.6	0.5	2.0	10.5	91.8	0.6	0.3	3.4	25.3
+IG+AGi	88.5	2.3	0.0	4.5	44.3	95.3	0.0	0.0	2.4	18.9	91.8	0.0	0.1	3.7	40.4
CodeMapper															
labels only	93.6	3.1	0.0	7.5	35.7	92.2	0.4	0.3	5.5	10.1	83.2	0.3	0.3	10.6	29.2
+IG+AGi	93.6	2.3	0.0	7.5	44.3	92.2	0.0	0.0	5.9	17.6	83.2	0.0	0.1	10.8	40.9
Cascade															
deployed	62.2	8.5	3.5	9.0	34.3	65.8	0.9	0.8	5.7	0.0	73.6	2.6	2.3	6.1	0.4

Same observations for every row. The language model (Beam 1.0, the model of the deployed portal; temperature 0) sees the old region with context and the complete new version of the paired file. *Own judgment*: the model's answer of whether the region changed decides the request. *Labels only* and *+IG+AGi*: the model's window passes through the quotation comparison and the window guards, as for CodeMapper; placement is unchanged. Unanswerable prompts (file too large for the context, provider refusal) count as execution errors and request re-check. The second model received the same prompts and settings; its window is placed by its own answer, so its placement differs. Columns as in Table XXIV.

on all three populations, Nemotron 3.5 Super, judged change more cautiously than Beam 1.0 on the open-source histories (12 against 41 missed re-checks of 1,930) and placed about as many targets (95.3% of the open-source and 91.8% of the student ones). Under IG+AGi it too left 0 needed re-checks unrequested on the open-source histories, 0 on the student repositories and 3 of 130 on the human-annotated regions; J1 was supported, supported and supported, and J2 supported, supported and not supported on the three populations. The safety layer therefore did not depend on which model located the code, and a model with a better judgment of change narrowed, but did not close, the gap the layer closes. Two other models, run on the human-annotated regions only, moved in the same direction. Missed re-checks fell from 7 to 4 of 130 for Step-3.7-Flash (J1 not supported) and from 8 to 4 for Step 5 Preview (J1 supported). Temperature 0 did not make the model deterministic: a repeat of the human-annotated run gave the same placement, range and label for 184 of 200 regions. These are dated observations of free models served by third parties.

Which locator to deploy. CodeMapper placed more anchors than the deployed cascade on the held-out seed (83.0% against 66.0%), the human-annotated regions (93.6% against 62.2% of surviving regions; Section VIII also counts correctly reported deletions, 91.5% against 66.0%) and the real histories (97.9% against 91.8%), but it loses anchors that move to another file. With the cross-file fallback it placed 98.3% on the real histories and 85.7% on the stress seed, the most of any method on both, and kept that placement under the guards with 2 missed re-checks of 1,770 on the student repositories; this combination was assembled after the outcomes had been read and is exploratory. The guarded cascade at the calibrated threshold of 0.55 came close on the real histories (97.8%) and placed the most on the student repositories (96.6%), so the ranking of locators depends on the population. The safety layer behaved alike under every locator, and that is the property a deployment needs: the locator can be replaced as better ones appear while the change labels, the two window guards and the issue states, and with them the safety argument, stay the same. The language model is such a candidate. On the same edit-targeted open-source and student rows it placed more targets than CodeMapper or the deployed cascade, and under the same layer it missed as few needed re-checks there as CodeMapper (0 and 0, against 0 and 0). It costs a model call per anchor, with a median of 2.0 s through a free service.

XIII. RQ6: THE IMPLEMENTED STATE CONTRACT

The safety argument of Sections III–IV rests on rules about who may change an issue's state. We checked those rules against the pinned application in three ways: a specification read from the code, a static scan with a bounded model check, and scenario tests that drive the application's own interfaces.

A. Specification

Table XXVI lists the 13 implemented transitions, with actor, source and target state, and the source lines that implement each. Line numbers are located by exact-substring search when the table is generated, so a stale citation fails the build. accepted is a faculty decision to keep an observation as a point the team must address; it is carried like open, and it is neither a student acknowledgment nor the acceptance of a fix. Faculty status edits have no source-state guard: a supervisor may set any state from any state.

B. Static Scan and Bounded Model

A scan of the abstract syntax trees found 26 call sites that create, update, delete, restore or undo a review issue, and no direct assignment to an issue's status. Each was reviewed. The system writers are carry-forward, the composition merge and composition's creators; none writes resolved.

The bounded model encodes the transitions of Table XXVI for two findings and two versions, with the relocation label of every carried issue chosen nondeterministically. Exhaustive search to depth 6 visited 7,179,369 distinct states and

TABLE XXVI
IMPLEMENTED ISSUE-STATE TRANSITIONS AT THE EVALUATED COMMIT

Event	Actor	From → to	Status	Source
create (faculty)	faculty	(none) → open or any valid status supplied	by design	`review_studio.py:258`; `service.py:409`
edit status (faculty)	faculty	any → any	unguarded	`review_studio.py:268`; `service.py:421`
carry-forward creation (system)	system	open / accepted / needs_recheck on version n → same status, or needs_recheck if any anchor is changed or removed	by design	`carry.py:251`; `carry.py:254`
finding raised again on a carried issue (system)	system	open / accepted / needs_recheck → needs_recheck (evidence replaced)	by design	`service.py:623`; `service.py:624`
finding raised again on a carried resolved/dismissed issue (system)	system	resolved / dismissed → unchanged status; evidence replaced	unguarded	`service.py:621`
dismissal-memory creation (system)	system	(none) → dismissed	by design	`service.py:628`; `service.py:633`
new finding (system/partner)	system	(none) → open	by design	`service.py:681`; `service.py:732`
assistant edit status	agent	any → open / accepted / needs_recheck	by design	`mcp_bridge.py:449`; `mcp_bridge.py:452`
assistant proposal applied by faculty	faculty (recorded as agent)	any → resolved / dismissed / other proposed fields	by design	`review_copilot.py:6591`; `service.py:898`
delete (faculty or assistant)	faculty / agent	any → deleted (status kept)	unguarded	`service.py:463`; `mcp_bridge.py:467`
restore / undo (faculty)	faculty	any → any earlier snapshot, including resolved or dismissed	by design	`service.py:479`; `changes.py:118`
undo (assistant)	agent	any → earlier snapshot unless it is resolved or dismissed	by design	`mcp_bridge.py:529`
system writes resolved	system	any → resolved	no code path	`carry.py:7`

"Unguarded": reachable, with no check on the source state. Line numbers refer to the pinned application export; every citation is located by an exact-substring search when the table is generated.

58,122,810 transitions. No system event wrote resolved (0 violations); revision history stayed append-only; repeated composition changed nothing; and the assistant never closed an issue without a supervisor's confirmation. One invariant failed. "An active issue reaches the next version unless a supervisor closes it" was violated on 629,056 transitions, all through deletion: 314,528 by a supervisor and 314,528 by the assistant, with 0 from any other cause. Deletion is journaled and undoable, but a deleted issue is not carried forward. The verified property is therefore narrow: no system event resolves an issue. It is not a guarantee that no concern leaves the workflow. The model abstracts the code as we read it; the scenarios below exercise the code itself.

C. Scenarios Against the Application

The pinned application's own review-state and artifact tests were run first: 82 collected, 80 passed, 1 failed and 1 skipped. The failure needs an external partner analyser that is not part of the export, and the skip needs a submodule pin; neither concerns review state. We then wrote 8 scenarios that submit versions, import signed analysis bundles and compose reviews through the application's interfaces (Table XXVII). All 8 ran as recorded characterizations; the result column states whether the stated claim held.

Five claims held. A three-version chain was labeled moved, changed and removed in turn; the issue returned for re-check after the change and kept its original quotation through lineage; a voice note stayed linked to it, and an image note followed its code until the code was removed, then became a document-level note with an explanation. A change to a supporting anchor alone returned a multi-anchor issue for re-check. Repeated composition was idempotent. On PostgreSQL 17.10, a second composer's carry-forward was held until the first committed its locked version and produced no duplicates; an interrupted composition left no review document. SQLite does not enforce the row lock, so only this scenario tests locking, at service level with an in-memory file loader.

TABLE XXVII
STATE-CONTRACT SCENARIOS RUN AGAINST THE PINNED APPLICATION

Claim checked	Engine	Result
Concurrent composition of one version creates no duplicates; an interrupted composition leaves no partial state.	PostgreSQL	holds
A dismissed finding raised again on a later version arrives dismissed, keyed by category and normalized title (earlier draft).	SQLite	does not hold
An issue returns for re-check when any of its anchors changes, including a supporting one.	SQLite	holds
Encoding, newline and size variants of unchanged code keep their anchors.	SQLite	does not hold
Composing a version repeatedly creates no duplicate issues or attachments.	SQLite	holds
A finding raised again after faculty resolved it creates a new issue; the resolved issue's history stays visible.	SQLite	holds
Over three versions an anchor is labeled moved, changed, removed; the issue returns for re-check after the change; voice and image notes follow it.	SQLite	holds
An open issue reaches every later version until faculty close it.	SQLite	does not hold

Three claims did not hold, and the design text has been

corrected accordingly.

Dismissal memory compares a newly raised finding with the findings dismissed on the immediately preceding version only, by category and the first 200 normalized characters of the description's first paragraph; issues written by a supervisor or the assistant are keyed by title. A different finding with the same opening paragraph was suppressed; a finding absent from one version and raised two versions later arrived open; and a supervisor's dismissal of a hand-written issue did not suppress the detector's finding on the same code. The key is a heuristic, not an identity.

Carry-forward reads only the preceding version's review. When an intermediate version had never been composed, its successor received none of the earlier issues until the intermediate review was composed and the successor recomposed. In normal operation every version is composed after analysis, so this needs a failed pipeline.

Normalization. CRLF/LF changes and a removed final newline kept anchors content-match. A byte-order mark changed an anchor on the first line only. A file with an invalid UTF-8 encoding rejected the whole upload rather than losing anchors. An unchanged file of 464,098 bytes exceeded the carry size limit, so its anchor was labeled removed and returned for re-check: a false re-check, not a silent error.

The scenarios also recorded two behaviors the earlier text did not describe. When analysis raises a finding again on a carried issue, the issue absorbs it: its evidence is replaced and an active issue returns for re-check, while a resolved or dismissed issue keeps its state. And the studio's "Compare with version n" panel shows only the immediately preceding version, so after several resubmissions it compares against the last carried quotation; the original survives in the stored lineage.

Of the seven implementation claims made in the earlier draft, 4 were confirmed, 3 were partially true and 0 were refuted; the partially true claims concerned the assistant's powers, interface wording and the comparison panel, and are corrected in Section IV.

XIV. Analyses Added for the Second Review

Each analysis below followed a protocol whose text and code hashes were recorded before its new outcomes were computed; the re-aggregation, strata, identity-floor and policy analyses read records whose outcomes had already been seen (tier E1) (`review/V12-PROTOCOL-K*.md`, `evidence/v12-wpk*`); amendments were recorded before the outcomes they concern. The 16-page paper summarizes them; the tables are in Appendix F.

A. Six Outcomes, Strata, Identity Floor and Policy Selection

Tables XXXIII–XXXVIII re-aggregate every saved population and configuration into the six outcomes (placement, missed re-check, silent misplacement, misdirected request, lost anchor, unnecessary re-check), each with its numerator and denominator, and Table XXXIX does the same for the ten-commit chains. Tables XL–LIV stratify the outcomes by anchor length, anchors per issue and the locator's confidence crossed with the guards. Tables LV–LX replay every identity-rule floor from 0.3 to 1.0 on the stored chain traces: missed changes since review do not depend on the floor, which only trades wrong endpoints for lost ones. Tables LXI and LXII select a whole deployment policy on eight projects and evaluate it on the ninth (post hoc): selection beat the deployed cascade on every held-out project but did not reliably beat a fixed CodeMapper+IG+AGi, and the folds chose different policies.

B. Blinded Adjudication of the Syntax-Tree Reference

Two hosted models, not people, re-decided 237 stratified targets and 50 excluded candidates blind to the reference and to every method, from the old excerpt with the reviewed lines marked and the paired new file (Table LXIII). They agreed with the reference on 94.0% and 98.0% of the targets decided by same-type pairing and on 98.0% of the aligned controls. Agreement was lowest where the reference relocates code to another file, which the input does not show (on targets where the methods disagree, 52.0% and 56.0% overall but 78.8% and 84.8% of the 33 kept in the file), and where it reports a deletion: of the 30 deletions and relocations inside the paired file, both models contradicted the reference on 10, and on 9 of them found the code edited rather than deleted, a change either way. On the deployed cascade's 37 misses the models agreed with the reference on 73.0% and 75.7%, and both contradicted it on 8: on 5 both called the code unchanged, which would lower the miss count, and on the other 3, which the reference calls deleted, one model read the code as edited and the other as unchanged. Of the 50 sampled exclusions both models found 28 edited and 14 deleted, so exclusion removes hard changed cases. Agreement statistics are in Table LXIII; a blinded packet of 120 items with an answer sheet is prepared for two human annotators and has not been annotated.

C. Parametric Copy Suite

A held-out suite of 6,800 constructed trials from 116 source files crosses 0, 1, 2, 4 or 8 pre-existing identical copies with the distance of the nearest one, anchors of 1–8 lines and the fate of the original; 800 more trials let the revision itself add the copy (Table LXIV). The deployed cascade missed 82.1% of the changes. Distance decided the outcome and the number of copies did not: the guarded cascade missed 8.6% of changes with one copy and 8.5% with eight, and every miss of a guarded locator had a copy directly next to the original (36.0% for the cascade, 14.1% for CodeMapper). AG requested a check on every original shifted by lines inserted directly above it (100.0%), AGi on none (0.0%); when the revision adds the copy, AGi missed more (26.5% against 21.5% over CodeMapper; paired difference 5.0 points, 3.6 to 6.5). Grouped into issues of two to four anchors, 1.1% of issues went unreturned for CodeMapper+IG+AGi.

D. Applied Review Suggestions in Five Languages

A GitHub review suggestion quotes lines O and proposes a replacement S; where they occur at the pull request's last head gives a reference no annotator made. We took 998 suggestions from 486 merged pull requests in 10 repositories

(Python, TypeScript, Go, Rust and Java): 361 applied, 189 untouched, 336 rewritten, 38 deleted and 74 ambiguous; two independent classifiers agreed on every suggestion (Table LXV). CodeMapper placed 91.1% of applied or untouched targets, the deployed cascade 78.9% and GitHub 36.7%. Every missed re-check of the cascade and of CodeMapper, guarded or not, came from applications that add lines around verbatim reviewed lines (23 of 361), which no quotation comparison can see; GitHub's flag, which follows the diff hunk, also missed rewritten and applied suggestions. No locator placed a target on wrong code silently. Asked whether the reviewer's concern was addressed rather than whether the code changed, two hosted models reached precision 92.4% and 93.0% and recall 97.2% and 98.3%, against 81.8%, 82.5%, 71.6% and 77.8% for the change question, but still called 14 and 13 of 169 untouched suggestions addressed. Application is a proxy for an addressed concern, so this shows that the concern question predicts applied suggestions better on this sample, not that it helps supervisors; such an assessment may annotate a request but never resolve an issue.

E. An External Benchmark: ReviewAnchorBench

ReviewAnchorBench [38] applies ten mechanical transformations to the files of public review suggestions written by bots, and records for each case the target file's hash and the oracle span and state. Its released packages do not contain the transformation builder, so we rebuilt each target from the pinned source file and the recorded parameters and accepted it only when its SHA-256 matched the released hash. Seven families rebuild exactly. The three that insert generated text (internal line edit, surrounding context edit, whitespace) do not, and the source files of 60 and 40 cases have since disappeared from GitHub. That leaves 2,058 cases from 294 anchors in the population-sampled stratum and 2,625 from 375 anchors in the hard held-out split (Table LXVI); the protocol was frozen before any locator ran. Our CodeMapper run agreed with the benchmark's released CodeMapper output on 2,047 of 2,058 population cases (the same start line, or no location from either).

The layer cut missed re-checks over the deployed cascade from 3.7% to 0.3% (held-out 4.1% to 0.2%) and over CodeMapper from 2.7% to 0.3% (2.3% to 0.2%), and removed every silent wrong placement of the deployed cascade (13 and 37 cases). The benchmark's own cascade, which also searches for the reviewer's replacement, missed 0.2% and 0.4% on the seven families and 0.3% and 0.5% on all ten, re-scored from its released predictions. Its difference from the guarded cascade in missed re-checks lay within the repository bootstrap interval (0.1 points, $[-0.4, 0.7]$, population). Two mechanisms separate the approaches; both were tabulated after the first summary was read (amendment 1). The copy guard flagged 291 of 294 moved blocks (held-out 372 of 375), because a moved block's text existed in the old file at its old position; this produces most of the layer's 33.1% unnecessary requests, against 2.4% for the benchmark's cascade. On duplicated pre-images the locators placed the target on the original lines, almost always without a request (293 of 294 and 371 of 375 for the guarded cascade), whereas the benchmark counts only abstention as correct. The file-pair diff does identify the original, so the placement is defensible, but a copy the revision adds is invisible to the guards, as the parametric copy suite found.

F. Supervisor Interventions in Ten-Commit Chains

On the 4,000 archived chains (Section XI-H) a simulated supervisor acts at commits 1, 2, 3 and 5, only on a request: P1 corrects a wrong or lost anchor to the reference location, P2 accepts a correctly placed changed anchor as reviewed, P3 does whichever fits, and P4 dismisses the issue at the first request (Table LXVII). With no action the stored cascade traces were reproduced exactly (0 mismatches), and CodeMapper's on 39,986 and, with the identity rule, 39,990 of 40,000 hops, since CodeMapper is not fully deterministic. Correction raised the deployed cascade's correct endpoints by 217 (95% chain-bootstrap interval 189–246) with 223 actions but removed only 36 of 90 wrong endpoints, because most of its wrong placements raise no request. Without the identity rule, acceptance lowered missed changes by exactly the changes it reviewed (210 and 367); with the rule it removed 599 and 996 later requests with missed changes unchanged. Dismissal ended between 425 and 536 chains per method, of which 12 to 14 had a later change that the dismissed issue never surfaces.

G. A Stronger Persistence Contract

Table LXVIII lists each lifecycle gap found in Section XIII, the implemented behavior and a proposed rule. A bounded model over two findings with colliding dismissal keys, three versions and all label assignments of two anchors, to depth 8 (3455 and 3748 distinct states), reproduced every gap in the implemented contract with a minimal trace and found none in the proposed one; evidence replacement wrote no closed state in either. The proposed contract is specified and model-checked, not implemented.

XV. Threats to Validity

A. Construct Validity

Every outcome is a property of locations and labels, not of review concerns. A correct start line does not show that a comment still applies, and a changed label does not show that a concern is affected. The controlled edit mix is uniform over twelve operators chosen by us, and one of them, re-indent, often yields invalid or behavior-changing Python (Section XI). Automatically sampled anchors need not resemble the spans supervisors select: supervisors may favor short, repeated or boundary-spanning lines, which the stress suites show to be harder. Changes outside the quoted range can matter to a concern and are not evaluated. The effort model's weights are illustrative, not measured costs.

B. Oracle Validity

The real-history reference is a syntax-tree correspondence procedure, not semantic ground truth: it aligns statements by normalized tree structure and uses heuristics for replacements, moves and deletions, and does not decide whether two

statements behave alike. It excludes 12.8% of candidate observations; excluded files have larger diffs (median 41 changed lines versus 12 for retained ones), so accuracy is conditional on a decidable and easier subset and may overestimate performance on unrestricted code evolution. The original manual audit covered only 40 decisions and was carried out by members of the development team, which risks confirmation bias; a 70-case blinded packet with separately held answers is prepared, but no independent responses have been collected. The human-annotated external benchmark of Section VIII is the only reference in this paper made independently of the authors. Every target the reference calls unchanged keeps its quoted lines at the reference location, but for relocated targets that location can be a copy that already existed, so its identification errors can overstate missed re-checks and also hide a few: they bound the deployed miss rate between 0.2% and 2.9%. Placement accuracy and unnecessary re-checks have no such bound.

C. Internal Validity

The default threshold was selected on a separate synthetic seed. Leave-one-project-out calibration, the factorial ablation and the guards' real-history results use histories that earlier revisions had inspected, so they are retrospective; the guards' held-out evidence is limited to stress suites generated after their protocol was fixed, the external benchmark and the student-authored repositories. The corrected copy guard AGi was specified after the first guard's student and compound results had been read, so its held-out evidence is the fresh stress seed of Section XII; CodeMapper with the cross-file fallback and AGi was composed after outcomes were read and is exploratory. The guarded ten-commit replay was first run with the rejected provenance rule and repeated with AGi under a recorded amendment before its outcomes existed. The pinned CodeMapper is not fully deterministic: recomputed on the same chains, it changed its answer on 3 of 4,000, so CodeMapper rates carry a small run-to-run variation beyond their sampling intervals. The protocols are timestamped locally, not externally preregistered. The conventional baselines are reimplementations of documented behaviors, not tests of the deployed products, and implementation details of GitHub, Gerrit or GitLab may differ.

D. Statistical Dependence

Observations from the same project, commit pair or source file are correlated. Wilson intervals are nominal per-observation intervals. We therefore also report cluster-bootstrap intervals by source file, commit pair and project; one internal comparison loses its resolved direction under project clustering (Section X). With nine projects, project-level intervals are coarse. The paper reports many paired comparisons and does not adjust them for multiplicity. The registered hypotheses are therefore stated per population and contrast together with the number that held (H1 was supported on 91 of 114 contrasts and H2 on 80 of 264), and an individual interval that excludes zero should be read as one comparison among many, not as a confirmed effect.

E. External Validity

All real histories are Python, from four internal projects and five widely used libraries; none consists of supervised student resubmissions. Student revisions may restructure more, repeat more code, be submitted incompletely or omit files. The public student-authored repositories of Section VIII-B are a stand-in that shares authorship but not the supervision setting. The algorithm operates on text, but cross-language effectiveness rests on the external benchmark of Section VIII, not on our own corpora. Errors may accumulate over many resubmissions; our multi-hop analysis replays real commit chains without independent intermediate identity labels or supervisor intervention.

F. Implemented but Not Evaluated

Media-note reliability for users, privacy and access control and concurrent composition are implemented design controls. Section XIII reports the scenario tests that exercise them, which are pass/fail checks of specified behavior, not a security assessment, penetration test or user study.

XVI. Discussion

A. What Was and Was Not Shown

The experiments show where anchored code is found, how its change is labeled and how often the label withholds a re-check, on defined distributions. They do not show that supervisors begin a re-review faster, revisit more of their earlier points, close fewer issues in error or that students respond better. Those benefits motivate the design and are plausible, but they are hypotheses. The direct test is a within-subjects study in which supervisors re-review the same revised submissions with a conventional interface and with Living Review, measuring time to locate earlier comments, the share of earlier issues correctly revisited, erroneous closures and workload.

B. Matcher Strength

The method is strongest where its design assumptions hold: code that is shifted, moved or edited lightly and that has no identical copies nearby. On naturally edited statements its default threshold is too strict, the earlier v1 design relocates more, an independent contextual line tracker relocates more again within files (Sections VII-G and VIII-B), and a dedicated region mapper, CodeMapper, locates human-annotated regions far more accurately and recovers their boundaries where the cascade mostly does not (Section VIII). The cascade is not a competitive locator. Its distinctive capabilities are cross-file recovery and the labels that drive the issue state. Because the labels are computed from the evidence a matcher returns, the relocation engine is replaceable. Section XII did this with CodeMapper: the same label rules and window guards (IG and AGi; the variable-height window is part of the cascade's own search) made it safer on every population without changing its placement. The workflow's safety properties are those of the combined system, and a deployment should pair the strongest available locator with the safety layer rather than choose between them.

C. Costs of Different Failures

The failures do not cost the same. A lost anchor is visible: the issue requests re-check and the supervisor searches for the code, a cost of time. An unnecessary re-check costs a short confirmation. A missed re-check or a silent misplacement costs attention that is never prompted, and can lead a supervisor to judge a concern against the wrong code. The evaluation therefore treats the last two as safety failures and the first two as effort, and recommends guards that convert silent failures into visible ones even at the price of more confirmations.

Whether that price is worth paying depends on how many carried issues point at code that changed, which no study here measured. Let p be that share and w the cost of a missed re-check in units of one unnecessary confirmation. The recommended guards (Table XXIV) cost no more than the deployed cascade when $w\,p$ times the fall in the missed re-check rate is at least $(1 - p)$ times the rise in the false re-check rate. With $w = 1$ the break-even share is 37.6% on real histories, 56.3% on the student-authored repositories and 68.4% on the human-annotated regions; with $w = 10$ it is 5.7%, 11.4% and 17.8%. These points ignore silent misplacements, which the guards also reduce on every population, so they favor the deployed cascade. For comparison, 64.6% of review threads in our open-source sample were outdated by GitHub, an upper bound on changed code rather than a measured share of unresolved concerns. A course that weighs a hidden change at ten confirmations or more gains from the guards on all three populations once more than 17.8% of its carried issues point at changed code.

D. Misplacement as a Traceability Problem

A misplaced anchor attaches feedback to code with a different meaning. The interface shows the label in plain language beside every relocated anchor, and the supervisor can edit an anchor; every edit is an attributed revision. When tier 3 accepts a changed window, however, the carried anchor's quotation is replaced by the matched text, and later versions are matched against that updated quotation. In the deployed code the original quotation survives only in the previous version's issue, reachable through the lineage reference. The multi-hop replay shows the consequence: after a wrong fuzzy update, later revisions can match the updated quotation exactly, so the latest label alone cannot certify continuity with the original target. The identity rule of Section III-E keeps the reviewed quotation in every carried anchor and, in replay, made the latest request carry the whole history (Section XI-H). It is evaluated in replay only; the deployed portal does not yet store it.

E. Media Notes

Voice, video and image notes follow their issue or are refound with the cascade when pinned to code. Their placement reliability is that of code anchors, and their handling across versions is covered by the scenario tests of Section XIII. No quantitative relocation study or user study of media notes was performed.

F. Deploying the Method in a Course

An institution adopting the method should keep the local search scope and tier order, choose the threshold as a policy between relocation and honest removal (Section X), enable the indentation guard and the corrected copy guard AGi over whichever locator it uses (Section XII), keep the reviewed quotation beside the current one, prefer few anchors per issue, warn supervisors when they anchor a recurring one-line span, and keep resolution with the supervisor. Supervisors' corrections of misplaced anchors should be logged, with consent, so that a course can calibrate locally.

G. Ethics and Data Handling

No student submission was used in this evaluation. The synthetic benchmark used our own project code; the real histories came from our projects, public open-source repositories and public student-authored repositories, all parsed and never executed. The student-authored repositories are pseudonymized here and no individual is identified. In the portal, feedback stays private to the supervisors until it is shared, and a student sees only the frozen packet addressed to them. Research telemetry in the portal is separate from course access: it requires opt-in consent, synthetic demonstration data are excluded from research aggregates, and declining consent does not change a student's access to the course. A classroom study would require institutional approval and an appropriate consent process.

XVII. Conclusion

Living Review separates where a review concern now points from whether it has been settled. Its contribution is the safety layer and state contract around the matcher, not the matcher: an independent contextual tracker and the region mapper CodeMapper locate edited code better, and the cascade contributes cross-file recovery. Missed re-checks are rare on real histories and arise from one identifiable mechanism, identical copies of edited code, which guards address on held-out student-authored, human-annotated and stress data at a measurable cost in confirmations. The same labels and window guards made CodeMapper safer without changing where it placed anything, so the locator can be upgraded while the safety argument stays the same. A language model, the one the portal already uses, located edited code well, but its own judgment of change missed re-checks that the same layer then caught. Content alone cannot tell two identical copies apart; the guards decide when to ask, not which copy is right. Keeping the reviewed quotation makes uncertain identity transitions visible across many resubmissions. The similarity threshold is a policy choice between relocating edited code and reporting deleted code honestly, and safety limits chosen on one group of projects did not hold on another. The implemented lifecycle never resolves an issue on its own, but deletion stops an issue from being carried and dismissal memory is a heuristic. Whether carrying feedback forward helps supervisors and students remains a question for a consented classroom study. In order of priority, the next steps are that study on real resubmissions, an independent audit of the syntax-tree reference with the prepared blinded packet, and deployment of the identity rule with the stronger locator in the portal.

Acknowledgment

OpenAI Codex [47] and Anthropic Claude Code were used extensively to assist with experimental planning and implementation, benchmark integration, analysis and reproducibility checks, and the drafting and revision of substantial portions of this manuscript. The authors directed the research, determined the scope and interpretation of the reported findings, and take responsibility for the accuracy and integrity of the final manuscript and its supporting evidence.

Reproducibility

Pinned inputs. Public real histories: Requests (https://github.com/psf/requests) at `3c7baaa05ab4aa49cc819ce3a819d260de19c157`; Flask (https://github.com/pallets/flask) at `d086db856be187255b8ec61ef409357393020f32`; Click (https://github.com/pallets/click) at `2247b35ea1c47c727d7a06e51fa280e12a863ff6`; HTTPX (https://github.com/encode/httpx) at `b5addb64f0161ff6bfe94c124ef76f6a1fba5254`; Rich (https://github.com/Textualize/rich) at `9d8f9a372cc5916fd4781fec207ced7ddac2f08f`, each fetched with Git depth 400, which follows every parent, and traversed by first parent (3,156 first-parent commits in all). Contextual tracker: https://github.com/oselvar/lhdiff at `5a9f7fd87193098cf90f7ee4081f6d95f04d4cc7`. CodeMapper: https://github.com/sola-st/CodeMapper at `944fa91e6b359322c1a91f255ed7f4dfc556b68f`. GumTree: release v4.1.0 at `bd712afda055f3b97dca0f33f2b73d36c712edfe`. Scale corpora: Django django-5.1/django-5.2 at `373cb3037fe4e67adbac9ac43340391e859aa957`/`9e7cc2b628fe8fd3895986af9b7fc9525034c1b0`; CPython cpython-v3.12.0/cpython-v3.13.0 at `0fb18b02c8ad56299d6a2910be0bab8ad601ef24`/`60403a5409ff2c3f3b07dd2ca91a7a3e096839c7`. The 20 student-authored repositories are pseudonymized in this paper because they are student work; their URLs and full head hashes are in the selection manifest (SHA-256 `daa5e12406698034 56ca3c69de8a1b43e8419ce354e9f7750a3a4de8f7155767`). The four internal histories and the controlled corpus are private.

Environment. Python 3.12.14 (the compound-edit and multi-anchor stages ran under Python 3.11.4, recorded in their start files); NumPy 2.5.4 and Matplotlib 3.10.7 vendored for analysis and figures; CodeMapper and GumTree run in isolated environments whose dependency manifests and hashes are retained with the locator-comparison evidence. Timings come from one workstation (13th Gen Intel(R) Core(TM) i9-13900HX, Windows 11).

Commands. Each work package has a frozen protocol and a runner under `tools/v11/` (`run_wpb2.py`, `run_wph2.py`, `run_wpc.py`, `run_wpd_tests.py` and the real-history gate), which writes JSON with the SHA-256 of its code and inputs. `build_v11_numbers.py` turns those JSON files into every number in the text, `build_floats.py` and `build_effort_figure.py` build the remaining floats, and this section is generated by `build_repro_tex.py`. The macro file has SHA-256 `7b33f31801bf8e0ab5a69119cb49ca3d588372502325dd9714f86ee7919f1305` and records the hashes of its 27 input files. Artifact: https://github.com/Noel-Alex/living-review-artifact; its verifier rebuilds every macro from the shipped evidence. Internal histories and student repositories are not released.

Appendix A
Per-Project Results

Table XXVIII lists each project of the real-history study with the number of first-parent commits, contributors (distinct author names), commit pairs and decidable anchors, and the correct outcome of each method on the edit-targeted sample. The claim tracer contributes few anchors because its history is short. Table XXIX splits commit pairs and anchors by revision horizon and gives the anchors the syntax-tree reference excluded in each project.

Appendix B
Threshold Sensitivity on Real Edits

Table XXX extends Table XVII to the full threshold grid, together with the share of all real anchors handled correctly at each threshold. Only the approximate tier depends on θ. Anchors settled by tiers 1 and 2 (untouched and shifted code) are therefore handled identically in every row; only anchors that reach tier 3 (edited and gone code, and code moved to another file) can change.

Appendix C
Oracle Audit

The recorded audit draws 40 in-house observations with seed 7: 20 from edited, gone and moved-file classes, and 20 from preserved classes. The file `oracle-audit-v1.txt` records the compared quotations and judgments. Thirty-nine decisions were accepted; one pairing of unrelated same-type statements was marked doubtful. These are the recorded audit judgments, not an independent multi-rater validation. The sample does not establish accuracy on public-project, excluded or rare-class observations, and the direction of reference error need not favor or penalize every matcher equally.

Appendix D
Reproducing the Results

Reproduction has three levels. First, export the pinned corpus and run `analysis_v3.py` to regenerate controlled trials and summaries; the additional harness retains individual rows. Second, `real_history.py` rebuilds reference observations from exact histories and `merge_oss.py` combines the public runs. The selection-only adapter verifies the same sample and exclusions without evaluating matchers. Third, `analyze_real.py` regenerates the real-history tables, while the controlled table/plot scripts regenerate their presentation. Re-aggregation alone does not validate reference judgments or reconstruct source histories.

TABLE XXVIII
REAL-HISTORY STUDY PER PROJECT

					uniform	edit-targeted (correct %)				
Project	commits	authors	pairs	anchors	cascade	cascade	v1	quote	drift	line
Review portal	444	3	193	1,584	99.0	69.4	73.2	40.9	39.8	16.3
Claim tracer	40	2	18	87	100.0	50.0	50.0	8.3	8.3	8.3
LM training	203	2	168	1,476	99.3	73.6	79.2	53.0	49.8	19.2
Evidence ingestion	108	2	78	658	98.9	64.1	72.3	40.3	40.3	12.6
Requests	626	240	229	1,260	99.5	64.8	74.1	16.0	16.0	11.1
Flask	448	179	245	1,792	99.0	54.3	69.4	14.0	10.8	7.6
Click	589	240	331	2,359	99.6	70.0	75.7	28.7	28.1	15.2
HTTPX	400	104	187	1,409	99.5	72.3	78.7	14.6	12.2	7.3
Rich	1,093	273	624	4,439	99.4	69.0	74.2	27.5	26.1	12.8

Uniform: cascade, correct %. Edit-targeted: correct % of each method. Above the rule our projects, below it the open-source ones (shallow clones of the default branch, depth 400).

TABLE XXIX
COMMIT PAIRS AND ANCHORS BY PROJECT AND REVISION HORIZON, RETAINED AND ORACLE-EXCLUDED

Project	Horizon	Attempted pairs	Retained pairs	Retained anchors	Excluded anchors
Click	1	275	274	1756	233
Click	10	57	57	603	129
Flask	1	202	202	1297	230
Flask	10	43	43	495	116
HTTPX	1	150	150	1003	158
HTTPX	10	37	37	406	80
LM training	1	148	148	1209	190
LM training	10	20	20	267	49
Evidence ingestion	1	69	69	548	59
Evidence ingestion	10	9	9	110	24
Requests	1	178	178	820	42
Requests	10	51	51	440	45
Rich	1	516	516	3178	282
Rich	10	108	108	1261	185
Claim tracer	1	16	16	75	17
Claim tracer	10	2	2	12	7
Review portal	1	153	153	1178	237
Review portal	10	40	40	406	122

Pair counts concern sampled targets and need not add across retained/excluded groups. Exclusions have no assigned matcher truth or accuracy.

TABLE XXX
FUZZY-TIER THRESHOLD ON REAL EDITS

θ	edited: re-anchored	gone: reported removed	all anchors
0.5	76.6	61.8	95.5
0.6	75.4	74.3	95.4
0.7	70.9	81.9	94.7
0.75	67.4	84.0	94.1
0.8	62.3	88.2	93.3
0.85	53.6	92.4	91.8
0.9	42.6	93.8	89.9
0.95	26.5	94.4	87.1

Every column is the final cascade; the columns are kinds of code, not methods. Correct outcome (%) on edited statements, on deleted statements and on all real anchors. The method uses the fixed-default 0.85; gone code has only 144 cases. Table XXXI compares Cascade v1 across the same thresholds.

TABLE XXXI
FINAL CASCADE AND INTERNAL COMPARATOR ACROSS THRESHOLDS

	ours, all anchors		open source, edit-targeted	
θ	cascade	v1	cascade	v1
0.5	94.1	93.3	82.6	78.2
0.6	93.8	93.1	82.5	78.2
0.7	93.0	92.5	79.7	76.4
0.75	92.5	92.0	76.9	74.3
0.8	91.9	91.5	72.9	71.1
0.85	90.2	90.0	66.8	65.6
0.9	88.4	88.1	58.8	58.0
0.95	85.7	85.5	46.9	46.3

Correct outcome (%). Left: all anchors of our four projects, the set on which each design's threshold is chosen. Right: the held-out open-source edit-targeted sample. Both designs select 0.5 on the choosing set.

The verification artifact documents commands for the reference-definition sensitivity, complete-source checks and independent tracker comparisons. The range experiment runs `evaluate_real_ranges.py` followed by `summarize_real_ranges.py`, retaining both thresholds' coordinates and the declared metric. `evaluate_revision_chains.py` replays the ten-hop comparisons and `summarize_revision_chains.py` aggregates endpoints and request histories. Their local protocols precede execution; neither is an external registration. A complete artifact must include input revisions, traversals, environment, seeds, outputs and checksums. Runtime depends on the environment; deterministic counts must match the recorded expectations. Private-source access remains a separate requirement.

APPENDIX E
CONTROLLED EDIT BREAKDOWN

Table XXXII gives exact per-operator values for the main controlled seed. It complements the outcome plot in Fig. 5; both use the same 300 trials per operator.

TABLE XXXII
CORRECT OUTCOME (%) PER EDIT TYPE, 300 TRIALS EACH, SEED 20261018

Edit type	Cascade (ours)	Cascade v1	Exact quote	Diff drift	Fixed line
no change	100.0	100.0	100.0	100.0	100.0
edit elsewhere	100.0	100.0	100.0	100.0	100.0
insert above	100.0	100.0	100.0	100.0	0.0
delete above	100.0	100.0	100.0	100.0	0.0
re-indent	100.0	100.0	100.0	0.0	100.0
move in file	99.0 (1.0)	99.0 (1.0)	99.0 (1.0)	0.0	0.0
move to other file	97.0 (3.0)	99.0 (1.0)	0.0 (0.7)	0.0	0.0
duplicate above	100.0	100.0	100.0	100.0	0.0
token edit	95.7 (4.3)	95.0 (5.0)	0.0 (0.7)	0.0	0.0
insert + token edit	96.7 (2.7)	95.0 (5.0)	0.0 (0.3)	0.0	0.0
rewrite (gone)	97.0 (3.0)	82.0 (18.0)	99.3 (0.7)	100.0	100.0
delete (gone)	96.3 (3.7)	86.3 (13.7)	100.0	100.0	100.0
all edits	98.5 (1.5)	96.4 (3.6)	74.9 (0.3)	58.3 (0.0)	41.7 (0.0)

In brown: share placed on wrong lines; the rest of the shortfall was lost (not re-anchored). Above the gap the anchored content is preserved; below it the content changed or is gone.

APPENDIX F
TABLES FOR THE SECOND REVIEW

Tables for Section XIV, in the order that section cites them.

TABLE XXXIII
SIX OUTCOMES ACROSS SAVED POPULATIONS (%), PART 1

Population	Configuration	Place	Miss	Silent	Misdirect	Lost	Extra
Fresh	Cascade, deployed	66.0 (13,258/20,100)	71.8 (6,386/8,898)	37.1 (8,727/23,550)	0.9 (214/23,550)	3.3 (661/20,100)	0.9 (137/14,652)
Fresh	Cascade, IG+AGi+VW	69.7 (14,011/20,100)	4.9 (438/8,898)	5.2 (1,214/23,550)	29.9 (7,041/23,550)	3.0 (594/20,100)	32.8 (4,800/14,652)
Fresh	Cascade, IG+AGi+VW, theta 0.55	69.6 (13,989/20,100)	4.7 (416/8,898)	5.0 (1,175/23,550)	32.6 (7,680/23,550)	1.4 (289/20,100)	34.0 (4,981/14,652)
Fresh	CodeMapper, quote	83.0 (16,675/20,100)	19.3 (1,715/8,898)	7.0 (1,652/23,550)	4.6 (1,072/23,550)	10.6 (2,127/20,100)	13.4 (1,970/14,652)
Fresh	CodeMapper, IG+AG	83.0 (16,675/20,100)	2.0 (179/8,898)	3.0 (698/23,550)	8.6 (2,026/23,550)	10.6 (2,127/20,100)	38.0 (5,567/14,652)
Fresh	CodeMapper, IG+AGi	83.0 (16,675/20,100)	3.1 (277/8,898)	3.7 (870/23,550)	7.9 (1,854/23,550)	10.6 (2,127/20,100)	37.0 (5,423/14,652)
Fresh	Tracker, quote	82.0 (16,488/20,100)	22.0 (1,959/8,898)	7.7 (1,822/23,550)	0.1 (13/23,550)	11.8 (2,378/20,100)	13.2 (1,936/14,652)
Fresh	Tracker, IG+AGi	82.0 (16,488/20,100)	3.3 (295/8,898)	4.1 (966/23,550)	3.7 (869/23,550)	11.8 (2,378/20,100)	37.9 (5,553/14,652)
FreshDuplicates	Cascade, deployed	38.1 (1,144/3,000)	89.7 (1,345/1,500)	69.1 (2,593/3,750)	0.3 (13/3,750)	0.0 (0/3,000)	0.1 (3/2,250)
FreshDuplicates	Cascade, IG+AGi+VW	40.9 (1,228/3,000)	29.2 (438/1,500)	32.3 (1,212/3,750)	34.9 (1,310/3,750)	0.0 (0/3,000)	24.6 (553/2,250)
FreshDuplicates	Cascade, IG+AGi+VW, theta 0.55	40.4 (1,212/3,000)	27.7 (416/1,500)	31.3 (1,174/3,750)	36.4 (1,364/3,750)	0.0 (0/3,000)	26.0 (586/2,250)
FreshDuplicates	CodeMapper, quote	56.7 (1,701/3,000)	46.1 (692/1,500)	43.8 (1,644/3,750)	1.8 (66/3,750)	6.0 (181/3,000)	10.8 (243/2,250)
FreshDuplicates	CodeMapper, IG+AG	56.7 (1,701/3,000)	11.9 (179/1,500)	18.6 (698/3,750)	27.0 (1,012/3,750)	6.0 (181/3,000)	36.4 (819/2,250)
FreshDuplicates	CodeMapper, IG+AGi	56.7 (1,701/3,000)	18.5 (277/1,500)	23.2 (870/3,750)	22.4 (840/3,750)	6.0 (181/3,000)	33.1 (745/2,250)
FreshDuplicates	Tracker, quote	54.4 (1,633/3,000)	50.3 (754/1,500)	48.2 (1,809/3,750)	0.2 (7/3,750)	5.0 (151/3,000)	6.9 (155/2,250)
FreshDuplicates	Tracker, IG+AGi	54.4 (1,633/3,000)	19.7 (295/1,500)	25.8 (966/3,750)	22.7 (850/3,750)	5.0 (151/3,000)	31.0 (698/2,250)
FreshIndentation	Cascade, deployed	100.0 (2,400/2,400)	100.0 (1,200/1,200)	0.0 (0/2,400)	0.0 (0/2,400)	0.0 (0/2,400)	0.0 (0/1,200)
FreshIndentation	Cascade, IG+AGi+VW	100.0 (2,400/2,400)	0.0 (0/1,200)	0.0 (0/2,400)	0.0 (0/2,400)	0.0 (0/2,400)	25.0 (300/1,200)
FreshIndentation	Cascade, IG+AGi+VW, theta 0.55	100.0 (2,400/2,400)	0.0 (0/1,200)	0.0 (0/2,400)	0.0 (0/2,400)	0.0 (0/2,400)	25.0 (300/1,200)
FreshIndentation	CodeMapper, quote	99.5 (2,389/2,400)	85.0 (1,020/1,200)	0.0 (0/2,400)	0.0 (0/2,400)	0.5 (11/2,400)	3.3 (40/1,200)
FreshIndentation	CodeMapper, IG+AG	99.5 (2,389/2,400)	0.0 (0/1,200)	0.0 (0/2,400)	0.0 (0/2,400)	0.5 (11/2,400)	26.9 (323/1,200)
FreshIndentation	CodeMapper, IG+AGi	99.5 (2,389/2,400)	0.0 (0/1,200)	0.0 (0/2,400)	0.0 (0/2,400)	0.5 (11/2,400)	26.9 (323/1,200)
FreshIndentation	Tracker, quote	100.0 (2,400/2,400)	100.0 (1,200/1,200)	0.0 (0/2,400)	0.0 (0/2,400)	0.0 (0/2,400)	16.7 (200/1,200)
FreshIndentation	Tracker, IG+AGi	100.0 (2,400/2,400)	0.0 (0/1,200)	0.0 (0/2,400)	0.0 (0/2,400)	0.0 (0/2,400)	41.7 (500/1,200)

Each cell shows rate (events/its outcome-specific denominator). Human-annotated and language-model cells use only the WP J populations recorded for that configuration. All results are retrospective and descriptive.

TABLE XXXIV
SIX OUTCOMES ACROSS SAVED POPULATIONS (%), PART 2

Population	Configuration	Place	Miss	Silent	Misdirect	Lost	Extra
FreshShort	Cascade, deployed	68.1 (9,194/13,500)	71.0 (3,832/5,400)	37.9 (6,132/16,200)	0.5 (82/16,200)	0.8 (102/13,500)	0.2 (18/10,800)
FreshShort	Cascade, IG+AGi+VW	72.0 (9,714/13,500)	0.0 (0/5,400)	0.0 (2/16,200)	35.1 (5,692/16,200)	0.8 (102/13,500)	34.1 (3,679/10,800)
FreshShort	Cascade, IG+AGi+VW, theta 0.55	71.9 (9,710/13,500)	0.0 (0/5,400)	0.0 (1/16,200)	37.5 (6,067/16,200)	0.0 (5/13,500)	35.3 (3,815/10,800)
FreshShort	CodeMapper, quote	89.1 (12,025/13,500)	0.0 (0/5,400)	0.0 (8/16,200)	5.2 (844/16,200)	10.8 (1,457/13,500)	13.5 (1,456/10,800)
FreshShort	CodeMapper, IG+AG	89.1 (12,025/13,500)	0.0 (0/5,400)	0.0 (0/16,200)	5.3 (852/16,200)	10.8 (1,457/13,500)	37.6 (4,063/10,800)
FreshShort	CodeMapper, IG+AGi	89.1 (12,025/13,500)	0.0 (0/5,400)	0.0 (0/16,200)	5.3 (852/16,200)	10.8 (1,457/13,500)	37.5 (4,050/10,800)
FreshShort	Tracker, quote	86.6 (11,688/13,500)	0.0 (2/5,400)	0.1 (13/16,200)	0.0 (0/16,200)	13.3 (1,800/13,500)	12.5 (1,350/10,800)
FreshShort	Tracker, IG+AGi	86.6 (11,688/13,500)	0.0 (0/5,400)	0.0 (0/16,200)	0.1 (13/16,200)	13.3 (1,800/13,500)	37.5 (4,050/10,800)
Controlled	Cascade, deployed	98.9 (2,968/3,000)	0.7 (8/1,200)	0.4 (15/3,600)	0.9 (32/3,600)	0.1 (4/3,000)	0.3 (8/2,400)
Controlled	Cascade, IG+AGi+VW	99.1 (2,973/3,000)	0.0 (0/1,200)	0.0 (0/3,600)	1.2 (45/3,600)	0.1 (3/3,000)	25.5 (612/2,400)
Controlled	Cascade, IG+AGi+VW, theta 0.55	97.3 (2,918/3,000)	0.0 (0/1,200)	0.0 (0/3,600)	6.4 (232/3,600)	0.1 (2/3,000)	27.9 (669/2,400)
Controlled	CodeMapper, quote	89.8 (2,694/3,000)	0.0 (0/1,200)	0.0 (0/3,600)	8.2 (294/3,600)	10.1 (304/3,000)	13.2 (317/2,400)
Controlled	CodeMapper, IG+AG	89.8 (2,694/3,000)	0.0 (0/1,200)	0.0 (0/3,600)	8.2 (294/3,600)	10.1 (304/3,000)	37.7 (905/2,400)
Controlled	CodeMapper, IG+AGi	89.8 (2,694/3,000)	0.0 (0/1,200)	0.0 (0/3,600)	8.2 (294/3,600)	10.1 (304/3,000)	37.6 (902/2,400)
Controlled	Tracker, quote	89.8 (2,695/3,000)	0.0 (0/1,200)	0.0 (0/3,600)	0.1 (2/3,600)	10.2 (305/3,000)	12.5 (300/2,400)
Controlled	Tracker, IG+AGi	89.8 (2,695/3,000)	0.0 (0/1,200)	0.0 (0/3,600)	0.1 (2/3,600)	10.2 (305/3,000)	37.5 (900/2,400)
Real	Cascade, deployed	91.8 (13,693/14,920)	1.3 (37/2,780)	0.3 (40/15,064)	1.6 (248/15,064)	6.4 (950/14,920)	0.0 (0/12,284)
Real	Cascade, IG+AGi+VW	93.7 (13,977/14,920)	0.1 (4/2,780)	0.0 (4/15,064)	1.0 (146/15,064)	5.4 (805/14,920)	0.7 (88/12,284)
Real	Cascade, IG+AGi+VW, theta 0.55	97.8 (14,589/14,920)	0.0 (1/2,780)	0.0 (1/15,064)	2.1 (318/15,064)	0.4 (57/14,920)	0.7 (91/12,284)
Real	CodeMapper, quote	97.9 (14,610/14,920)	0.4 (12/2,780)	0.1 (12/15,064)	1.5 (227/15,064)	0.5 (68/14,920)	0.8 (99/12,284)
Real	CodeMapper, IG+AG	97.9 (14,610/14,920)	0.0 (1/2,780)	0.0 (1/15,064)	1.6 (238/15,064)	0.5 (68/14,920)	1.9 (230/12,284)
Real	CodeMapper, IG+AGi	97.9 (14,610/14,920)	0.0 (1/2,780)	0.0 (1/15,064)	1.6 (238/15,064)	0.5 (68/14,920)	1.3 (160/12,284)
Real	Tracker, quote	95.6 (14,264/14,920)	0.3 (8/2,780)	0.1 (8/15,064)	0.6 (97/15,064)	2.9 (430/14,920)	1.7 (210/12,284)
Real	Tracker, IG+AGi	95.6 (14,264/14,920)	0.0 (0/2,780)	0.0 (0/15,064)	0.7 (105/15,064)	2.9 (430/14,920)	2.1 (264/12,284)

Each cell shows rate (events/its outcome-specific denominator). Human-annotated and language-model cells use only the WP J populations recorded for that configuration. All results are retrospective and descriptive.

TABLE XXXV
SIX OUTCOMES ACROSS SAVED POPULATIONS (%), PART 3

Population	Configuration	Place	Miss	Silent	Misdirect	Lost	Extra
Students	Cascade, deployed	91.8 (7,036/7,661)	3.1 (55/1,770)	1.5 (118/7,882)	1.7 (135/7,882)	5.5 (420/7,661)	0.0 (3/6,112)
Students	Cascade, IG+AGi+VW	93.2 (7,140/7,661)	0.2 (4/1,770)	0.4 (34/7,882)	2.0 (161/7,882)	4.9 (374/7,661)	3.8 (230/6,112)
Students	Cascade, IG+AGi+VW, theta 0.55	96.6 (7,401/7,661)	0.1 (2/1,770)	0.4 (32/7,882)	3.0 (239/7,882)	0.7 (53/7,661)	3.8 (233/6,112)
Students	CodeMapper, quote	92.4 (7,077/7,661)	0.5 (9/1,770)	0.2 (13/7,882)	3.6 (280/7,882)	3.6 (278/7,661)	6.1 (374/6,112)
Students	CodeMapper, IG+AG	92.4 (7,077/7,661)	0.1 (2/1,770)	0.1 (6/7,882)	3.6 (287/7,882)	3.6 (278/7,661)	8.3 (509/6,112)
Students	CodeMapper, IG+AGi	92.4 (7,077/7,661)	0.1 (2/1,770)	0.1 (6/7,882)	3.6 (287/7,882)	3.6 (278/7,661)	7.4 (453/6,112)
Students	Tracker, quote	91.9 (7,041/7,661)	0.5 (9/1,770)	0.2 (13/7,882)	1.2 (91/7,882)	7.2 (553/7,661)	7.2 (441/6,112)
Students	Tracker, IG+AGi	91.9 (7,041/7,661)	0.1 (2/1,770)	0.1 (6/7,882)	1.2 (98/7,882)	7.2 (553/7,661)	8.9 (542/6,112)
Human	Cascade, deployed	62.2 (97/156)	8.5 (11/130)	3.5 (7/200)	9.0 (18/200)	27.6 (43/156)	34.3 (24/70)
Human	Cascade, IG+AGi+VW	67.3 (105/156)	3.8 (5/130)	0.5 (1/200)	9.0 (18/200)	26.3 (41/156)	44.3 (31/70)
Human	Cascade, IG+AGi+VW, theta 0.55	87.2 (136/156)	3.8 (5/130)	0.5 (1/200)	21.0 (42/200)	1.9 (3/156)	44.3 (31/70)
Human	CodeMapper, quote	93.6 (146/156)	3.1 (4/130)	0.0 (0/200)	7.5 (15/200)	1.3 (2/156)	35.7 (25/70)
Human	CodeMapper, IG+AG	93.6 (146/156)	0.8 (1/130)	0.0 (0/200)	7.5 (15/200)	1.3 (2/156)	50.0 (35/70)
Human	CodeMapper, IG+AGi	93.6 (146/156)	2.3 (3/130)	0.0 (0/200)	7.5 (15/200)	1.3 (2/156)	44.3 (31/70)
Human	Tracker, quote	82.7 (129/156)	3.8 (5/130)	0.0 (0/200)	3.0 (6/200)	14.7 (23/156)	35.7 (25/70)
Human	Tracker, IG+AGi	82.7 (129/156)	3.1 (4/130)	0.0 (0/200)	3.0 (6/200)	14.7 (23/156)	44.3 (31/70)
FreshCompoundKFour	Cascade, deployed	10.7 (32/300)	0.7 (2/272)	0.7 (2/300)	6.7 (20/300)	82.0 (246/300)	60.7 (17/28)
FreshCompoundKFour	Cascade, IG+AGi+VW	13.7 (41/300)	0.0 (0/272)	0.0 (0/300)	5.7 (17/300)	80.7 (242/300)	85.7 (24/28)
FreshCompoundKFour	Cascade, IG+AGi+VW, theta 0.55	15.0 (45/300)	0.0 (0/272)	0.0 (0/300)	35.7 (107/300)	49.3 (148/300)	89.3 (25/28)
FreshCompoundKFour	CodeMapper, quote	12.0 (36/300)	0.0 (0/272)	0.0 (0/300)	28.0 (84/300)	60.0 (180/300)	96.4 (27/28)
FreshCompoundKFour	CodeMapper, IG+AG	12.0 (36/300)	0.0 (0/272)	0.0 (0/300)	28.0 (84/300)	60.0 (180/300)	100.0 (28/28)
FreshCompoundKFour	CodeMapper, IG+AGi	12.0 (36/300)	0.0 (0/272)	0.0 (0/300)	28.0 (84/300)	60.0 (180/300)	100.0 (28/28)
FreshCompoundKFour	Tracker, quote	43.0 (129/300)	0.0 (0/272)	0.0 (0/300)	0.3 (1/300)	56.7 (170/300)	96.4 (27/28)
FreshCompoundKFour	Tracker, IG+AGi	43.0 (129/300)	0.0 (0/272)	0.0 (0/300)	0.3 (1/300)	56.7 (170/300)	100.0 (28/28)

Each cell shows rate (events/its outcome-specific denominator). Human-annotated and language-model cells use only the WP J populations recorded for that configuration. All results are retrospective and descriptive.

TABLE XXXVI
SIX OUTCOMES ACROSS SAVED POPULATIONS (%), PART 4

Population	Configuration	Place	Miss	Silent	Misdirect	Lost	Extra
BeamHumanAb	Cascade, deployed	62.2 (97/156)	8.5 (11/130)	3.5 (7/200)	9.0 (18/200)	27.6 (43/156)	34.3 (24/70)
BeamHumanAb	Cascade, IG+AGi+VW	67.3 (105/156)	3.8 (5/130)	0.5 (1/200)	9.0 (18/200)	26.3 (41/156)	44.3 (31/70)
BeamHumanAb	Cascade, IG+AGi+VW, theta 0.55	87.2 (136/156)	3.8 (5/130)	0.5 (1/200)	21.0 (42/200)	1.9 (3/156)	44.3 (31/70)
BeamHumanAb	CodeMapper, quote	93.6 (146/156)	3.1 (4/130)	0.0 (0/200)	7.5 (15/200)	1.3 (2/156)	35.7 (25/70)
BeamHumanAb	CodeMapper, IG+AG	93.6 (146/156)	0.8 (1/130)	0.0 (0/200)	7.5 (15/200)	1.3 (2/156)	50.0 (35/70)
BeamHumanAb	CodeMapper, IG+AGi	93.6 (146/156)	2.3 (3/130)	0.0 (0/200)	7.5 (15/200)	1.3 (2/156)	44.3 (31/70)
BeamHumanAb	Tracker, quote	82.7 (129/156)	3.8 (5/130)	0.0 (0/200)	3.0 (6/200)	14.7 (23/156)	35.7 (25/70)
BeamHumanAb	Tracker, IG+AGi	82.7 (129/156)	3.1 (4/130)	0.0 (0/200)	3.0 (6/200)	14.7 (23/156)	44.3 (31/70)
BeamHumanAb	Beam, own verdict	91.7 (143/156)	6.2 (8/130)	2.0 (4/200)	3.0 (6/200)	1.3 (2/156)	35.7 (25/70)
BeamHumanAb	Beam, quote	91.7 (143/156)	6.2 (8/130)	1.5 (3/200)	3.5 (7/200)	1.3 (2/156)	37.1 (26/70)
BeamHumanAb	Beam, IG+AGi	91.7 (143/156)	3.1 (4/130)	0.0 (0/200)	5.0 (10/200)	1.3 (2/156)	45.7 (32/70)
BeamRealOssTargeted	Cascade, deployed	65.8 (1,552/2,359)	0.9 (17/1,930)	0.8 (20/2,454)	5.7 (140/2,454)	27.8 (655/2,359)	0.0 (0/524)
BeamRealOssTargeted	Cascade, IG+AGi+VW	72.1 (1,701/2,359)	0.1 (2/1,930)	0.1 (2/2,454)	3.7 (91/2,454)	24.3 (574/2,359)	9.5 (50/524)
BeamRealOssTargeted	Cascade, IG+AGi+VW, theta 0.55	90.9 (2,144/2,359)	0.0 (0/1,930)	0.0 (0/2,454)	8.8 (217/2,454)	1.3 (31/2,359)	10.1 (53/524)
BeamRealOssTargeted	CodeMapper, quote	92.2 (2,174/2,359)	0.4 (8/1,930)	0.3 (8/2,454)	5.5 (136/2,454)	1.7 (39/2,359)	10.1 (53/524)
BeamRealOssTargeted	CodeMapper, IG+AG	92.2 (2,174/2,359)	0.0 (0/1,930)	0.0 (0/2,454)	5.9 (144/2,454)	1.7 (39/2,359)	21.0 (110/524)
BeamRealOssTargeted	CodeMapper, IG+AGi	92.2 (2,174/2,359)	0.0 (0/1,930)	0.0 (0/2,454)	5.9 (144/2,454)	1.7 (39/2,359)	17.6 (92/524)
BeamRealOssTargeted	Tracker, quote	83.5 (1,969/2,359)	0.3 (6/1,930)	0.2 (6/2,454)	2.6 (64/2,454)	11.7 (277/2,359)	13.9 (73/524)
BeamRealOssTargeted	Tracker, IG+AGi	83.5 (1,969/2,359)	0.0 (0/1,930)	0.0 (0/2,454)	2.9 (70/2,454)	11.7 (277/2,359)	20.0 (105/524)
BeamRealOssTargeted	Beam, own verdict	94.4 (2,226/2,359)	2.1 (41/1,930)	0.4 (11/2,454)	3.1 (76/2,454)	3.0 (71/2,359)	11.3 (59/524)
BeamRealOssTargeted	Beam, quote	94.4 (2,226/2,359)	0.4 (8/1,930)	0.3 (8/2,454)	3.2 (79/2,454)	3.0 (71/2,359)	10.5 (55/524)
BeamRealOssTargeted	Beam, IG+AGi	94.4 (2,226/2,359)	0.0 (0/1,930)	0.0 (0/2,454)	3.5 (87/2,454)	3.0 (71/2,359)	18.3 (96/524)
BeamStudentsTargeted	Cascade, deployed	73.6 (1,265/1,719)	2.6 (37/1,402)	2.3 (42/1,857)	6.1 (113/1,857)	19.5 (335/1,719)	0.4 (2/455)
BeamStudentsTargeted	Cascade, IG+AGi+VW	78.6 (1,351/1,719)	0.1 (1/1,402)	0.3 (5/1,857)	5.4 (101/1,857)	17.3 (298/1,719)	25.9 (118/455)

Each cell shows rate (events/its outcome-specific denominator). Human-annotated and language-model cells use only the WP J populations recorded for that configuration. All results are retrospective and descriptive.

TABLE XXXVII
SIX OUTCOMES ACROSS SAVED POPULATIONS (%), PART 5

Population	Configuration	Place	Miss	Silent	Misdirect	Lost	Extra
BeamStudentsTargeted	Cascade, IG+AGi+VW, theta 0.55	90.8 (1,560/1,719)	0.0 (0/1,402)	0.2 (4/1,857)	8.6 (160/1,857)	2.7 (46/1,719)	26.4 (120/455)
BeamStudentsTargeted	CodeMapper, quote	83.2 (1,431/1,719)	0.3 (4/1,402)	0.3 (5/1,857)	10.6 (197/1,857)	5.5 (94/1,719)	29.2 (133/455)
BeamStudentsTargeted	CodeMapper, IG+AG	83.2 (1,431/1,719)	0.0 (0/1,402)	0.1 (1/1,857)	10.8 (201/1,857)	5.5 (94/1,719)	45.3 (206/455)
BeamStudentsTargeted	CodeMapper, IG+AGi	83.2 (1,431/1,719)	0.0 (0/1,402)	0.1 (1/1,857)	10.8 (201/1,857)	5.5 (94/1,719)	40.9 (186/455)
BeamStudentsTargeted	Tracker, quote	84.4 (1,451/1,719)	0.3 (4/1,402)	0.3 (5/1,857)	3.8 (71/1,857)	13.0 (224/1,719)	27.7 (126/455)
BeamStudentsTargeted	Tracker, IG+AGi	84.4 (1,451/1,719)	0.0 (0/1,402)	0.1 (1/1,857)	4.0 (75/1,857)	13.0 (224/1,719)	41.8 (190/455)
BeamStudentsTargeted	Beam, own verdict	90.2 (1,551/1,719)	0.5 (7/1,402)	0.4 (7/1,857)	4.6 (86/1,857)	6.7 (115/1,719)	25.3 (115/455)
BeamStudentsTargeted	Beam, quote	90.2 (1,551/1,719)	0.3 (4/1,402)	0.3 (5/1,857)	4.7 (88/1,857)	6.7 (115/1,719)	25.9 (118/455)
BeamStudentsTargeted	Beam, IG+AGi	90.2 (1,551/1,719)	0.0 (0/1,402)	0.1 (1/1,857)	5.0 (92/1,857)	6.7 (115/1,719)	40.4 (184/455)
NemotronHumanAb	Cascade, deployed	62.2 (97/156)	8.5 (11/130)	3.5 (7/200)	9.0 (18/200)	27.6 (43/156)	34.3 (24/70)
NemotronHumanAb	Cascade, IG+AGi+VW	67.3 (105/156)	3.8 (5/130)	0.5 (1/200)	9.0 (18/200)	26.3 (41/156)	44.3 (31/70)
NemotronHumanAb	Cascade, IG+AGi+VW, theta 0.55	87.2 (136/156)	3.8 (5/130)	0.5 (1/200)	21.0 (42/200)	1.9 (3/156)	44.3 (31/70)
NemotronHumanAb	CodeMapper, quote	93.6 (146/156)	3.1 (4/130)	0.0 (0/200)	7.5 (15/200)	1.3 (2/156)	35.7 (25/70)
NemotronHumanAb	CodeMapper, IG+AG	93.6 (146/156)	0.8 (1/130)	0.0 (0/200)	7.5 (15/200)	1.3 (2/156)	50.0 (35/70)
NemotronHumanAb	CodeMapper, IG+AGi	93.6 (146/156)	2.3 (3/130)	0.0 (0/200)	7.5 (15/200)	1.3 (2/156)	44.3 (31/70)
NemotronHumanAb	Tracker, quote	82.7 (129/156)	3.8 (5/130)	0.0 (0/200)	3.0 (6/200)	14.7 (23/156)	35.7 (25/70)
NemotronHumanAb	Tracker, IG+AGi	82.7 (129/156)	3.1 (4/130)	0.0 (0/200)	3.0 (6/200)	14.7 (23/156)	44.3 (31/70)
NemotronHumanAb	Beam, quote	88.5 (138/156)	5.4 (7/130)	1.5 (3/200)	3.0 (6/200)	3.8 (6/156)	35.7 (25/70)
NemotronHumanAb	Beam, IG+AGi	88.5 (138/156)	2.3 (3/130)	0.0 (0/200)	4.5 (9/200)	3.8 (6/156)	44.3 (31/70)
NemotronHumanAb	Nemotron, own verdict	88.5 (138/156)	5.4 (7/130)	1.5 (3/200)	3.0 (6/200)	3.8 (6/156)	35.7 (25/70)
NemotronHumanAb	Nemotron, quote	88.5 (138/156)	5.4 (7/130)	1.5 (3/200)	3.0 (6/200)	3.8 (6/156)	35.7 (25/70)
NemotronHumanAb	Nemotron, IG+AGi	88.5 (138/156)	2.3 (3/130)	0.0 (0/200)	4.5 (9/200)	3.8 (6/156)	44.3 (31/70)
NemotronRealOssTargeted	Cascade, deployed	65.8 (1,552/2,359)	0.9 (17/1,930)	0.8 (20/2,454)	5.7 (140/2,454)	27.8 (655/2,359)	0.0 (0/524)
NemotronRealOssTargeted	Cascade, IG+AGi+VW	72.1 (1,701/2,359)	0.1 (2/1,930)	0.1 (2/2,454)	3.7 (91/2,454)	24.3 (574/2,359)	9.5 (50/524)

Each cell shows rate (events/its outcome-specific denominator). Human-annotated and language-model cells use only the WP J populations recorded for that configuration. All results are retrospective and descriptive.

TABLE XXXVIII
SIX OUTCOMES ACROSS SAVED POPULATIONS (%), PART 6

Population	Configuration	Place	Miss	Silent	Misdirect	Lost	Extra
NemotronRealOssTargeted	Cascade, IG+AGi+VW, theta 0.55	90.9 (2,144/2,359)	0.0 (0/1,930)	0.0 (0/2,454)	8.8 (217/2,454)	1.3 (31/2,359)	10.1 (53/524)
NemotronRealOssTargeted	CodeMapper, quote	92.2 (2,174/2,359)	0.4 (8/1,930)	0.3 (8/2,454)	5.5 (136/2,454)	1.7 (39/2,359)	10.1 (53/524)
NemotronRealOssTargeted	CodeMapper, IG+AG	92.2 (2,174/2,359)	0.0 (0/1,930)	0.0 (0/2,454)	5.9 (144/2,454)	1.7 (39/2,359)	21.0 (110/524)
NemotronRealOssTargeted	CodeMapper, IG+AGi	92.2 (2,174/2,359)	0.0 (0/1,930)	0.0 (0/2,454)	5.9 (144/2,454)	1.7 (39/2,359)	17.6 (92/524)
NemotronRealOssTargeted	Tracker, quote	83.5 (1,969/2,359)	0.3 (6/1,930)	0.2 (6/2,454)	2.6 (64/2,454)	11.7 (277/2,359)	13.9 (73/524)
NemotronRealOssTargeted	Tracker, IG+AGi	83.5 (1,969/2,359)	0.0 (0/1,930)	0.0 (0/2,454)	2.9 (70/2,454)	11.7 (277/2,359)	20.0 (105/524)
NemotronRealOssTargeted	Beam, quote	95.3 (2,248/2,359)	0.4 (8/1,930)	0.3 (8/2,454)	2.1 (52/2,454)	2.7 (64/2,359)	11.3 (59/524)
NemotronRealOssTargeted	Beam, IG+AGi	95.3 (2,248/2,359)	0.0 (0/1,930)	0.0 (0/2,454)	2.4 (60/2,454)	2.7 (64/2,359)	18.9 (99/524)
NemotronRealOssTargeted	Nemotron, own verdict	95.3 (2,248/2,359)	0.6 (12/1,930)	0.5 (12/2,454)	2.0 (48/2,454)	2.7 (64/2,359)	10.5 (55/524)
NemotronRealOssTargeted	Nemotron, quote	95.3 (2,248/2,359)	0.4 (8/1,930)	0.3 (8/2,454)	2.1 (52/2,454)	2.7 (64/2,359)	11.3 (59/524)
NemotronRealOssTargeted	Nemotron, IG+AGi	95.3 (2,248/2,359)	0.0 (0/1,930)	0.0 (0/2,454)	2.4 (60/2,454)	2.7 (64/2,359)	18.9 (99/524)
NemotronStudentsTargeted	Cascade, deployed	73.6 (1,265/1,719)	2.6 (37/1,402)	2.3 (42/1,857)	6.1 (113/1,857)	19.5 (335/1,719)	0.4 (2/455)
NemotronStudentsTargeted	Cascade, IG+AGi+VW	78.6 (1,351/1,719)	0.1 (1/1,402)	0.3 (5/1,857)	5.4 (101/1,857)	17.3 (298/1,719)	25.9 (118/455)
NemotronStudentsTargeted	Cascade, IG+AGi+VW, theta 0.55	90.8 (1,560/1,719)	0.0 (0/1,402)	0.2 (4/1,857)	8.6 (160/1,857)	2.7 (46/1,719)	26.4 (120/455)
NemotronStudentsTargeted	CodeMapper, quote	83.2 (1,431/1,719)	0.3 (4/1,402)	0.3 (5/1,857)	10.6 (197/1,857)	5.5 (94/1,719)	29.2 (133/455)
NemotronStudentsTargeted	CodeMapper, IG+AG	83.2 (1,431/1,719)	0.0 (0/1,402)	0.1 (1/1,857)	10.8 (201/1,857)	5.5 (94/1,719)	45.3 (206/455)
NemotronStudentsTargeted	CodeMapper, IG+AGi	83.2 (1,431/1,719)	0.0 (0/1,402)	0.1 (1/1,857)	10.8 (201/1,857)	5.5 (94/1,719)	40.9 (186/455)
NemotronStudentsTargeted	Tracker, quote	84.4 (1,451/1,719)	0.3 (4/1,402)	0.3 (5/1,857)	3.8 (71/1,857)	13.0 (224/1,719)	27.7 (126/455)
NemotronStudentsTargeted	Tracker, IG+AGi	84.4 (1,451/1,719)	0.0 (0/1,402)	0.1 (1/1,857)	4.0 (75/1,857)	13.0 (224/1,719)	41.8 (190/455)
NemotronStudentsTargeted	Beam, quote	91.8 (1,578/1,719)	0.3 (4/1,402)	0.3 (5/1,857)	3.4 (64/1,857)	6.6 (114/1,719)	25.3 (115/455)
NemotronStudentsTargeted	Beam, IG+AGi	91.8 (1,578/1,719)	0.0 (0/1,402)	0.1 (1/1,857)	3.7 (68/1,857)	6.6 (114/1,719)	40.4 (184/455)
NemotronStudentsTargeted	Nemotron, own verdict	91.8 (1,578/1,719)	0.6 (8/1,402)	0.3 (5/1,857)	3.4 (64/1,857)	6.6 (114/1,719)	25.3 (115/455)
NemotronStudentsTargeted	Nemotron, quote	91.8 (1,578/1,719)	0.3 (4/1,402)	0.3 (5/1,857)	3.4 (64/1,857)	6.6 (114/1,719)	25.3 (115/455)
NemotronStudentsTargeted	Nemotron, IG+AGi	91.8 (1,578/1,719)	0.0 (0/1,402)	0.1 (1/1,857)	3.7 (68/1,857)	6.6 (114/1,719)	40.4 (184/455)

Each cell shows rate (events/its outcome-specific denominator). Human-annotated and language-model cells use only the WP J populations recorded for that configuration. All results are retrospective and descriptive.

TABLE XXXIX
TEN-HOP CHAIN OUTCOMES

Base	Correct	Wrong	Lost	Missed-Last	Silent-Wrong-Last
v0.85-off	3,575	90	335	563	85
v0.85-alli	3,670	49	281	609	46
v0.55-alli	3,878	102	20	858	93
cm-direct	3,908	80	12	3	3
cm-direct-gi	3,908	80	12	0	0

Each cell is a count among N=4,000 chain rows. Missed-last and silent-wrong-last are endpoint request outcomes. Retrospective and descriptive.

TABLE XL
ANCHOR-LENGTH AND CONFIDENCE-BY-GUARD STRATA, PART 1

Population	Stratum	Configuration	Outcome	Rate (k/n)
real	1	t0.85-deployed	missed	2.4 (27/1,120)
real	1	t0.85-deployed	silent	0.4 (30/8,549)
real	1	t0.85-deployed	misdirected	0.4 (36/8,549)
real	1	t0.85-deployed	unnecessary	0.0 (0/7,429)
real	1	t0.85-IG+AGi+VW	missed	0.3 (3/1,120)
real	1	t0.85-IG+AGi+VW	silent	0.0 (3/8,549)
real	1	t0.85-IG+AGi+VW	misdirected	0.8 (66/8,549)
real	1	t0.85-IG+AGi+VW	unnecessary	0.8 (60/7,429)
real	1	codemapper+quote	missed	0.8 (9/1,120)
real	1	codemapper+quote	silent	0.1 (9/8,549)
real	1	codemapper+quote	misdirected	2.0 (169/8,549)
real	1	codemapper+quote	unnecessary	0.9 (70/7,429)
real	1	codemapper+IG+AG	missed	0.0 (0/1,120)
real	1	codemapper+IG+AG	silent	0.0 (0/8,549)
real	1	codemapper+IG+AG	misdirected	2.1 (178/8,549)
real	1	codemapper+IG+AG	unnecessary	2.1 (154/7,429)
real	1	codemapper+IG+AGi	missed	0.0 (0/1,120)
real	1	codemapper+IG+AGi	silent	0.0 (0/8,549)
real	1	codemapper+IG+AGi	misdirected	2.1 (178/8,549)
real	1	codemapper+IG+AGi	unnecessary	1.5 (110/7,429)
real	2-3	t0.85-deployed	missed	0.3 (2/616)
real	2-3	t0.85-deployed	silent	0.1 (2/3,143)
real	2-3	t0.85-deployed	misdirected	1.1 (36/3,143)
real	2-3	t0.85-deployed	unnecessary	0.0 (0/2,527)

Span bins are 1, 2-3, 4-9, and at least 10 lines. The summary also contains cascade-tier by guard-fire outcome cross-tabs for real, student, and fresh rows. CodeMapper has no stored confidence field; old-copy strata and multi-anchor issue outcomes are unavailable in the saved row schema.

TABLE XLI
ANCHOR-LENGTH AND CONFIDENCE-BY-GUARD STRATA, PART 2

Population	Stratum	Configuration	Outcome	Rate (k/n)
real	2-3	t0.85-IG+AGi+VW	missed	0.0 (0/616)
real	2-3	t0.85-IG+AGi+VW	silent	0.0 (0/3,143)
real	2-3	t0.85-IG+AGi+VW	misdirected	0.5 (16/3,143)
real	2-3	t0.85-IG+AGi+VW	unnecessary	0.7 (17/2,527)
real	2-3	codemapper+quote	missed	0.0 (0/616)
real	2-3	codemapper+quote	silent	0.0 (0/3,143)
real	2-3	codemapper+quote	misdirected	1.0 (32/3,143)
real	2-3	codemapper+quote	unnecessary	0.6 (16/2,527)
real	2-3	codemapper+IG+AG	missed	0.0 (0/616)
real	2-3	codemapper+IG+AG	silent	0.0 (0/3,143)
real	2-3	codemapper+IG+AG	misdirected	1.0 (32/3,143)
real	2-3	codemapper+IG+AG	unnecessary	1.7 (42/2,527)
real	2-3	codemapper+IG+AGi	missed	0.0 (0/616)
real	2-3	codemapper+IG+AGi	silent	0.0 (0/3,143)
real	2-3	codemapper+IG+AGi	misdirected	1.0 (32/3,143)
real	2-3	codemapper+IG+AGi	unnecessary	1.2 (30/2,527)
real	4-9	t0.85-deployed	missed	0.6 (6/968)
real	4-9	t0.85-deployed	silent	0.2 (6/3,127)
real	4-9	t0.85-deployed	misdirected	5.0 (155/3,127)
real	4-9	t0.85-deployed	unnecessary	0.0 (0/2,159)
real	4-9	t0.85-IG+AGi+VW	missed	0.1 (1/968)
real	4-9	t0.85-IG+AGi+VW	silent	0.0 (1/3,127)
real	4-9	t0.85-IG+AGi+VW	misdirected	1.9 (58/3,127)
real	4-9	t0.85-IG+AGi+VW	unnecessary	0.5 (11/2,159)

Span bins are 1, 2-3, 4-9, and at least 10 lines. The summary also contains cascade-tier by guard-fire outcome cross-tabs for real, student, and fresh rows. CodeMapper has no stored confidence field; old-copy strata and multi-anchor issue outcomes are unavailable in the saved row schema.

TABLE XLII
ANCHOR-LENGTH AND CONFIDENCE-BY-GUARD STRATA, PART 3

Population	Stratum	Configuration	Outcome	Rate (k/n)
real	4-9	codemapper+quote	missed	0.3 (3/968)
real	4-9	codemapper+quote	silent	0.1 (3/3,127)
real	4-9	codemapper+quote	misdirected	0.8 (26/3,127)
real	4-9	codemapper+quote	unnecessary	0.6 (13/2,159)
real	4-9	codemapper+IG+AG	missed	0.1 (1/968)
real	4-9	codemapper+IG+AG	silent	0.0 (1/3,127)
real	4-9	codemapper+IG+AG	misdirected	0.9 (28/3,127)
real	4-9	codemapper+IG+AG	unnecessary	1.6 (34/2,159)
real	4-9	codemapper+IG+AGi	missed	0.1 (1/968)
real	4-9	codemapper+IG+AGi	silent	0.0 (1/3,127)
real	4-9	codemapper+IG+AGi	misdirected	0.9 (28/3,127)
real	4-9	codemapper+IG+AGi	unnecessary	0.9 (20/2,159)
real	>=10	t0.85-deployed	missed	2.6 (2/76)
real	>=10	t0.85-deployed	silent	0.8 (2/245)
real	>=10	t0.85-deployed	misdirected	8.6 (21/245)
real	>=10	t0.85-deployed	unnecessary	0.0 (0/169)
real	>=10	t0.85-IG+AGi+VW	missed	0.0 (0/76)
real	>=10	t0.85-IG+AGi+VW	silent	0.0 (0/245)
real	>=10	t0.85-IG+AGi+VW	misdirected	2.4 (6/245)
real	>=10	t0.85-IG+AGi+VW	unnecessary	0.0 (0/169)
real	>=10	codemapper+quote	missed	0.0 (0/76)
real	>=10	codemapper+quote	silent	0.0 (0/245)
real	>=10	codemapper+quote	misdirected	0.0 (0/245)
real	>=10	codemapper+quote	unnecessary	0.0 (0/169)

Span bins are 1, 2-3, 4-9, and at least 10 lines. The summary also contains cascade-tier by guard-fire outcome cross-tabs for real, student, and fresh rows. CodeMapper has no stored confidence field; old-copy strata and multi-anchor issue outcomes are unavailable in the saved row schema.

TABLE XLIII
ANCHOR-LENGTH AND CONFIDENCE-BY-GUARD STRATA, PART 4

Population	Stratum	Configuration	Outcome	Rate (k/n)
real	>=10	codemapper+IG+AG	missed	0.0 (0/76)
real	>=10	codemapper+IG+AG	silent	0.0 (0/245)
real	>=10	codemapper+IG+AG	misdirected	0.0 (0/245)
real	>=10	codemapper+IG+AG	unnecessary	0.0 (0/169)
real	>=10	codemapper+IG+AGi	missed	0.0 (0/76)
real	>=10	codemapper+IG+AGi	silent	0.0 (0/245)
real	>=10	codemapper+IG+AGi	misdirected	0.0 (0/245)
real	>=10	codemapper+IG+AGi	unnecessary	0.0 (0/169)
real	same_exact|guard=False	Cascade deployed	missed	1/1
real	same_exact|guard=False	Cascade deployed	wrong_silent	1/12,108
real	same_exact|guard=False	Cascade deployed	wrong_with_recheck	0/12,108
real	same_exact|guard=False	Cascade deployed	false_recheck	0/12,107
real	same_exact|guard=False	Cascade deployed	lost	0/12,108
real	variable|guard=False	Cascade deployed	missed	0/1,810
real	variable|guard=False	Cascade deployed	wrong_silent	0/1,810
real	variable|guard=False	Cascade deployed	wrong_with_recheck	114/1,810
real	variable|guard=False	Cascade deployed	false_recheck	0/0
real	variable|guard=False	Cascade deployed	lost	0/1,805
real	no_match|guard=False	Cascade deployed	missed	0/937
real	no_match|guard=False	Cascade deployed	wrong_silent	0/937
real	no_match|guard=False	Cascade deployed	wrong_with_recheck	0/937
real	no_match|guard=False	Cascade deployed	false_recheck	0/0
real	no_match|guard=False	Cascade deployed	lost	805/805
real	cross_exact|guard=False	Cascade deployed	missed	3/3

Span bins are 1, 2-3, 4-9, and at least 10 lines. The summary also contains cascade-tier by guard-fire outcome cross-tabs for real, student, and fresh rows. CodeMapper has no stored confidence field; old-copy strata and multi-anchor issue outcomes are unavailable in the saved row schema.

TABLE XLIV
ANCHOR-LENGTH AND CONFIDENCE-BY-GUARD STRATA, PART 5

Population	Stratum	Configuration	Outcome	Rate (k/n)
real	cross_exact|guard=False	Cascade deployed	wrong_silent	3/92
real	cross_exact|guard=False	Cascade deployed	wrong_with_recheck	0/92
real	cross_exact|guard=False	Cascade deployed	false_recheck	0/89
real	cross_exact|guard=False	Cascade deployed	lost	0/92
real	cross_exact|guard=True	Cascade deployed	missed	0/16
real	cross_exact|guard=True	Cascade deployed	wrong_silent	0/38
real	cross_exact|guard=True	Cascade deployed	wrong_with_recheck	19/38
real	cross_exact|guard=True	Cascade deployed	false_recheck	22/22
real	cross_exact|guard=True	Cascade deployed	lost	0/38
real	same_exact|guard=True	Cascade deployed	missed	0/13
real	same_exact|guard=True	Cascade deployed	wrong_silent	0/79
real	same_exact|guard=True	Cascade deployed	wrong_with_recheck	13/79
real	same_exact|guard=True	Cascade deployed	false_recheck	66/66
real	same_exact|guard=True	Cascade deployed	lost	0/72
students	1	t0.85-deployed	missed	3.3 (26/788)
students	1	t0.85-deployed	silent	1.3 (64/4,790)
students	1	t0.85-deployed	misdirected	1.1 (52/4,790)
students	1	t0.85-deployed	unnecessary	0.1 (3/4,002)
students	1	t0.85-IG+AGi+VW	missed	0.3 (2/788)
students	1	t0.85-IG+AGi+VW	silent	0.4 (19/4,790)
students	1	t0.85-IG+AGi+VW	misdirected	2.0 (96/4,790)
students	1	t0.85-IG+AGi+VW	unnecessary	3.7 (148/4,002)
students	1	codemapper+quote	missed	1.1 (9/788)
students	1	codemapper+quote	silent	0.3 (13/4,790)

Span bins are 1, 2-3, 4-9, and at least 10 lines. The summary also contains cascade-tier by guard-fire outcome cross-tabs for real, student, and fresh rows. CodeMapper has no stored confidence field; old-copy strata and multi-anchor issue outcomes are unavailable in the saved row schema.

TABLE XLV
ANCHOR-LENGTH AND CONFIDENCE-BY-GUARD STRATA, PART 6

Population	Stratum	Configuration	Outcome	Rate (k/n)
students	1	codemapper+quote	misdirected	4.2 (199/4,790)
students	1	codemapper+quote	unnecessary	5.2 (207/4,002)
students	1	codemapper+IG+AG	missed	0.3 (2/788)
students	1	codemapper+IG+AG	silent	0.1 (6/4,790)
students	1	codemapper+IG+AG	misdirected	4.3 (206/4,790)
students	1	codemapper+IG+AG	unnecessary	7.6 (304/4,002)
students	1	codemapper+IG+AGi	missed	0.3 (2/788)
students	1	codemapper+IG+AGi	silent	0.1 (6/4,790)
students	1	codemapper+IG+AGi	misdirected	4.3 (206/4,790)
students	1	codemapper+IG+AGi	unnecessary	6.5 (262/4,002)
students	2-3	t0.85-deployed	missed	5.1 (17/331)
students	2-3	t0.85-deployed	silent	1.8 (23/1,313)
students	2-3	t0.85-deployed	misdirected	1.4 (18/1,313)
students	2-3	t0.85-deployed	unnecessary	0.0 (0/982)
students	2-3	t0.85-IG+AGi+VW	missed	0.0 (0/331)
students	2-3	t0.85-IG+AGi+VW	silent	0.2 (3/1,313)
students	2-3	t0.85-IG+AGi+VW	misdirected	2.1 (28/1,313)
students	2-3	t0.85-IG+AGi+VW	unnecessary	2.6 (26/982)
students	2-3	codemapper+quote	missed	0.0 (0/331)
students	2-3	codemapper+quote	silent	0.0 (0/1,313)
students	2-3	codemapper+quote	misdirected	3.3 (43/1,313)
students	2-3	codemapper+quote	unnecessary	7.4 (73/982)
students	2-3	codemapper+IG+AG	missed	0.0 (0/331)
students	2-3	codemapper+IG+AG	silent	0.0 (0/1,313)

Span bins are 1, 2-3, 4-9, and at least 10 lines. The summary also contains cascade-tier by guard-fire outcome cross-tabs for real, student, and fresh rows. CodeMapper has no stored confidence field; old-copy strata and multi-anchor issue outcomes are unavailable in the saved row schema.

TABLE XLVI
ANCHOR-LENGTH AND CONFIDENCE-BY-GUARD STRATA, PART 7

Population	Stratum	Configuration	Outcome	Rate (k/n)
students	2-3	codemapper+IG+AG	misdirected	3.3 (43/1,313)
students	2-3	codemapper+IG+AG	unnecessary	9.0 (88/982)
students	2-3	codemapper+IG+AGi	missed	0.0 (0/331)
students	2-3	codemapper+IG+AGi	silent	0.0 (0/1,313)
students	2-3	codemapper+IG+AGi	misdirected	3.3 (43/1,313)
students	2-3	codemapper+IG+AGi	unnecessary	8.1 (80/982)
students	4-9	t0.85-deployed	missed	2.0 (12/598)
students	4-9	t0.85-deployed	silent	1.8 (31/1,679)
students	4-9	t0.85-deployed	misdirected	3.0 (51/1,679)
students	4-9	t0.85-deployed	unnecessary	0.0 (0/1,081)
students	4-9	t0.85-IG+AGi+VW	missed	0.3 (2/598)
students	4-9	t0.85-IG+AGi+VW	silent	0.7 (12/1,679)
students	4-9	t0.85-IG+AGi+VW	misdirected	2.0 (33/1,679)
students	4-9	t0.85-IG+AGi+VW	unnecessary	4.9 (53/1,081)
students	4-9	codemapper+quote	missed	0.0 (0/598)
students	4-9	codemapper+quote	silent	0.0 (0/1,679)
students	4-9	codemapper+quote	misdirected	2.0 (34/1,679)
students	4-9	codemapper+quote	unnecessary	8.4 (91/1,081)
students	4-9	codemapper+IG+AG	missed	0.0 (0/598)
students	4-9	codemapper+IG+AG	silent	0.0 (0/1,679)
students	4-9	codemapper+IG+AG	misdirected	2.0 (34/1,679)
students	4-9	codemapper+IG+AG	unnecessary	10.5 (114/1,081)
students	4-9	codemapper+IG+AGi	missed	0.0 (0/598)
students	4-9	codemapper+IG+AGi	silent	0.0 (0/1,679)

Span bins are 1, 2-3, 4-9, and at least 10 lines. The summary also contains cascade-tier by guard-fire outcome cross-tabs for real, student, and fresh rows. CodeMapper has no stored confidence field; old-copy strata and multi-anchor issue outcomes are unavailable in the saved row schema.

TABLE XLVII
ANCHOR-LENGTH AND CONFIDENCE-BY-GUARD STRATA, PART 8

Population	Stratum	Configuration	Outcome	Rate (k/n)
students	4-9	codemapper+IG+AGi	misdirected	2.0 (34/1,679)
students	4-9	codemapper+IG+AGi	unnecessary	10.0 (108/1,081)
students	>=10	t0.85-deployed	missed	0.0 (0/53)
students	>=10	t0.85-deployed	silent	0.0 (0/100)
students	>=10	t0.85-deployed	misdirected	14.0 (14/100)
students	>=10	t0.85-deployed	unnecessary	0.0 (0/47)
students	>=10	t0.85-IG+AGi+VW	missed	0.0 (0/53)
students	>=10	t0.85-IG+AGi+VW	silent	0.0 (0/100)
students	>=10	t0.85-IG+AGi+VW	misdirected	4.0 (4/100)
students	>=10	t0.85-IG+AGi+VW	unnecessary	6.4 (3/47)
students	>=10	codemapper+quote	missed	0.0 (0/53)
students	>=10	codemapper+quote	silent	0.0 (0/100)
students	>=10	codemapper+quote	misdirected	4.0 (4/100)
students	>=10	codemapper+quote	unnecessary	6.4 (3/47)
students	>=10	codemapper+IG+AG	missed	0.0 (0/53)
students	>=10	codemapper+IG+AG	silent	0.0 (0/100)
students	>=10	codemapper+IG+AG	misdirected	4.0 (4/100)
students	>=10	codemapper+IG+AG	unnecessary	6.4 (3/47)
students	>=10	codemapper+IG+AGi	missed	0.0 (0/53)
students	>=10	codemapper+IG+AGi	silent	0.0 (0/100)
students	>=10	codemapper+IG+AGi	misdirected	4.0 (4/100)
students	>=10	codemapper+IG+AGi	unnecessary	6.4 (3/47)
students	same_exact|guard=False	Cascade deployed	missed	2/2
students	same_exact|guard=False	Cascade deployed	wrong_silent	6/5,566

Span bins are 1, 2-3, 4-9, and at least 10 lines. The summary also contains cascade-tier by guard-fire outcome cross-tabs for real, student, and fresh rows. CodeMapper has no stored confidence field; old-copy strata and multi-anchor issue outcomes are unavailable in the saved row schema.

TABLE XLVIII
ANCHOR-LENGTH AND CONFIDENCE-BY-GUARD STRATA, PART 9

Population	Stratum	Configuration	Outcome	Rate (k/n)
students	same_exact|guard=False	Cascade deployed	wrong_with_recheck	0/5,566
students	same_exact|guard=False	Cascade deployed	false_recheck	0/5,564
students	same_exact|guard=False	Cascade deployed	lost	0/5,566
students	variable|guard=False	Cascade deployed	missed	0/1,183
students	variable|guard=False	Cascade deployed	wrong_silent	0/1,186
students	variable|guard=False	Cascade deployed	wrong_with_recheck	92/1,186
students	variable|guard=False	Cascade deployed	false_recheck	3/3
students	variable|guard=False	Cascade deployed	lost	0/1,150
students	no_match|guard=False	Cascade deployed	missed	0/547
students	no_match|guard=False	Cascade deployed	wrong_silent	0/547
students	no_match|guard=False	Cascade deployed	wrong_with_recheck	0/547
students	no_match|guard=False	Cascade deployed	false_recheck	0/0
students	no_match|guard=False	Cascade deployed	lost	374/374
students	same_exact|guard=True	Cascade deployed	missed	0/7
students	same_exact|guard=True	Cascade deployed	wrong_silent	0/108
students	same_exact|guard=True	Cascade deployed	wrong_with_recheck	7/108
students	same_exact|guard=True	Cascade deployed	false_recheck	101/101
students	same_exact|guard=True	Cascade deployed	lost	0/104
students	cross_exact|guard=False	Cascade deployed	missed	2/2
students	cross_exact|guard=False	Cascade deployed	wrong_silent	28/309
students	cross_exact|guard=False	Cascade deployed	wrong_with_recheck	0/309
students	cross_exact|guard=False	Cascade deployed	false_recheck	0/307
students	cross_exact|guard=False	Cascade deployed	lost	0/309
students	cross_exact|guard=True	Cascade deployed	missed	0/29

Span bins are 1, 2-3, 4-9, and at least 10 lines. The summary also contains cascade-tier by guard-fire outcome cross-tabs for real, student, and fresh rows. CodeMapper has no stored confidence field; old-copy strata and multi-anchor issue outcomes are unavailable in the saved row schema.

TABLE XLIX
ANCHOR-LENGTH AND CONFIDENCE-BY-GUARD STRATA, PART 10

Population	Stratum	Configuration	Outcome	Rate (k/n)
students	cross_exact|guard=True	Cascade deployed	wrong_silent	0/155
students	cross_exact|guard=True	Cascade deployed	wrong_with_recheck	62/155
students	cross_exact|guard=True	Cascade deployed	false_recheck	126/126
students	cross_exact|guard=True	Cascade deployed	lost	0/147
students	hash|guard=False	Cascade deployed	missed	0/0
students	hash|guard=False	Cascade deployed	wrong_silent	0/11
students	hash|guard=False	Cascade deployed	wrong_with_recheck	0/11
students	hash|guard=False	Cascade deployed	false_recheck	0/11
students	hash|guard=False	Cascade deployed	lost	0/11
fresh	1	t0.85-deployed	missed	70.8 (3,947/5,572)
fresh	1	t0.85-deployed	silent	37.9 (6,322/16,667)
fresh	1	t0.85-deployed	misdirected	0.6 (93/16,667)
fresh	1	t0.85-deployed	unnecessary	0.3 (32/11,095)
fresh	1	t0.85-IG+AGi+VW	missed	0.3 (15/5,572)
fresh	1	t0.85-IG+AGi+VW	silent	0.4 (60/16,667)
fresh	1	t0.85-IG+AGi+VW	misdirected	35.0 (5,831/16,667)
fresh	1	t0.85-IG+AGi+VW	unnecessary	34.0 (3,768/11,095)
fresh	1	codemapper+quote	missed	1.2 (69/5,572)
fresh	1	codemapper+quote	silent	0.8 (126/16,667)
fresh	1	codemapper+quote	misdirected	5.2 (861/16,667)
fresh	1	codemapper+quote	unnecessary	13.6 (1,510/11,095)
fresh	1	codemapper+IG+AG	missed	0.1 (3/5,572)
fresh	1	codemapper+IG+AG	silent	0.2 (26/16,667)
fresh	1	codemapper+IG+AG	misdirected	5.8 (961/16,667)

Span bins are 1, 2-3, 4-9, and at least 10 lines. The summary also contains cascade-tier by guard-fire outcome cross-tabs for real, student, and fresh rows. CodeMapper has no stored confidence field; old-copy strata and multi-anchor issue outcomes are unavailable in the saved row schema.

TABLE L
ANCHOR-LENGTH AND CONFIDENCE-BY-GUARD STRATA, PART 11

Population	Stratum	Configuration	Outcome	Rate (k/n)
fresh	1	codemapper+IG+AG	unnecessary	37.7 (4,185/11,095)
fresh	1	codemapper+IG+AGi	missed	0.1 (6/5,572)
fresh	1	codemapper+IG+AGi	silent	0.2 (29/16,667)
fresh	1	codemapper+IG+AGi	misdirected	5.7 (958/16,667)
fresh	1	codemapper+IG+AGi	unnecessary	37.5 (4,160/11,095)
fresh	2-3	t0.85-deployed	missed	72.5 (623/859)
fresh	2-3	t0.85-deployed	silent	34.4 (626/1,822)
fresh	2-3	t0.85-deployed	misdirected	1.9 (35/1,822)
fresh	2-3	t0.85-deployed	unnecessary	4.4 (42/963)
fresh	2-3	t0.85-IG+AGi+VW	missed	8.7 (75/859)
fresh	2-3	t0.85-IG+AGi+VW	silent	13.5 (246/1,822)
fresh	2-3	t0.85-IG+AGi+VW	misdirected	21.3 (388/1,822)
fresh	2-3	t0.85-IG+AGi+VW	unnecessary	32.7 (315/963)
fresh	2-3	codemapper+quote	missed	48.2 (414/859)
fresh	2-3	codemapper+quote	silent	20.7 (377/1,822)
fresh	2-3	codemapper+quote	misdirected	3.3 (60/1,822)
fresh	2-3	codemapper+quote	unnecessary	15.3 (147/963)
fresh	2-3	codemapper+IG+AG	missed	2.8 (24/859)
fresh	2-3	codemapper+IG+AG	silent	7.0 (128/1,822)
fresh	2-3	codemapper+IG+AG	misdirected	17.0 (309/1,822)
fresh	2-3	codemapper+IG+AG	unnecessary	42.3 (407/963)
fresh	2-3	codemapper+IG+AGi	missed	4.5 (39/859)
fresh	2-3	codemapper+IG+AGi	silent	8.8 (160/1,822)
fresh	2-3	codemapper+IG+AGi	misdirected	15.2 (277/1,822)

Span bins are 1, 2-3, 4-9, and at least 10 lines. The summary also contains cascade-tier by guard-fire outcome cross-tabs for real, student, and fresh rows. CodeMapper has no stored confidence field; old-copy strata and multi-anchor issue outcomes are unavailable in the saved row schema.

TABLE LI
ANCHOR-LENGTH AND CONFIDENCE-BY-GUARD STRATA, PART 12

Population	Stratum	Configuration	Outcome	Rate (k/n)
fresh	2-3	codemapper+IG+AGi	unnecessary	39.5 (380/963)
fresh	4-9	t0.85-deployed	missed	73.0 (1,602/2,194)
fresh	4-9	t0.85-deployed	silent	35.0 (1,575/4,504)
fresh	4-9	t0.85-deployed	misdirected	1.7 (77/4,504)
fresh	4-9	t0.85-deployed	unnecessary	2.3 (54/2,310)
fresh	4-9	t0.85-IG+AGi+VW	missed	13.2 (290/2,194)
fresh	4-9	t0.85-IG+AGi+VW	silent	17.2 (774/4,504)
fresh	4-9	t0.85-IG+AGi+VW	misdirected	16.6 (747/4,504)
fresh	4-9	t0.85-IG+AGi+VW	unnecessary	27.4 (634/2,310)
fresh	4-9	codemapper+quote	missed	48.3 (1,060/2,194)
fresh	4-9	codemapper+quote	silent	21.9 (986/4,504)
fresh	4-9	codemapper+quote	misdirected	3.0 (137/4,504)
fresh	4-9	codemapper+quote	unnecessary	11.9 (274/2,310)
fresh	4-9	codemapper+IG+AG	missed	5.3 (116/2,194)
fresh	4-9	codemapper+IG+AG	silent	9.7 (438/4,504)
fresh	4-9	codemapper+IG+AG	misdirected	15.2 (685/4,504)
fresh	4-9	codemapper+IG+AG	unnecessary	37.6 (868/2,310)
fresh	4-9	codemapper+IG+AGi	missed	8.6 (188/2,194)
fresh	4-9	codemapper+IG+AGi	silent	12.6 (567/4,504)
fresh	4-9	codemapper+IG+AGi	misdirected	12.3 (556/4,504)
fresh	4-9	codemapper+IG+AGi	unnecessary	33.7 (779/2,310)
fresh	>=10	t0.85-deployed	missed	78.4 (214/273)
fresh	>=10	t0.85-deployed	silent	36.6 (204/557)
fresh	>=10	t0.85-deployed	misdirected	1.6 (9/557)

Span bins are 1, 2-3, 4-9, and at least 10 lines. The summary also contains cascade-tier by guard-fire outcome cross-tabs for real, student, and fresh rows. CodeMapper has no stored confidence field; old-copy strata and multi-anchor issue outcomes are unavailable in the saved row schema.

TABLE LII
ANCHOR-LENGTH AND CONFIDENCE-BY-GUARD STRATA, PART 13

Population	Stratum	Configuration	Outcome	Rate (k/n)
fresh	>=10	t0.85-deployed	unnecessary	3.2 (9/284)
fresh	>=10	t0.85-IG+AGi+VW	missed	21.2 (58/273)
fresh	>=10	t0.85-IG+AGi+VW	silent	24.1 (134/557)
fresh	>=10	t0.85-IG+AGi+VW	misdirected	13.5 (75/557)
fresh	>=10	t0.85-IG+AGi+VW	unnecessary	29.2 (83/284)
fresh	>=10	codemapper+quote	missed	63.0 (172/273)
fresh	>=10	codemapper+quote	silent	29.3 (163/557)
fresh	>=10	codemapper+quote	misdirected	2.5 (14/557)
fresh	>=10	codemapper+quote	unnecessary	13.7 (39/284)
fresh	>=10	codemapper+IG+AG	missed	13.2 (36/273)
fresh	>=10	codemapper+IG+AG	silent	19.0 (106/557)
fresh	>=10	codemapper+IG+AG	misdirected	12.7 (71/557)
fresh	>=10	codemapper+IG+AG	unnecessary	37.7 (107/284)
fresh	>=10	codemapper+IG+AGi	missed	16.1 (44/273)
fresh	>=10	codemapper+IG+AGi	silent	20.5 (114/557)
fresh	>=10	codemapper+IG+AGi	misdirected	11.3 (63/557)
fresh	>=10	codemapper+IG+AGi	unnecessary	36.6 (104/284)
fresh	same_exact|guard=True	Cascade deployed	missed	0/5,711
fresh	same_exact|guard=True	Cascade deployed	wrong_silent	0/10,234
fresh	same_exact|guard=True	Cascade deployed	wrong_with_recheck	6,615/10,234
fresh	same_exact|guard=True	Cascade deployed	false_recheck	4,523/4,523
fresh	same_exact|guard=True	Cascade deployed	lost	0/8,059
fresh	same_exact|guard=False	Cascade deployed	missed	212/212
fresh	same_exact|guard=False	Cascade deployed	wrong_silent	846/7,932

Span bins are 1, 2-3, 4-9, and at least 10 lines. The summary also contains cascade-tier by guard-fire outcome cross-tabs for real, student, and fresh rows. CodeMapper has no stored confidence field; old-copy strata and multi-anchor issue outcomes are unavailable in the saved row schema.

TABLE LIII
ANCHOR-LENGTH AND CONFIDENCE-BY-GUARD STRATA, PART 14

Population	Stratum	Configuration	Outcome	Rate (k/n)
fresh	same_exact|guard=False	Cascade deployed	wrong_with_recheck	0/7,932
fresh	same_exact|guard=False	Cascade deployed	false_recheck	0/7,720
fresh	same_exact|guard=False	Cascade deployed	lost	0/7,788
fresh	hash|guard=False	Cascade deployed	missed	83/83
fresh	hash|guard=False	Cascade deployed	wrong_silent	161/1,661
fresh	hash|guard=False	Cascade deployed	wrong_with_recheck	0/1,661
fresh	hash|guard=False	Cascade deployed	false_recheck	0/1,578
fresh	hash|guard=False	Cascade deployed	lost	0/1,578
fresh	variable|guard=False	Cascade deployed	missed	0/1,279
fresh	variable|guard=False	Cascade deployed	wrong_silent	0/1,366
fresh	variable|guard=False	Cascade deployed	wrong_with_recheck	132/1,366
fresh	variable|guard=False	Cascade deployed	false_recheck	87/87
fresh	variable|guard=False	Cascade deployed	lost	0/1,305
fresh	cross_exact|guard=False	Cascade deployed	missed	143/143
fresh	cross_exact|guard=False	Cascade deployed	wrong_silent	207/697
fresh	cross_exact|guard=False	Cascade deployed	wrong_with_recheck	0/697
fresh	cross_exact|guard=False	Cascade deployed	false_recheck	0/554
fresh	cross_exact|guard=False	Cascade deployed	lost	0/555
fresh	cross_exact|guard=True	Cascade deployed	missed	0/237
fresh	cross_exact|guard=True	Cascade deployed	wrong_silent	0/376
fresh	cross_exact|guard=True	Cascade deployed	wrong_with_recheck	294/376
fresh	cross_exact|guard=True	Cascade deployed	false_recheck	139/139
fresh	cross_exact|guard=True	Cascade deployed	lost	0/221
fresh	no_match|guard=False	Cascade deployed	missed	0/1,233

Span bins are 1, 2-3, 4-9, and at least 10 lines. The summary also contains cascade-tier by guard-fire outcome cross-tabs for real, student, and fresh rows. CodeMapper has no stored confidence field; old-copy strata and multi-anchor issue outcomes are unavailable in the saved row schema.

TABLE LIV
ANCHOR-LENGTH AND CONFIDENCE-BY-GUARD STRATA, PART 15

Population	Stratum	Configuration	Outcome	Rate (k/n)
fresh	no_match|guard=False	Cascade deployed	wrong_silent	0/1,284
fresh	no_match|guard=False	Cascade deployed	wrong_with_recheck	0/1,284
fresh	no_match|guard=False	Cascade deployed	false_recheck	51/51
fresh	no_match|guard=False	Cascade deployed	lost	594/594

Span bins are 1, 2-3, 4-9, and at least 10 lines. The summary also contains cascade-tier by guard-fire outcome cross-tabs for real, student, and fresh rows. CodeMapper has no stored confidence field; old-copy strata and multi-anchor issue outcomes are unavailable in the saved row schema.

TABLE LV
IDENTITY-FLOOR SENSITIVITY AT TEN HOPS, PART 1

Base	Floor	N	Correct	Wrong	Lost	Missed-last	Silent-wrong
chains:v0.85-off	0.55	4,000	3,575	88	335	8	8
chains:v0.85-off	0.6	4,000	3,575	88	335	8	8
chains:v0.85-off	0.65	4,000	3,575	88	335	8	8
chains:v0.85-off	0.7	4,000	3,575	88	335	8	8
chains:v0.85-off	0.75	4,000	3,575	88	335	8	8
chains:v0.85-off	0.8	4,000	3,575	88	335	8	8
chains:v0.85-off	0.85	4,000	3,573	88	337	8	8
chains:v0.85-off	0.9	4,000	3,457	59	482	8	8
chains:v0.85-off	0.95	4,000	3,314	34	650	8	8
chains:v0.85-off	1.0	4,000	3,045	8	945	8	8
chains:v0.85-off	0.3	4,000	3,575	88	335	8	8
chains:v0.85-off	0.35	4,000	3,575	88	335	8	8
chains:v0.85-off	0.4	4,000	3,575	88	335	8	8
chains:v0.85-off	0.45	4,000	3,575	88	335	8	8
chains:v0.85-off	0.5	4,000	3,575	88	335	8	8
chains:v0.85-all	0.55	4,000	3,670	47	281	8	7
chains:v0.85-all	0.6	4,000	3,670	47	281	8	7
chains:v0.85-all	0.65	4,000	3,670	47	281	8	7
chains:v0.85-all	0.7	4,000	3,670	47	281	8	7
chains:v0.85-all	0.75	4,000	3,670	47	281	8	7
chains:v0.85-all	0.8	4,000	3,670	47	281	8	7
chains:v0.85-all	0.85	4,000	3,668	47	283	8	7
chains:v0.85-all	0.9	4,000	3,546	27	425	8	7
chains:v0.85-all	0.95	4,000	3,373	16	609	8	7

Counts are endpoint outcomes at h=10. Cascade traces replay from saved identity traces; CodeMapper replay is limited to floors at or above 0.50. Gate mismatches and ratio distributions are recorded in the summary.

TABLE LVI
IDENTITY-FLOOR SENSITIVITY AT TEN HOPS, PART 2

Base	Floor	N	Correct	Wrong	Lost	Missed-last	Silent-wrong
chains:v0.85-all	1.0	4,000	3,045	8	945	8	7
chains:v0.85-all	0.3	4,000	3,670	47	281	8	7
chains:v0.85-all	0.35	4,000	3,670	47	281	8	7
chains:v0.85-all	0.4	4,000	3,670	47	281	8	7
chains:v0.85-all	0.45	4,000	3,670	47	281	8	7
chains:v0.85-all	0.5	4,000	3,670	47	281	8	7
chains:v0.85-all2	0.55	4,000	3,670	47	281	8	7
chains:v0.85-all2	0.6	4,000	3,670	47	281	8	7
chains:v0.85-all2	0.65	4,000	3,670	47	281	8	7
chains:v0.85-all2	0.7	4,000	3,670	47	281	8	7
chains:v0.85-all2	0.75	4,000	3,670	47	281	8	7
chains:v0.85-all2	0.8	4,000	3,670	47	281	8	7
chains:v0.85-all2	0.85	4,000	3,668	47	283	8	7
chains:v0.85-all2	0.9	4,000	3,546	27	425	8	7
chains:v0.85-all2	0.95	4,000	3,373	16	609	8	7
chains:v0.85-all2	1.0	4,000	3,045	8	945	8	7
chains:v0.85-all2	0.3	4,000	3,670	47	281	8	7
chains:v0.85-all2	0.35	4,000	3,670	47	281	8	7
chains:v0.85-all2	0.4	4,000	3,670	47	281	8	7
chains:v0.85-all2	0.45	4,000	3,670	47	281	8	7
chains:v0.85-all2	0.5	4,000	3,670	47	281	8	7
chains:v0.55-all2	0.55	4,000	3,878	85	20	3	2
chains:v0.55-all2	0.6	4,000	3,874	84	28	3	2
chains:v0.55-all2	0.65	4,000	3,867	77	47	3	2

Counts are endpoint outcomes at h=10. Cascade traces replay from saved identity traces; CodeMapper replay is limited to floors at or above 0.50. Gate mismatches and ratio distributions are recorded in the summary.

TABLE LVII
IDENTITY-FLOOR SENSITIVITY AT TEN HOPS, PART 3

Base	Floor	N	Correct	Wrong	Lost	Missed-last	Silent-wrong
chains:v0.55-all2	0.7	4,000	3,844	71	77	3	2
chains:v0.55-all2	0.75	4,000	3,808	67	118	3	2
chains:v0.55-all2	0.8	4,000	3,748	54	194	3	2
chains:v0.55-all2	0.85	4,000	3,668	42	288	3	2
chains:v0.55-all2	0.9	4,000	3,546	22	430	3	2
chains:v0.55-all2	0.95	4,000	3,373	11	614	3	2
chains:v0.55-all2	1.0	4,000	3,045	3	950	3	2
chains:v0.55-all2	0.3	4,000	3,878	85	20	3	2
chains:v0.55-all2	0.35	4,000	3,878	85	20	3	2
chains:v0.55-all2	0.4	4,000	3,878	85	20	3	2
chains:v0.55-all2	0.45	4,000	3,878	85	20	3	2
chains:v0.55-all2	0.5	4,000	3,878	85	20	3	2
chains:cm-seq-id	0.5	3,998	3,881	36	81	3	1
chains:cm-seq-id	0.55	3,998	3,881	36	81	3	1
chains:cm-seq-id	0.6	3,998	3,881	36	81	3	1
chains:cm-seq-id	0.65	3,998	3,881	36	81	3	1
chains:cm-seq-id	0.7	3,998	3,881	36	81	3	1
chains:cm-seq-id	0.75	3,998	3,881	36	81	3	1
chains:cm-seq-id	0.8	3,998	3,881	36	81	3	1
chains:cm-seq-id	0.85	3,998	3,881	36	81	3	1
chains:cm-seq-id	0.9	3,998	3,881	36	81	3	1
chains:cm-seq-id	0.95	3,998	3,881	36	81	3	1
chains:cm-seq-id	1.0	3,998	3,881	36	81	3	1
chains-agi:v0.85-off	0.55	4,000	3,575	88	335	8	8

Counts are endpoint outcomes at h=10. Cascade traces replay from saved identity traces; CodeMapper replay is limited to floors at or above 0.50. Gate mismatches and ratio distributions are recorded in the summary.

TABLE LVIII
IDENTITY-FLOOR SENSITIVITY AT TEN HOPS, PART 4

Base	Floor	N	Correct	Wrong	Lost	Missed-last	Silent-wrong
chains-agi:v0.85-off	0.6	4,000	3,575	88	335	8	8
chains-agi:v0.85-off	0.65	4,000	3,575	88	335	8	8
chains-agi:v0.85-off	0.7	4,000	3,575	88	335	8	8
chains-agi:v0.85-off	0.75	4,000	3,575	88	335	8	8
chains-agi:v0.85-off	0.8	4,000	3,575	88	335	8	8
chains-agi:v0.85-off	0.85	4,000	3,573	88	337	8	8
chains-agi:v0.85-off	0.9	4,000	3,457	59	482	8	8
chains-agi:v0.85-off	0.95	4,000	3,314	34	650	8	8
chains-agi:v0.85-off	1.0	4,000	3,045	8	945	8	8
chains-agi:v0.85-off	0.3	4,000	3,575	88	335	8	8
chains-agi:v0.85-off	0.35	4,000	3,575	88	335	8	8
chains-agi:v0.85-off	0.4	4,000	3,575	88	335	8	8
chains-agi:v0.85-off	0.45	4,000	3,575	88	335	8	8
chains-agi:v0.85-off	0.5	4,000	3,575	88	335	8	8
chains-agi:v0.85-alli	0.55	4,000	3,670	47	281	8	7
chains-agi:v0.85-alli	0.6	4,000	3,670	47	281	8	7
chains-agi:v0.85-alli	0.65	4,000	3,670	47	281	8	7
chains-agi:v0.85-alli	0.7	4,000	3,670	47	281	8	7
chains-agi:v0.85-alli	0.75	4,000	3,670	47	281	8	7
chains-agi:v0.85-alli	0.8	4,000	3,670	47	281	8	7
chains-agi:v0.85-alli	0.85	4,000	3,668	47	283	8	7
chains-agi:v0.85-alli	0.9	4,000	3,546	27	425	8	7
chains-agi:v0.85-alli	0.95	4,000	3,373	16	609	8	7
chains-agi:v0.85-alli	1.0	4,000	3,045	8	945	8	7

Counts are endpoint outcomes at h=10. Cascade traces replay from saved identity traces; CodeMapper replay is limited to floors at or above 0.50. Gate mismatches and ratio distributions are recorded in the summary.

TABLE LIX
IDENTITY-FLOOR SENSITIVITY AT TEN HOPS, PART 5

Base	Floor	N	Correct	Wrong	Lost	Missed-last	Silent-wrong
chains-agi:v0.85-alli	0.3	4,000	3,670	47	281	8	7
chains-agi:v0.85-alli	0.35	4,000	3,670	47	281	8	7
chains-agi:v0.85-alli	0.4	4,000	3,670	47	281	8	7
chains-agi:v0.85-alli	0.45	4,000	3,670	47	281	8	7
chains-agi:v0.85-alli	0.5	4,000	3,670	47	281	8	7
chains-agi:v0.55-alli	0.55	4,000	3,878	85	20	3	2
chains-agi:v0.55-alli	0.6	4,000	3,874	84	28	3	2
chains-agi:v0.55-alli	0.65	4,000	3,867	77	47	3	2
chains-agi:v0.55-alli	0.7	4,000	3,844	71	77	3	2
chains-agi:v0.55-alli	0.75	4,000	3,808	67	118	3	2
chains-agi:v0.55-alli	0.8	4,000	3,748	54	194	3	2
chains-agi:v0.55-alli	0.85	4,000	3,668	42	288	3	2
chains-agi:v0.55-alli	0.9	4,000	3,546	22	430	3	2
chains-agi:v0.55-alli	0.95	4,000	3,373	11	614	3	2
chains-agi:v0.55-alli	1.0	4,000	3,045	3	950	3	2
chains-agi:v0.55-alli	0.3	4,000	3,878	85	20	3	2
chains-agi:v0.55-alli	0.35	4,000	3,878	85	20	3	2
chains-agi:v0.55-alli	0.4	4,000	3,878	85	20	3	2
chains-agi:v0.55-alli	0.45	4,000	3,878	85	20	3	2
chains-agi:v0.55-alli	0.5	4,000	3,878	85	20	3	2
chains-agi:cm-seq-id	0.5	3,998	3,881	36	81	3	1
chains-agi:cm-seq-id	0.55	3,998	3,881	36	81	3	1
chains-agi:cm-seq-id	0.6	3,998	3,881	36	81	3	1
chains-agi:cm-seq-id	0.65	3,998	3,881	36	81	3	1

Counts are endpoint outcomes at h=10. Cascade traces replay from saved identity traces; CodeMapper replay is limited to floors at or above 0.50. Gate mismatches and ratio distributions are recorded in the summary.

TABLE LX
IDENTITY-FLOOR SENSITIVITY AT TEN HOPS, PART 6

Base	Floor	N	Correct	Wrong	Lost	Missed-last	Silent-wrong
chains-agi:cm-seq-id	0.7	3,998	3,881	36	81	3	1
chains-agi:cm-seq-id	0.75	3,998	3,881	36	81	3	1
chains-agi:cm-seq-id	0.8	3,998	3,881	36	81	3	1
chains-agi:cm-seq-id	0.85	3,998	3,881	36	81	3	1
chains-agi:cm-seq-id	0.9	3,998	3,881	36	81	3	1
chains-agi:cm-seq-id	0.95	3,998	3,881	36	81	3	1
chains-agi:cm-seq-id	1.0	3,998	3,881	36	81	3	1
chains-agi:cm-seq-gi-id	0.5	3,998	3,881	36	81	3	1
chains-agi:cm-seq-gi-id	0.55	3,998	3,881	36	81	3	1
chains-agi:cm-seq-gi-id	0.6	3,998	3,881	36	81	3	1
chains-agi:cm-seq-gi-id	0.65	3,998	3,881	36	81	3	1
chains-agi:cm-seq-gi-id	0.7	3,998	3,881	36	81	3	1
chains-agi:cm-seq-gi-id	0.75	3,998	3,881	36	81	3	1
chains-agi:cm-seq-gi-id	0.8	3,998	3,881	36	81	3	1
chains-agi:cm-seq-gi-id	0.85	3,998	3,881	36	81	3	1
chains-agi:cm-seq-gi-id	0.9	3,998	3,881	36	81	3	1
chains-agi:cm-seq-gi-id	0.95	3,998	3,881	36	81	3	1
chains-agi:cm-seq-gi-id	1.0	3,998	3,881	36	81	3	1

Counts are endpoint outcomes at h=10. Cascade traces replay from saved identity traces; CodeMapper replay is limited to floors at or above 0.50. Gate mismatches and ratio distributions are recorded in the summary.

TABLE LXI
MINIMUM-COST POLICY BY POPULATION AND WEIGHTS

Population	w1_m1	w1_m3	w3_m1	w3_m3	w10_m1	w10_m3	w30_m1	w30_m3
fresh	hybrid+AG2 C=0.316	hybrid+quote C=0.319	tracker+IG+AG C=0.511	tracker+IG C=0.585	codemapper+IG+AG C=0.785	tracker+IG+AG C=0.887	codemapper+IG+AG C=1.530	codemapper+IG+AG C=1.702
fresh:duplicates	codemapper+IG C=0.754	tracker+quote t0.55-IG	codemapper+AG t0.55-IG	line_number t0.55-IG	line_number t0.55-IG	line_number t0.55-IG	line_number t0.55-IG	line_number t0.55-IG
fresh:indentation	C=0.125 hybrid+AG2	C=0.125 hybrid+quote	C=0.125 hybrid+AG2	C=0.125 hybrid+AG2	C=0.125 hybrid+AG2	C=0.125 hybrid+AG2	C=0.125 hybrid+AG2	C=0.125 hybrid+AG2
fresh:short	C=0.164 hybrid+AG2	C=0.167 hybrid+quote	C=0.165 hybrid+AG2	C=0.197 hybrid+AG2	C=0.167 hybrid+AG2	C=0.199 hybrid+AG2	C=0.175 hybrid+AG2	C=0.207 hybrid+AG2
controlled:replication	C=0.009 cmhybrid+quote	C=0.011 hybrid+quote	C=0.009 cmhybrid+IG+AG2	C=0.013 hybrid+IG	C=0.009 cmhybrid+IG+AGi	C=0.013 hybrid+IG+AG2	C=0.009 cmhybrid+IG+AGi	C=0.013 gumtree+IG+AGi
real	C=0.020 hybrid+quote	C=0.049 hybrid+quote	C=0.023 hybrid+AG2	C=0.053 hybrid+quote	C=0.026 cmhybrid+IG+AGi	C=0.055 tracker_adapter	C=0.029 cmhybrid+IG+AGi	C=0.057 tracker_adapter
students	C=0.042 cmhybrid+IG+AG2	C=0.065 hybrid+IG	C=0.067 cmhybrid+IG+AG2	C=0.095 hybrid+AG	C=0.099 codemapper+AG	C=0.127 hybrid+AG	C=0.132 codemapper+AG	C=0.183 hybrid+AG
external:human-AB	C=0.230 hybrid+quote	C=0.355 hybrid+quote	C=0.260 hybrid+AG2	C=0.390 hybrid+quote	C=0.305 hybrid+IG+AG	C=0.425 hybrid+IG+AG	C=0.405 hybrid+IG+AG	C=0.525 hybrid+IG+AG
fresh:compound:k=4	C=0.597 llm+verdict	C=0.603 llm+verdict	C=0.607 cmhybrid+IG+AG2	C=0.617 llm+IG+AGi	C=0.617 codemapper+AG	C=0.630 hybrid+AG	C=0.617 codemapper+AG	C=0.630 hybrid+AG
wpj:beam:human-AB	C=0.225 cmhybrid+quote	C=0.285 llm+quote	C=0.260 cmhybrid+IG+AG2	C=0.380 llm+IG	C=0.305 cmhybrid+IG+AGi	C=0.425 llm+IG+AGi	C=0.405 cmhybrid+IG+AGi	C=0.525 llm+IG+AGi
wpj:beam:real-oss-targeted	C=0.075 hybrid+quote	C=0.154 hybrid+quote	C=0.087 hybrid+AG2	C=0.167 hybrid+quote	C=0.091 hybrid+AGi	C=0.174 hybrid+AGi	C=0.091 cmhybrid+IG+AGi	C=0.174 hybrid+IG+AGi
wpj:beam:students-targeted	C=0.110 cmhybrid+IG+AG2	C=0.186 llm+IG	C=0.142 cmhybrid+IG+AG2	C=0.226 llm+IG+AGi	C=0.172 codemapper+AG	C=0.266 hybrid+AG	C=0.219 codemapper+AG	C=0.320 hybrid+AG
wpj:nemotron:human-AB	C=0.230 cmhybrid+quote	C=0.295 llm-self	C=0.260 llm+IG	C=0.365 llm+IG	C=0.305 llm+IG+AGi	C=0.425 llm+IG+AGi	C=0.405 llm+IG+AGi	C=0.525 llm+IG+AGi
wpj:nemotron:real-oss-targeted	C=0.075 hybrid+quote	C=0.117 hybrid+quote	C=0.085 hybrid+AG2	C=0.132 hybrid+quote	C=0.091 hybrid+AGi	C=0.140 llm+AGi	C=0.091 llm+IG+AGi	C=0.140 llm+IG+AGi
wpj:nemotron:students-targeted	C=0.110	C=0.186	C=0.142	C=0.226	C=0.172	C=0.264	C=0.213	C=0.286

Each cell gives the minimizing configuration and its C per carried target. The complete Pareto sets and all six outcomes are in the K1 JSON. Ties follow the frozen protocol. C is descriptive.

TABLE LXII
LEAVE-ONE-PROJECT-OUT SELECTION AT W=3 AND M=3

Source	Held-out group	Selected	C	Oracle regret	Deployed regret	CodeMapper+IG+AGi regret
real	click	cmhybrid+quote	0.062	0.026	-0.031	-0.001
real	flask	cmhybrid+quote	0.079	0.023	-0.086	-0.006
real	httpx	hybrid+IG	0.065	0.024	-0.041	-0.013
real	p3	hybrid+AG2	0.068	0.009	-0.102	-0.017
real	p4	hybrid+IG	0.049	0.032	-0.149	0.026
real	requests	gumtree+IG	0.060	0.044	-0.002	0.028
real	rich	hybrid+IG	0.064	0.023	-0.055	0.019
real	tracer	hybrid+IG	0.011	0.011	-0.149	0.011
real	vcap	cmhybrid+quote	0.096	0.050	-0.068	-0.004
students	S1	hybrid+quote	0.038	0.011	-0.086	-0.002
students	S10	hybrid+quote	0.142	0.094	0.031	-0.094
students	S11	hybrid+quote	0.375	0.375	0.333	0.042
students	S12	hybrid+quote	0.469	0.069	0.046	-0.194
students	S13	hybrid+quote	0.513	0.294	-0.086	-0.289
students	S14	hybrid+quote	0.091	0.013	-0.026	-0.089
students	S15	hybrid+quote	0.000	0.000	0.000	0.000
students	S16	hybrid+quote	0.042	0.000	-0.096	-0.117
students	S17	hybrid+quote	0.117	0.064	-0.154	-0.021
students	S18	hybrid+quote	0.067	0.067	0.000	-0.382
students	S19	hybrid+quote	0.225	0.050	-0.003	0.013
students	S2	hybrid+quote	0.053	0.000	-0.050	-0.009
students	S20	hybrid+quote	0.112	0.000	-0.007	-0.428
students	S3	hybrid+quote	0.091	0.019	-0.038	0.005
students	S4	hybrid+quote	0.010	0.000	-0.139	-0.061
students	S5	hybrid+quote	0.088	0.022	-0.110	-0.044
students	S6	hybrid+quote	0.000	0.000	-0.017	-0.200
students	S7	hybrid+quote	0.090	0.003	-0.164	-0.130
students	S8	hybrid+quote	0.000	0.000	0.000	-0.333
students	S9	hybrid+quote	0.099	0.000	-0.028	-0.250

All six held-out outcomes for every weight pair are in the K1 JSON. Negative fixed-policy regret means the selected policy had lower held-out cost. Real history has nine project folds; students have twenty repository folds. Retrospective and descriptive.

TABLE LXIII
BLINDED MODEL ADJUDICATION OF THE SYNTAX-TREE REFERENCE

		Status (%)		Start ±2 (%)		Suspects	
Stratum	n	Beam	Nem.	Beam	Nem.	all	in file
Same-type pairing	50	94.0	98.0	100.0	100.0	1	1/50
Deleted or relocated	50	32.0	40.0	75.0	85.7	30	10/30
Methods disagree	50	52.0	56.0	89.7	96.4	21	4/33
Deployed misses	37	73.0	75.7	90.0	96.7	8	8/37
Aligned controls	50	98.0	98.0	100.0	98.0	1	1/50
All decided	237	69.6	73.4	95.2	97.6	61	24/200

Agreement of two hosted language models, not people, with the reference. Each model saw the old excerpt with the reviewed lines marked and the paired new file, blind to the reference and every method. Status: unchanged, edited or deleted. Start ±2: inside the paired file, over items the model places. Suspects: items on which both models contradict the reference status; in file: excluding reference locations in another file, which the input cannot show. Pooled κ with the reference: 0.51 (Beam), 0.58 (Nemotron); between the models 0.87.

TABLE LXIV
PARAMETRIC COPY SUITE: SIX OUTCOMES, MISSED RE-CHECKS BY THE NEAREST COPY, AND TWO STRESS CONDITIONS (%)

	All held-out trials						Miss, nearest copy				Extra	Miss	Miss
Method	Place	Miss	Silent	Misdir	Lost	Extra	adj.	$\leq$20	$>$50	file	shifted	same rev.	issues
Cascade, deployed	73.9	82.1	42.9	0.2	0.0	0.0	100.0	100.0	100.0	48.8	0.0	87.2	74.4
Cascade+IG+AG+VW	73.9	8.5	4.2	38.9	0.0	50.0	36.0	0.0	0.0	0.0	100.0	34.1	1.2
Cascade+IG+AGi+VW	76.5	8.5	4.2	37.0	0.0	0.0	36.0	0.0	0.0	0.0	0.0	40.1	2.5
CodeMapper+quote	100.0	4.1	2.1	1.6	0.0	0.0	17.4	0.2	0.0	0.0	0.1	49.6	1.4
+IG+AG	100.0	3.3	1.7	2.1	0.0	50.0	14.1	0.0	0.0	0.0	100.0	21.5	0.5
+IG+AGi	100.0	3.3	1.7	2.1	0.0	0.0	14.1	0.0	0.0	0.0	0.1	26.5	1.1

6,800 constructed trials from 116 source files: 0, 1, 2, 4 or 8 identical copies of the anchored lines before the revision, the nearest adjacent, within 20 lines, more than 50 lines away or in another file, anchors of 1–8 lines, and the original unchanged, shifted by comment lines inserted directly above, token-edited or deleted; 20 trials per cell, every file parses. Nearest-copy columns: trials with at least one copy. Extra, shifted: unnecessary requests on shifted originals. Same rev.: 800 trials in which the revision edits or deletes the original and adds one copy. Issues: 2,295 issues of 2–4 anchors, missed when an issue with a changed anchor raises no request.

TABLE LXV
SUGGESTION-APPLICATION ORACLE: SIX OUTCOMES (%)

Method	Place	Miss	Silent	Misdir	Lost	Extra
GitHub outdating	36.7	16.2	3.2	0.0	59.8	12.2
Cascade, deployed	78.9	3.1	0.0	2.2	19.5	0.0
Cascade+IG+AGi+VW	80.5	3.1	0.0	1.9	18.2	4.2
CodeMapper+quote	91.1	2.7	0.0	5.1	5.6	4.2
+IG+AG	91.1	1.1	0.0	5.1	5.6	9.5
+IG+AGi	91.1	2.7	0.0	5.1	5.6	5.8

998 review suggestions from 10 repositories in 5 languages; scope is the file pair. Place and Lost: over the 550 applied or untouched targets (applied: start inside the suggested text). Miss: over the 735 changed targets (applied, rewritten, deleted). Extra: over the 189 untouched targets. Silent and Misdir: over all 588 scored targets. GitHub: the thread's current outdated flag. Ambiguous targets (74) are not scored.

TABLE LXVI
REVIEWANCHORBENCH: RE-CHECK OUTCOMES (%)

	Population-sampled				Hard held-out			
Method	Miss	Silent	Extra	Dup	Miss	Silent	Extra	Dup
Cascade, deployed	3.7	0.7	0.0	0.0	4.1	1.6	0.0	0.0
Cascade+IG+AGi+VW	0.3	0.0	33.1	0.0	0.2	0.0	33.4	0.0
CodeMapper+quote	2.7	0.0	3.0	0.3	2.3	0.0	2.6	0.0
+IG+AG	0.1	0.0	33.5	0.3	0.1	0.0	33.6	0.0
+IG+AGi	0.3	0.0	33.1	0.3	0.2	0.0	33.1	0.0
Benchmark[a]	0.2	0.1	2.4	100.0	0.4	0.2	3.8	100.0

Seven transformation families rebuilt to the released target hashes: 2,058 and 2,625 cases from 294 and 375 anchors. Miss: changed targets (replacement applied, moved and replaced, deleted) with no re-check request. Silent: wrong placement with no request, over all scored cases. Extra: requests on unchanged targets (identity, prefix insert, block move). Dup: request rate on duplicated pre-images. [a]The benchmark's own cascade: its released predictions, re-scored, not rerun.

TABLE LXVII
SUPERVISOR INTERVENTIONS IN TEN-COMMIT CHAINS, EVERY METHOD

Policy	Correct	Wrong	Lost	Miss, last	Wrong, silent	Req-uests	Act-ions
Cascade, deployed							
none	3,575	90	335	563	85	1,121	0
correct	3,792	54	154	528	49	724	223
accept	3,575	90	335	353	85	1,121	216
both	3,792	54	154	316	49	724	442
Cascade, deployed, +identity rule							
none	3,575	90	335	8	10	2,123	0
correct	3,800	46	154	9	11	1,636	232
accept	3,575	90	335	8	10	1,524	275
both	3,800	46	154	9	11	1,034	510
Cascade+IG+AGi+VW							
none	3,670	49	281	609	46	1,038	0
correct	3,838	30	132	592	27	725	172
accept	3,670	49	281	349	46	1,038	269
both	3,838	30	132	330	27	725	443
Cascade+IG+AGi+VW, +identity rule							
none	3,670	49	281	8	9	2,138	0
correct	3,841	27	132	5	6	1,794	175
accept	3,670	49	281	8	9	1,400	338
both	3,841	27	132	5	6	1,053	515
Cascade+IG+AGi+VW, $\theta = 0.55$							
none	3,878	102	20	858	93	588	0
correct	3,925	67	8	822	58	583	49
accept	3,878	102	20	504	93	588	366
both	3,925	67	8	468	58	583	415
Cascade+IG+AGi+VW, $\theta = 0.55$*, +identity rule*							
none	3,878	102	20	3	4	2,147	0
correct	3,939	53	8	2	3	2,045	63
accept	3,878	102	20	3	4	1,158	459
both	3,939	53	8	2	3	1,056	522
CodeMapper+IG+AGi							
none	3,909	79	12	864	70	604	0
correct	3,943	48	9	836	39	600	38
accept	3,909	79	12	497	70	604	379
both	3,943	48	9	469	39	600	417
CodeMapper+IG+AGi, +identity rule							
none	3,881	38	81	3	3	2,177	0
correct	3,937	21	42	1	1	2,068	61
accept	3,881	38	81	3	3	1,181	464
both	3,937	21	42	1	1	1,072	525

4,000 ten-commit chains decidable at commit 10. A simulated supervisor acts at commits 1, 2, 3 and 5, only on a request: correct re-anchors a wrong or lost anchor to the reference location (or stops carrying it, issue open, if the code is gone); accept marks a correctly placed changed anchor as reviewed; both applies whichever fits. Correct, wrong and lost: endpoints at commit 10. Miss, last: code changed since the last supervisor review with no request at commit 10. Wrong, silent: wrong endpoints with no request. Requests: requests seen at commits 1, 2, 3, 5 and 10. Actions: supervisor actions.

TABLE LXVIII
IMPLEMENTED AND PROPOSED REVIEW-PERSISTENCE CONTRACTS

Gap	Implemented	Proposed rule	Invariant	Impl.	Prop.
Deletion (faculty)	Deleted item excluded from carry	Faculty withdrawal leaves visible tombstone	S5, S5-vis	1	0
Deletion (assistant)	Assistant deletion stops carry	Assistant proposal; faculty applies withdrawal	S4c, S5, S5-vis	1	0
Restoration across versions	Undo restores prior snapshot only	Recompose descendants after undo	Restoration, S5	1	0
Dismissal key collision	Category plus first paragraph key conflates anchors	Category + normalized title + reviewed-quotation digest	Dismissal identity, S4b	2	0
Dismissal lapse after absent version	Only immediately previous version is checked	Compare dismissals across all earlier versions	Dismissal memory, S5	1	0
Dismissed finding created closed by system	Matching new finding is created dismissed	Create open with prior-dismissal annotation	S4b	1	0
Uncomposed intermediate version	v3 reads only v2 and skips v1 issues	Compose latest ancestor; later v2 recomposes descendants	S5	1	0
System write on closed issue (evidence replaced)	System replaces evidence and retains closed state	Evidence replacement never writes a closed state	S4a, S4b	0	0

REFERENCES

[1] D. Carless and D. Boud, "The development of student feedback literacy: Enabling uptake of feedback," *Assess. Eval. High. Educ.*, vol. 43, no. 8, pp. 1315–1325, 2018. doi: https://doi.org/10.1080/02602938.2018.1463354.

[2] N. E. Winstone, R. A. Nash, M. Parker, and J. Rowntree, "Supporting learners' agentic engagement with feedback: A systematic review and a taxonomy of recipience processes," *Educ. Psychol.*, vol. 52, no. 1, pp. 17–37, 2017. doi: https://doi.org/10.1080/00461520.2016.1207538.

[3] A. J. B. Brush, D. Bargeron, A. Gupta, and J. J. Cadiz, "Robust annotation positioning in digital documents," in *Proc. SIGCHI Conf. Human Factors in Computing Systems (CHI)*, Seattle, WA, USA, 2001, pp. 285–292. doi: https://doi.org/10.1145/365024.365117.

[4] R. Sanderson, P. Ciccarese, and B. Young, "Web annotation data model," W3C Recommendation, Feb. 2017. [Online]. Available: https://www.w3.org/TR/annotation-model/

[5] M. Asaduzzaman, C. K. Roy, K. A. Schneider, and M. Di Penta, "LHDiff: A language-independent hybrid approach for tracking source code lines," in *Proc. IEEE Int. Conf. Software Maintenance (ICSM)*, Eindhoven, The Netherlands, 2013, pp. 230–239. doi: https://doi.org/10.1109/ICSM.2013.34.

[6] J.-R. Falleri, F. Morandat, X. Blanc, M. Martinez, and M. Monperrus, "Fine-grained and accurate source code differencing," in *Proc. 29th ACM/IEEE Int. Conf. Automated Software Engineering (ASE)*, Västerås, Sweden, 2014, pp. 313–324. doi: https://doi.org/10.1145/2642937.2642982.

[7] S. P. Reiss, "Tracking source locations," in *Proc. 30th Int. Conf. Software Engineering (ICSE)*, Leipzig, Germany, 2008, pp. 11–20. doi: https://doi.org/10.1145/1368088.1368091.

[8] H. Hu and M. Pradel, "CodeMapper: A language-agnostic approach to mapping code regions across commits," in *Proc. IEEE/ACM 48th Int. Conf. Softw. Eng. (ICSE)*, 2026, pp. 2287–2298, doi: 10.1145/3744916.3773267.

[9] Gerrit Code Review, "Porting Comments User Documentation," accessed Oct. 28, 2026. [Online]. Available: https://gerrit-review.googlesource.com/Documentation/user-porting-comments.html

[10] GitHub, "Precision review comment outdating," GitHub changelog, Sep. 2018, Accessed: Oct. 27, 2026. [Online]. Available: https://github.blog/changelog/2018-09-25-precision-review-comment-outdating/

[11] GitHub, "GraphQL API: Pull requests (`PullRequestReviewThread.isOutdated`)," GitHub Docs. Accessed: Oct. 31, 2026. [Online]. Available: https://docs.github.com/en/graphql/reference/pulls

[12] T. A. Phelps and R. Wilensky, "Robust intra-document locations," *Comput. Netw.*, vol. 33, no. 1–6, pp. 105–118, Jun. 2000. doi: https://doi.org/10.1016/S1389-1286(00)00043-8.

[13] K. Csillag, "Fuzzy anchoring," Hypothesis blog, Apr. 2013, Accessed: Oct. 27, 2026. [Online]. Available: https://web.hypothes.is/blog/fuzzy-anchoring/

[14] A. Horvath, A. Macvean, and B. A. Myers, "Meta-Manager: A tool for collecting and exploring meta information about code," in *Proc. CHI Conf. Human Factors Comput. Syst.*, 2024, doi: 10.1145/3613904.3642676.

[15] E. Misback, Z. Tatlock, and S. L. Tanimoto, "Magic Markup: Maintaining document-external markup with an LLM," in *Companion Proc. 8th Int. Conf. Art, Sci., Eng. Program. (⟨Programming⟩)*, 2024, pp. 22–31, doi: 10.1145/3660829.3660836.

[16] A. Bacchelli and C. Bird, "Expectations, outcomes, and challenges of modern code review," in *Proc. 35th Int. Conf. Software Engineering (ICSE)*, San Francisco, CA, USA, 2013, pp. 712–721. doi: https://doi.org/10.1109/ICSE.2013.6606617.

[17] C. Sadowski, E. Söderberg, L. Church, M. Sipko, and A. Bacchelli, "Modern code review: A case study at Google," in *Proc. 40th Int. Conf. Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, Gothenburg, Sweden, 2018, pp. 181–190. doi: https://doi.org/10.1145/3183519.3183525.

[18] GitLab, "Merge requests: Automatically resolve threads when they become outdated," GitLab Documentation. Accessed: Oct. 31, 2026. [Online]. Available: https://docs.gitlab.com/user/project/merge_requests/

[19] H. Y. Lin, C. Liu, H. Gao, P. Thongtanunam, and C. Treude, "CodeReviewQA: The code review comprehension assessment for large language models," in *Findings Assoc. Comput. Linguist.: ACL 2025*, 2025, pp. 9138–9166, doi: 10.18653/v1/2025.findings-acl.476.

[20] P. Avgustinov, A. I. Baars, A. S. Henriksen, G. Lavender, G. Menzel, O. de Moor, M. Schäfer, and J. Tibble, "Tracking static analysis violations over time to capture developer characteristics," in *Proc. 37th IEEE/ACM Int. Conf. Software Engineering (ICSE)*, vol. 1, 2015, pp. 437–447. doi: https://doi.org/10.1109/ICSE.2015.62.

[21] J. Li and J. Yang, "Tracking the evolution of static code warnings: The state-of-the-art and a better approach," *IEEE Trans. Softw. Eng.*, vol. 50, no. 3, pp. 534–550, 2024. doi: https://doi.org/10.1109/TSE.2024.3358283.

[22] J. Spacco, D. Hovemeyer, and W. Pugh, "Tracking defect warnings across versions," in *Proc. Int. Workshop Mining Software Repositories (MSR)*, Shanghai, China, 2006, pp. 133–136. doi: https://doi.org/10.1145/1137983.1138014.

[23] SonarSource, "SonarQube documentation: Issue tracking across analyses," Accessed: Oct. 27, 2026. [Online]. Available: https://docs.sonarsource.com/sonarqube-server/8.9/user-guide/issues

[24] P. Yu, Y. Wu, X. Peng, J. Peng, J. Zhang, P. Xie, and W. Zhao, "ViolationTracker: Building precise histories for static analysis violations," in *Proc. IEEE/ACM 45th Int. Conf. Softw. Eng. (ICSE)*, 2023, pp. 2022–2034, doi: 10.1109/ICSE48619.2023.00171.

[25] E. W. Myers, "An $O(ND)$ difference algorithm and its variations," *Algorithmica*, vol. 1, pp. 251–266, 1986. doi: https://doi.org/10.1007/BF01840446.

[26] J. W. Ratcliff and D. E. Metzener, "Pattern matching: The Gestalt approach," *Dr. Dobb's J.*, Jul. 1988. [Online]. Original article mirror: https://jacobfilipp.com/DrDobbs/articles/DDJ/1988/8807/8807c/8807c.htm

[27] Python Software Foundation, "difflib—Helpers for computing deltas," Python 3.12 documentation, accessed Oct. 28, 2026. [Online]. Available: https://docs.python.org/3.12/library/difflib.html

[28] G. Canfora, L. Cerulo, and M. Di Penta, "Identifying changed source code lines from version repositories," in *Proc. Int. Workshop Mining Software Repositories (MSR)*, Minneapolis, MN, USA, 2007, p. 14. doi: https://doi.org/10.1109/MSR.2007.14.

[29] M. Kim and D. Notkin, "Program element matching for multi-version program analyses," in *Proc. Int. Workshop Mining Software Repositories (MSR)*, Shanghai, China, 2006, pp. 58–64. doi: https://doi.org/10.1145/1137983.1137999.

[30] B. Fluri, M. Würsch, M. Pinzger, and H. C. Gall, "Change distilling: Tree differencing for fine-grained source code change extraction," *IEEE Trans. Softw. Eng.*, vol. 33, no. 11, pp. 725–743, Nov. 2007. doi: https://doi.org/10.1109/TSE.2007.70731.

[31] P. Alikhanifard and N. Tsantalis, "A novel refactoring and semantic aware abstract syntax tree differencing tool and a benchmark for evaluating the accuracy of diff tools," *ACM Trans. Softw. Eng. Methodol.*, vol. 34, no. 2, pp. 1–63, 2025, doi: 10.1145/3696002.

[32] M. W. Godfrey and L. Zou, "Using origin analysis to detect merging and splitting of source code entities," *IEEE Trans. Softw. Eng.*, vol. 31, no. 2, pp. 166–181, Feb. 2005. doi: https://doi.org/10.1109/TSE.2005.28.

[33] F. Grund, S. A. Chowdhury, N. C. Bradley, B. Hall, and R. Holmes, "CodeShovel: Constructing method-level source code histories," in *Proc. IEEE/ACM 43rd Int. Conf. Softw. Eng. (ICSE)*, 2021, pp. 1510–1522, doi: 10.1109/ICSE43902.2021.00135.

[34] M. Jodavi and N. Tsantalis, "Accurate method and variable tracking in commit history," in *Proc. 30th ACM Joint Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng. (ESEC/FSE)*, 2022, pp. 183–195, doi: 10.1145/3540250.3549079.

[35] M. T. Hasan, N. Tsantalis, and P. Alikhanifard, "Refactoring-aware block tracking in commit history," *IEEE Trans. Softw. Eng.*, vol. 50, no. 12, pp. 3330–3350, Dec. 2024, doi: 10.1109/TSE.2024.3484586.

[36] S. Islam, A. Aowal, M. S. Uddin, and S. Chowdhury, "HistoryFinder: Advancing method-level source code history generation with accurate oracles and enhanced algorithm," in *Proc. IEEE/ACM 48th Int. Conf. Softw. Eng. (ICSE)*, 2026, pp. 2262–2273, doi: 10.1145/3744916.3773238.

[37] K. Sun, Y. Lyu, X. Ma, H. Kuang, R. Widyasari, H. Zhang, X. Ma, J. Lawall, and D. Lo, "We must have missed this comment: Detecting and repairing stale function references in Linux kernel comments," in *Proc. 41st IEEE/ACM Int. Conf. Autom. Softw. Eng. (ASE)*, 2026, pp. 2254–2266, doi: 10.1145/3832783.3837474.

[38] J. Fu, "ReviewAnchorBench: Executable benchmark and replication package," version 1.1.0, Zenodo, 2026, dataset, doi: 10.5281/zenodo.22217751.

[39] J. Hattie and H. Timperley, "The power of feedback," *Rev. Educ. Res.*, vol. 77, no. 1, pp. 81–112, 2007. doi: https://doi.org/10.3102/003465430298487.

[40] V. J. Shute, "Focus on formative feedback," *Rev. Educ. Res.*, vol. 78, no. 1, pp. 153–189, 2008. doi: https://doi.org/10.3102/0034654307313795.

[41] H. Keuning, J. Jeuring, and B. Heeren, "A systematic literature review of automated feedback generation for programming exercises," *ACM Trans. Comput. Educ.*, vol. 19, no. 1, art. 3, 2018. doi: https://doi.org/10.1145/3231711.

[42] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *J. Amer. Statist. Assoc.*, vol. 22, no. 158, pp. 209–212, 1927. doi: https://doi.org/10.1080/01621459.1927.10502953.

[43] Q. McNemar, “Note on the sampling error of the difference between correlated proportions or percentages,” *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947. doi: https://doi.org/10.1007/BF02295996.

[44] Oselvar, “lhdiff: A lightweight hybrid approach for tracking source lines,” GitHub repository, revision 5a9f7fd87193098cf90f7ee4081f6d95f04d4cc7, accessed Oct. 28, 2026. [Online]. Available: https://github.com/oselvar/lhdiff

[45] Pallets Projects, “Click: tests/test_chain.py,” revision 0dc6d74df5c984ff923f5d17e97bf94a21b3a3cd, compared with e3594e7f8a76dd15109d29570a6f263e56e412f2, accessed Oct. 28, 2026. [Online]. Available: https://github.com/pallets/click/commit/0dc6d74df5c984ff923f5d17e97bf94a21b3a3cd

[46] Pallets Projects, “Flask: tests/test_cli.py,” revisions 12e95c93b488725f80753f34b2e0d24838ca4646 and a411a2434b09d9cbaa4074232b2c247d591bb655, accessed Oct. 28, 2026. [Online]. Available: https://github.com/pallets/flask/blob/a411a2434b09d9cbaa4074232b2c247d591bb655/tests/test_cli.py

[47] OpenAI, “Codex,” AI-assisted research-code and manuscript preparation, Oct. 28, 2026. [Online]. Available: https://openai.com/codex/